P1787R6: Declarations and where to find them

Audience: CWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
October 23, 2020

History

Since r5:

- Made implicit header-unit export override internal linkage for “precedes”
- Disallowed redeclarations of internal-linkage entities from header units
- Specified anonymous union member collisions in terms of correspondence
- Applied “reachable” to more cases that assumed a total order
- Allowed `decltype(f)` if `f` is made unique by constraints
- Defined “primary template” generically
- Required equivalent template argument lists for all references to partial specializations, resolving CWG2065
- Generalized instantiation wording for partial specializations
- Fixed nesting of template parameter scopes
- Fixed list of declaration types
- Fixed default argument availability through a *using-declaration*
- Fixed explicit specialization declarations of classes
- Fixed correspondence check for redeclarations in *template-declaration*s
- Suppressed lookup for *type-parameter* declarations
- Specified lookup before < even in a *declarator-id*
- Required destructor declarations to use the injected-class-name, avoiding name lookup
- Simplified lookup for destructors
- Specified that < begins a template argument list in a destructor name
- Removed transformation of *unqualified-id*s that refer to static members
- Required *operator-function-id*s be used only for functions
- Generalized “first declaration” lookup for modules
- Fixed module attachment of friend declarations
- Limited lookup around a friend to one scope, leaving CWG1906 closed and replacing the resolution for CWG2370
- Clarified definition of “naming class”, resolving CWG952
- Generalized *using-declaration* access control check, resolving CWG360
- Consistently described access control in terms of naming members, not their names
- Consistently specified overload resolution even for singleton sets
- Removed “overloaded function [template]” term
- Removed “global function” term
- Removed “in scope” term, simplifying the description of local variable lifetime
- Removed “scope” for macro parameters
- Removed residual “looked up in” usage
- Removed redundant normative wording for `using B::operator=;`
- Removed redundant prohibition of `this` in a declarator
- Removed redundant wording for anonymous union linkage
- Removed blanket statement of namespace scope in the library
- Clarified wording for introduction of type names
- Clarified that inline class members are always defined
- Added excerpts from removed paragraphs
- Made wording sections #[searchable]

Since r4:

- Rebased onto N4861 (significant for `boolean-testable`)
- Allowed *using-declaration*s to refer to non-dependent conversion function templates
- Limited hiding among conversion functions and dependent conversion function templates
- Clarified and simplified conversion function template argument deduction
- Allowed lookup in reachable class definitions
- Excluded internal-linkage namespaces from “precedes” generally
- Allowed *using-directive*s to operate across translation units (though exported only in header units)
- Restored most *using-directive* wording as notes
- Gave namespaces a locus
- Clarified the status of the global namespace
- Clarified the lookup for a namespace to extend
- Clarified when declarations refer to the same entity, resolving CWG279, CWG338, CWG1884, CWG1898, CWG1252, CWG2062, and CWG1896
- Preserved function-type language linkage in redeclarations
- Restored language linkage for class members (with external linkage)
- Restored explicit specialization/instantiation declarations for member functions
- Restored initialization of an object via conversion to rvalue reference
- Specified choice between `operator delete` and `operator delete[]` for a *delete-expression*
- Addressed non-*init-declarator* declarators and non-*identifier declarator-id*s

- Defined declaration matching and lookup for *nested-name-specifier*s in redeclarations and specializations
- Removed type-only lookup for …`::template A<0>::`
- Removed special name treatment for `friend struct S* f();`
- Allowed definitions of nested (local) types at block scope
- Introduced “component name” to avoid recursing into template arguments or *decltype-specifier*s, replacing “appears as a type”
- Simplified “name” to be nothing more than one of the relevant grammar non-terminals
- Restricted “qualified name” to the name in question
- Removed “closed context”, unresolving CWG325
- Gave dependent class members priority over unqualified lookup, unresolving CWG1089
- Made `operator=` dependent in any templated class
- Separated “dependent call” from “dependent name”
- Fixed dependent *using-declaration*s that refer to templates
- Specified that dependent *using-declaration*s can be dependent members of the current instantiation
- Clarified template argument deduction from declarations, resolving CWG271
- Forbid use of a function with an undeduced return type without naming it
- Specified default argument conflicts via different scopes rather than a common parameter
- Reduced extent of several kinds of scopes
- Rewrote ADL in terms of searches
- Rephrased language linkage in terms of entities (properly including variables)
- Accounted for entities declared with C language linkage belonging to multiple scopes
- Expressed special *declarator-id*s as requirements
- Merged [temp.inject] into [temp.friend]
- Removed disused notion of exporting a name (as opposed to a declaration)
- Restored explicit introduction of *class-specifier* names
- Removed redundant class member function wording
- Removed definition of inherited non-constructor members
- Replaced “local variable” with “block variable”
- Replaced some “anonymous union variable”s with “anonymous union member”
- Rephrased usage of members of an anonymous union as an *id-expression* interpretation
- Allowed anonymous unions to omit `static` in any internal-linkage namespace
- Removed “-scope” adjectives in normative text
- Fixed cross reference to removed subclause

Since r3:

- Rebased onto N4849
- Forbid `decltype(…)::~…`
- Restored block scope for most compound statements
- Fixed scope of parameters of a function declaration
- Fixed list of declarations and scope introducers
- Removed “direct function lvalue” interpretation for a *template-id*
- Made scope-based rules for copy elision and implicit moves consistent
- Removed truly final “point of declaration” usage
- Clarified terminology (in particular, for enclosing scopes)
- Added forward reference note for unusual declaration targets/binding
- Removed or labeled deprecated `::template` in examples
- Labeled [class.mfct.non-static] edits correctly

Since r2:

- Rebased onto N4835
- Added definition for “reachable from”
- Restored Richard’s rule for cross-TU *using-declaration* conflicts
- Generalized consistent-linkage rule for CWG1839
- Corrected scope of function-local predefined variables
- Updated all references to removed [basic.lookup.classref], resolving CWG255
- Removed (remainder of) [class.this]
- Removed “global object” term

Since r1:

- Rebased onto N4830 (significant for `using` enumerators/enumerations)
- Added a locus for a *concept-definition*
- Extended containing scopes into redeclarations of classes and enumerations
- Properly limited cross-TU qualified lookup in namespaces
- Fixed rules for conflicts between *using-declaration*s
- Renamed “anonymous union object” to “anonymous union variable”

Since r0:

- Fixed reversed *using-directive* containment test
- Updated **signature** definitions
- Removed residual “point of declaration” usage
- Fixed several uses of (new) “precede” to be (existing) “reachable”
- Removed dangling “considered to refer to a template” reference
- Used “be attached” for modules instead of (new, unrelated) “belong”

- Included class templates with classes for all scope purposes
- Distinguished entities from their associated scopes
- Accounted for missing kinds of declarations
- Removed superfluous definition of “denotes”
- Disentangled “enclose” and “contain” definitions
- Specified no lookup for `X` in `using X=...;`
- Allowed a *namespace-name* to be a member-qualified name
- Allowed in-class lookup of `X` in `p->~X()` and `decltype(...)::~X()`
- Ignored non-type `X` in `struct N::X x;`
- Limited `this`/`static` prohibition with “current class”
- Deprecated `A::template B` (as a template template argument or CTAD placeholder)
- Allowed forward declarations of partial specializations
- Removed mention of resolved issue
- Added explanatory examples to aid review

Introduction

The current descriptions of scope and name lookup are confusing, incomplete, and at times incorrect. [basic.scope] defines the word “scope” in two different ways, and then implicitly and indirectly specifies many aspects of name lookup with imperfect correspondence to the descriptions in [basic.lookup]. Lookup is often described for units of program text that may include multiple names and < tokens that must be disambiguated, such as a *template-id* ([temp.dep]/2), *id-expression* ([over.call.func]/2), or *conversion-type-id* ([class.qual]/1.2). Namespaces, and the global namespace in particular, are inconsistently described as being one or many declarative regions ([basic.scope.namespace]/1, /4). There are dozens of Core issues on the subject; the general vagueness of the wording on the subject interferes with resolving them individually.

Modules significantly complicate the situation, principally by invalidating the tacit assumption that “every” lookup is restricted to preceding declarations in the same translation unit. There were already exceptions to that rule (*e.g.*, complete class contexts and *using-declaration* conflicts), which can then easily be (mis)read as applying to all contents of all translation units. Many rules, like those for lookup through a *using-directive*, do not make any mention of location at all, and so become ambiguous.

The similar process of associating a declaration with a previous declaration of the same name is also poorly specified. In particular, there is no consistent rule as to whether multiple declarations of the same name introduce distinct entities (that might conflict) or one entity (and might conflict).

This paper rewrites large sections of [basic.scope] and [basic.lookup] and parts of [temp.res] (and makes the requisite changes elsewhere to adapt to new definitions) to address these issues holistically. As discussed individually below, 63 Core issues are resolved, as well as 21 points discussed on the reflector but not (yet) on the issues list. Many changes are (collectively) editorial, but their very number demands CWG review. Other changes generally formalize current implementation practice or, in the absence of implementation consensus, establish language consistency.

Approach

To avoid redundant specification of scope and of name lookup, all description of where a name might be used ([basic.scope.block], [basic.scope.enum], *etc.*) is phrased merely in terms of the total extent of each scope where a name might be introduced. Except for template parameter scopes, scopes are hierarchical, although unqualified lookups in certain friend declarators consider a scope that does not contain the point of the lookup. Explicit wording is introduced to require that lookup occur from a program point (replacing lookup “in the context” of something) and consider only previous declarations (except in complete-class contexts) and those made available by `import`. Most declarations are said to “bind” the names they introduce to themselves, replacing the notion of “not found by lookup”. To avoid confusion, the word “bind” is not used for describing locked-in lookup results as for default arguments (which can instead be described as being looked up from their declarations). The use of the word “visible” to indicate a declaration’s placement is avoided in normative text.

Several kinds of scope search are defined, and several kinds of qualified lookup are combined, to minimize redundancy in the definition of the lookup algorithms. The word “lookup” or phrase “look up” is used only for resolution of a name that is complete (at least in the simple context; it might be reused as a subroutine elsewhere). Lookup is defined for *nested-name-specifier*s in *declarator-id*s.

The descriptions of the algorithms generally favor local completeness at the expense of additional references to distant clauses (that would otherwise specify special cases for their respective language features). Several rule restatements are turned into notes.

Every lookup is explicitly for a name, not an *id-expression* or *type-id*. Template arguments in an *id-expression* are addressed by defining which names are subject to qualified lookup and by excluding a *template-id* from being a name. Because labels interact with so little of the language, their identifiers are excluded from names and name lookup entirely.

To avoid confusion when a name is declared multiple times in a scope, careful distinction is made between a name and a declaration of that name. In particular, the “point of declaration” of a name (usually; [basic.scope.pdecl]/3 describes one for an unnamed class or enumeration) is replaced with the “locus” of a declaration.

The phrases “global name” and “global object” are removed to reduce the apparent special status of the global namespace (but “global scope” still exists because of its lack of syntax and as a shorthand).

Wording for the (possible) declaration of a class by an *elaborated-type-specifier* is removed from [basic.scope.pdecl]/7, [basic.lookup.elab]/2, [namespace.memdef]/3, and [class.friend]/11 and merged into [dcl.type.elab].

Richard’s proposal for “using-declarations & {,!}export” is followed: a (non-dependent) *using-declaration* is replaced with its referent(s) during name lookup rather than introducing additional declarations. Because it is consistent with that replacement, and for consistency with existing implementations, the inclusion of hidden type names by a member *using-declaration* is extended to those not in a class.

Declarations are deemed to declare the same entity when they must do so, categorizing conflicts between, say, `namespace x {}` and `template<class> concept x=true;` as inconsistent redeclarations. The word “redeclaration” is restricted to that case (and not a declaration of an existing

name in a new scope). A procedure is specified for connecting a declaration of a qualified name or a template specialization to the original declaration of the name or template, aggregating wording scattered across [namespace.qual]/7, [basic.link]/6–7, [namespace.memdef]/2–3, [class.mfct]/1–3, [class.static.data]/3, [temp.class]/3, and [temp.class.spec.mfunc]/1. (This is not considered name lookup, partly because a mechanism other than overload resolution is used to resolve ambiguities.) Language linkage is respecified as being an attribute of an entity (or a function type), not a name.

Examples

The lookup for the following examples is narrated to help CWG members relate old and new terminology. Cross references prefixed with + are to N4861 as edited here.

using-directive vs. *using-declaration*

```
namespace F {float x;}
namespace A {
  namespace I {int x;}
  using namespace I;
  using F::x;
  void f() {
    x=0;     // #1: error
    A::x=1;  // #2: OK: F::x
  }
}
```

1. At #1, `x` is looked up as an unqualified name (+[basic.lookup.unqual]/4) via an unqualified search of the function block scope of `A::f` (/3).
2. When continuing on past the function parameter scope of `A::f` to the namespace scope of `A`, the unqualified search searches `A` and also every namespace contained by `A` and nominated by an active *using-directive* (/2).
3. The single *using-directive* is active in the block scope (/1), which is contained by `A`, as is `I`.
4. Therefore `A` and `I` are searched simultaneously.
5. The *using-declaration* is replaced with `F::x` (+[basic.lookup]/4), and the lookup is ambiguous ([basic.lookup]/1).

1. At #2, `A` is looked up as an unqualified name. `x` is a qualified name (+[basic.lookup.qual]/2), so undergoes qualified name lookup (/3) which is just a search of `A`.
2. The search finds `F::x`, so the *using-directive* is not considered by +[namespace.qual]/1.

CWG459

```
struct A {using T=int;};  // given
template<class T>
struct B : A {
  void f() {T i;}
};
```

1. `T` (in the *template-head*), `B`, and `f` are not looked up (+[dcl.meaning]/3, +[class.pre]/1) because they introduce entities. (They may be used to recognize existing declarations being redeclared or with which there is a conflict, but they are not “evaluated” to an entity.)
2. `A` is looked up as an unqualified name (+[basic.lookup.unqual]/4), ignoring non-type names ([class.derived]/2).
3. `T` is looked up as an unqualified name when defining the class template `B` (+[temp.res]/1):
    1. Unqualified name lookup performs an unqualified search in the enclosing scope (+[basic.lookup.unqual]/3), which is the function block scope of `B::f` (+[basic.scope.block]/1).
    2. For each enclosing scope, an unqualified search is just a search except when considering a namespace (+[basic.lookup.unqual]/2); a search is just a single search except when considering a class (template) (+[class.member.lookup]/1). The function block scope is neither and has no binding for `T` (and the parameter scope (+[basic.scope.param]/1) has only `__func__`).
    3. The next enclosing scope is that of `B`; classes and class templates have scopes (+[basic.scope.class]/1). It is a class template, and a single search in it finds nothing ([class.member.lookup]/4), so its non-dependent base classes are considered (+[class.member.lookup]/5).
    4. A single search in the scope of `A` finds the *typedef-name*.
    5. The next enclosing scope is the template parameter scope to which the template parameter `T` belongs ([basic.scope.temp]/2), but it is never considered.

Calling a conversion function

```
auto x = s.operator R T::*();
```

1. `x` is not looked up (+[dcl.meaning]/3).
2. `s` is subject to normal unqualified lookup (+[basic.lookup.unqual]/4).
3. The names used in the *conversion-type-id* are looked up “in the same fashion as the *conversion-function-id*” (/5).
4. While that lookup can’t happen yet, the *conversion-function-id* is a member-qualified name (+[basic.lookup.qual]/2) and thus undergoes qualified lookup in the class of `s` (call it `S`; /3).
5. If `S::R` finds nothing, `R` is looked up as an unqualified name (+[basic.lookup.unqual]/5 again); `T` is considered the same way (separately), except that non-type names are ignored because of the following `::` ([basic.lookup.qual]/1).
6. Once the type named by the *conversion-type-id* is known ([class.conv.fct]/1), the *conversion-function-id* is looked up as described above (including conversion function templates via +[class.member.lookup]/8).

Calling destructors

```
struct A {using X=A;};
struct B : A {struct X {};};
void f(B *b) {
```

```
  b->A::~X();           // error: A::X not considered
}
```

1. In `f`, `A` undergoes qualified lookup in `B` (+[basic.lookup.qual]/3).
2. Because the terminal name (+[expr.prim.id.unqual]/2) of its *nested-name-specifier* is a (member-)qualified name (+[basic.lookup.qual]/2), `X` is looked up if in `b->~X` (/4.3).
3. That lookup finds `B::X` by qualified lookup and nothing by unqualified lookup (/4.1), but not the required lookup context `A` (/4.6).

Issues

Definitions

CWG554 is resolved by using the word "scope" *instead of* "declarative region", consistent with its very common use in phrases like "namespace scope". The other uses of "scope" are implied by the explicit unqualified lookup algorithm. CWG2165 is resolved by removing the conflicting definition of it in [basic.scope].

CWG405 is resolved by stating that argument-dependent lookup (sometimes) occurs after an ordinary unqualified lookup (making statements like "finding a variable prevents argument-dependent lookup" formally correct).

CWG36 (which has the dubious distinction of having the earliest recorded date) is resolved by limiting the restriction in question to class scopes, where ambiguous accessibility would be a problem, since a *using-declaration* does not introduce declarations. CWG418 is resolved by trivial rephrasing along with that necessary for the new *using-declaration* interpretation. CWG1960 (currently closed as NAD) is resolved by removing the rule in question (which is widely ignored by implementations and gives subtle interactions between *using-declaration*s).

CWG1291 and CWG2396 are resolved by explicitly specifying how to look up names in a *conversion-type-id*.

CWG600 is resolved by explaining that accessibility affects naming a member in the sense of the ODR. CWG360 is resolved by applying access control to *using-declaration*s. CWG952 is resolved by refining the definition of "naming class" per Richard's suggestion in "CWG1621 and [class.static]/2".

My proposed regularization of *try-block* containment in "Cases missed by P1825R0 (Merged wording for P0527R1 (Implicitly move from rvalue references in return statements) and P1155R3 (More implicit moves))" is included in rewording the relevant bullets.

Lookup completeness

CWG191 and CWG1200 are resolved by defining unqualified lookup in terms of every enclosing scope. CWG536 (long partially resolved) is resolved by specifying that unqualified lookup occurs for any kind of *unqualified-id* as an *id-expression*. CWG399 is resolved by explicitly appealing to the lookup for the last component of any suitable *nested-name-specifier*. CWG562 is resolved by defining lookup as occurring from a program point.

CWG2370 is resolved by performing a search in (only) the immediate scope of any friend, per the CWG opinion from San Diego.

My proposed simplification for "Missing associated namespace for member template template arguments?" is incorporated on the assumption that the failure to examine the namespace of an associated class is an oversight. (GCC and ICC, but not Clang or MSVC, accept the example in that message.)

CWG1822 is resolved by specifying that the body of a lambda remains in the surrounding (function parameter) scope.

CWG1894 and its duplicate CWG2199 are resolved per Richard's proposal for "dr407 still leaves open questions about typedef / tag hiding", using generic conflicting-declaration rules even for `typedef`, and discarding a redundant *typedef-name* when looking up an *elaborated-type-specifier*.

The suggestion in [namespace.udir]/5 that declarations become available only after a *namespace-definition* is removed.

Jens' interpretation of "On beyond CWG372" is included as a note (*cf.* Richard's proposed restriction). CWG607 is resolved by looking up unqualified names in a *mem-initializer-id* from outside the parameter scope. CWG1837 is resolved by restricting `this` to referring to the innermost enclosing class.

CWG2007 is resolved by skipping unqualified lookup for operators that must be member functions. CWG255 (partially resolved by N3778 is resolved by generally specifying the restriction to usual deallocation functions.

Richard's correction to template parameter scope for constrained template template parameters for "CWG186 is NAD (or 'Resolved')" is incorporated. His correction to function parameter scope for non-defining declarations in "Using a function parameter in a requires clause" is applied directly.

The missing point of declaration for a concept identified by Andrew in "recursive concepts" is supplied (without any prohibition on self-referential concepts).

Complete-class context

The intent for CWG2335 (contra those of the older CWG1890, CWG1626, CWG1255, and CWG287) is supported by retaining the unrestricted forward lookup in complete-class contexts (despite current implementation behavior for non-templates). CWG361 is partially resolved by that confirmation; the rest must be addressed along with CWG2335. CWG2331 is resolved by defining lookup from complete-class contexts and out-of-line member definitions. The rest of CWG2009 is resolved by making it ill-formed NDR for forward lookup outside a complete-class context to change the results (before overload resolution, to avoid differences in instantiation). A workaround for the practical issue of CWG325 is introduced by generalizing the `template` prefix to support lookup in an incomplete class. CWG1635 is partially resolved by also making template default arguments a complete-class context.

Declaration matching

My clarification in "Where can namespace-scope functions and variables be redeclared?" is applied, resolving CWG1821. My generalization for member redeclarations in "CWG498 and class member redeclarations" is applied.

CWG1820 is resolved by requiring that a qualified *declarator-id* declare an entity. CWG386, CWG1839, CWG1818, CWG2058, CWG1900, and Richard's observation in "are non-type names ignored in a class-head-name or enum-head-name?" are resolved by describing the limited lookup that occurs for a *declarator-id*, including the changes in Richard's proposed resolution for CWG1839 (which also resolves CWG1818 and what of CWG2058 was not resolved along with CWG2059) and rejecting the example from CWG1477. Consistent linkage is preserved by strengthening [dcl.stc]/6. CWG138 is resolved according to N1229, except that *using-directive*s that nominate nested namespaces are considered.

CWG563 is resolved by simplifications that follow its suggestions.

CWG110 is resolved by reducing the restriction in [temp.pre] to a note (matching the behavior of GCC, Clang, and ICC). CWG1898 is resolved by explicitly using the defined term parameter-type-list. CWG1884, CWG279, and CWG338 are resolved by defining entity identity explicitly. CWG1252 is resolved by defining it for member and non-member functions as consistently as is appropriate. The incomplete consideration of constraints identified by Barry in "Using-declaration with constrained members ([namespace.udecl]/14)" is addressed by reusing the general function identity definition. CWG2062, CWG1896, and CWG1729 (which, as indicated, was already partially resolved) are resolved by specifying consistency requirements for all entities.

Modules

Richard's proposed resolution for "missing restriction on private-module-fragment" is incorporated. His suggestion for "modules & friends" is also applied.

Nathan's question about `export using` in "using-declarations & {,!}export" is avoided by not introducing synonyms that might or might not be exported. Richard's issue submission for *using-declaration* conflicts is resolved by combining the generic notions of "conflicts with" and "precedes". A declaration being exported is made to override its having internal linkage to give effect to the universal export in header units.

Parsing

CWG1828 is resolved by disfavoring the interpretation of `::` as an entire *nested-name-specifier*. CWG1771 is resolved by describing the lookup of only an *identifier* preceding a `::` (as in the suggested resolution for CWG1828).

My proposal for "'Down with template!'" is incorporated by generalizing "type-id-only context" to "type-only context" and applying it to `template` as well as `typename`. CWG1835 is resolved by removing the special provision for `->/.` *identifier* < in [basic.lookup.classref]/1, requiring that `template` be used to refer even to a non-member template in that position. CWG1908 is resolved per its suggestion of performing qualified lookup first and stopping if it produces a *template-id*, always taking < to indicate one (per Richard's proposal for "On beyond CWG1908"). My further proposal for parsing a *qualified-id* in a dependent class member access is applied. My proposal for "grammar ambiguities with non-simple template-ids" is applied, allowing any kind of name to be considered to refer to a template, except for a *conversion-function-id* for which the rules would be more complicated despite never applying. CWG1841 is resolved by considering the injected-class-name of a class template to refer to a template.

CWG1616 is resolved by explicitly characterizing the forbidden case in terms of lookup results.

Templates

CWG459 is not affected: base classes are still searched in unqualified lookup before template parameters are considered.

CWG1936 is resolved by expanding the definition of "dependent name" to include qualified names (*i.e.*, names previously said to be "a member of an unknown specialization"). CWG1500 is resolved by further expanding it to include any *conversion-function-id* with a dependent type (and clarifying that dependent lookup takes place after substitution). My proposed regularization of *conversion-type-id* interpretation (which includes resolving CWG2413) in "'Down with template!'" and proposed rule for considering conversion function templates in "When are conversion function templates hidden?" are incorporated. The assumption that a dependent qualified name names a type is disabled inside a `decltype` or array bound. CWG1028 is resolved by moving the "lookup from first declaration" rule from [temp.over.link]/5 to [temp.res]/1 (and generalizing it for modules). CWG1907 is resolved by forbidding dependence on default (template) arguments not found by *lookup* for a dependent name.

CWG2065 is resolved by consistently appealing to equivalence (which automatically accounts for mere functional equivalence) and specifying the interpretation of declarative *nested-name-specifier*s. CWG1829 is resolved by checking for direct members of a class that is the current instantiation. Nathan's proposal of making the name `operator=` dependent in "When are class template assignment operators declared?" is incorporated.

Prioritizing member lookup over unqualified lookup affirms that the templates in CWG1089 are not functionally equivalent, leaving the matter as a missing condition for them to not be equivalent. CWG1478 is resolved per my interpretation as independently suggested by Richard in "Implementation status of core issue 96".

CWG2070 is resolved by specifying that a dependent *using-declarator* is considered to name a constructor if and only if it ends with `B::B`. CWG852 is resolved by explicitly mentioning *using-declaration*s in class templates (and making it diagnosable in most cases to refer to a class that is definitely not a base).

CWG2213 is resolved by allowing an *elaborated-type-specifier* to contain a *simple-template-id* without `friend`.

Richard's simplification in "Deduction failures and template-ids" is applied as part of regularizing the interpretation of function names.

CWG271, long partially resolved by the addition of [temp.deduct.decl], is resolved by replacing the explicit mentions of function parameters with cross references to it.

Wording

Relative to N4861.

#[intro.defs]

#[defns.signature]

Change the definition (leaving the note):

- **signature**
  ⟨function⟩ name, parameter-type-list ([dcl.fct]), and enclosing namespace ~~(if any)~~

#[defns.signature.templ]

Change:

- **signature**
  ⟨function template⟩ name, parameter-type-list ([dcl.fct]), enclosing namespace ~~(if any)~~, return type, *template-head*, and trailing *requires-clause* ([dcl.decl]) (if any)

#[intro.compliance]

Change paragraph 5:

- ~~The names defined in the library have namespace scope ([basic.namespace]).~~ A C++ translation unit ([lex.phases]) obtains access to the~~se~~ names defined in the library by including the appropriate standard library header or importing the appropriate standard library named header unit ([using.headers]).

#[lex.ext]

Change paragraph 2:

- A *user-defined-literal* is treated as a call to a literal operator or literal operator template ([over.literal]). To determine the form of this call for a given *user-defined-literal* *L* with *ud-suffix* *X*, first let *S* be the set of declarations found by unqualified lookup for the *literal-operator-id* whose literal suffix identifier is *X* ~~is looked up in the context of *L* using the rules for unqualified name lookup~~ ([basic.lookup.unqual]). ~~Let *S* be the set of declarations found by this lookup.~~ *S* shall not be empty.

#[basic]

#[basic.pre]

Change paragraph 4:

- A *name* is ~~a use of~~ an *identifier* ([lex.name]), *operator-function-id* ([over.oper]), *literal-operator-id* ([over.literal]), or *conversion-function-id* ([class.conv.fct])~~, or *template-id* ([temp.names]) that denotes an entity or label ([stmt.goto], [stmt.label])~~.

Change paragraph 5:

- Every name ~~that denotes an entity~~ is introduced by a *declaration*~~. Every name that denotes a label is introduced either by a `goto` statement ([stmt.goto]) or a *labeled-statement* ([stmt.label])~~, which is a

  1. *declaration*, *block-declaration*, or *member-declaration* ([dcl.pre], [class.mem])
  2. *init-declarator* ([dcl.decl]),
  3. *identifier* in a structured binding declaration ([dcl.struct.bind]),
  4. *init-capture* ([expr.prim.lambda.capture])
  5. *condition* with a *declarator* ([stmt.stmt]),
  6. *member-declarator* ([class.mem]),
  7. *using-declarator* ([namespace.udecl]),
  8. *parameter-declaration* ([dcl.fct]),
  9. *type-parameter* ([temp.param]),
  10. *elaborated-type-specifier* that introduces a name ([dcl.type.elab]),
  11. *class-specifier* ([class.pre]),
  12. *enum-specifier* or *enumerator-definition* ([dcl.enum]),
  13. *exception-declaration* ([except.pre])
  14. implicit declaration of an injected-class-name ([class.pre]).

  [*Note*: The interpretation of a *for-range-declaration* produces one or more of the above ([stmt.ranged]). — *end note*] An entity *E* is *denoted* by the name (if any) that is introduced by a declaration of *E* or by a *typedef-name* introduced by a declaration specifying *E*.

Change paragraph 9:

- Two names are *the same* if

  1. they are *identifier*s composed of the same character sequence, or

2. they are *operator-function-id*s formed with the same operator, or
3. they are *conversion-function-id*s formed with ~~the same~~ equivalent ([temp.over.link]) type~~s~~, or
4. ~~they are *template-ids* that refer to the same class, function, or variable ([temp.type]), or~~
5. they are *literal-operator-id*s ([over.literal]) formed with the same literal suffix identifier.

#[basic.def]

Change paragraph 1:

- A declaration ([dcl.dcl]) may ~~(~~re~~)~~introduce one or more names and/or entities into a translation unit ~~or redeclare names introduced by previous declarations~~. If so, the declaration specifies the interpretation and semantic properties of these names. A declaration of an entity or *typedef-name* *X* is a *redeclaration* of *X* if another declaration of *X* is reachable from it ([module.reach]). […]

Change bullet (2.5):

- it is ~~introduced by~~ an *elaborated-type-specifier* ([class.name]),

#[basic.def.odr]

Change paragraph 4:

- A variable is named by an expression if the expression is an *id-expression* that denotes it. A variable x ~~whose~~ named ~~appears as~~ by a potentially-evaluated expression *E* […]

#[basic.scope]

#[basic.scope.declarative]

Remove subclause, referring the old stable name to [basic.scope.scope] (added below). Remove the cross reference in [dcl.struct.bind]/1. Update the 2 surviving cross references (in [basic.def.odr]/9) to refer to [basic.scope.scope]. Replace the 8 surviving mentions of “declarative region” (in [basic.def.odr]/9, [expr.prim.id.unqual]/2, and [expr.prim.lambda.capture]/7) with “scope”.

General #[basic.scope.scope]

Add subclause to front of [basic.scope], adapting a note and examples from [over.load] and [namespace.alias]/3:

- The declarations in a program appear in a number of *scopes* that are in general discontiguous. The *global scope* contains the entire program; every other scope *S* is introduced by a declaration, *parameter-declaration-clause*, statement, or *handler* (as described in the following subclauses of [basic.scope]) appearing in another scope which thereby contains *S*. An *enclosing* scope at a program point is any scope that contains it; the smallest such scope is said to be the *immediate scope* at that point. A scope *intervenes* between a program point *P* and a scope *S* (that does not contain *P*) if it is or contains *S* but does not contain *P*.

- Unless otherwise specified:

  1. The smallest scope that contains a scope *S* is the *parent scope* of *S*.
  2. No two declarations (re)introduce the same entity.
  3. A declaration *inhabits* the immediate scope at its locus ([basic.scope.pdecl]).
  4. A declaration’s *target* scope is the scope it inhabits.
  5. Any names (re)introduced by a declaration are *bound* to it in its target scope.

  An entity *belongs* to a scope *S* if *S* is the target scope of a declaration of the entity. [*Note*: Special cases include that:

  1. Template parameter scopes are parents only to other template parameter scopes ([basic.scope.temp]).
  2. Corresponding declarations with appropriate linkage declare the same entity ([basic.link]).
  3. The declaration in a *template-declaration* inhabits the same scope as the *template-declaration*.
  4. Friend declarations and declarations of qualified names and template specializations do not bind names ([dcl.meaning]); those with qualified names target a specified scope, and other friend declarations and certain *elaborated-type-specifier*s ([dcl.type.elab]) target a larger enclosing scope.
  5. Block-scope extern declarations target a larger enclosing scope but bind a name in their immediate scope.
  6. The names of unscoped enumerators are bound in the two innermost enclosing scopes ([dcl.enum]).
  7. A class’s name is also bound in its own scope ([class.pre]).
  8. The names of the members of an anonymous union are bound in the union’s parent scope ([class.union.anon]).

  — *end note*]

- Two declarations *correspond* if they (re)introduce the same name, both declare constructors, or both declare destructors, unless

  1. either is a *using-declarator*, or
  2. one declares a type (not a *typedef-name*) and the other declares a variable, non-static data member other than of an anonymous union ([class.union.anon]), enumerator, function, or function template, or
  3. each declares a function or function template, except when
     1. both declare functions with the same parameter-type-list[*Footnote*: An implicit object parameter ([over.match.funcs]) is not part of the parameter-type-list. — *end footnote*], equivalent ([temp.over.link]) trailing *requires-clause*s (if any, except as specified in [temp.friend]), and, if both are non-static members, the same *cv*-qualifiers (if any) and *ref-qualifier* (if both have one), or

2. both declare function templates with equivalent parameter-type-lists, return types (if any), *template-head*s, and trailing *requires-clause*s (if any), and, if both are non-static members, the same *cv*-qualifiers (if any) and *ref-qualifier* (if both have one).

[*Note*: Declarations can correspond even if neither binds a name. [*Example*:

```
struct A {
  friend void f();    // #1
};
struct B {
  friend void f() {}  // corresponds to, and defines, #1
};
```

— *end example*] — *end note*] [*Example*:

```
typedef int Int;
enum E : int { a };
void f(int);       // #1
void f(Int) {}     // defines #1
void f(E) {}       // OK: another overload

struct X {
  static void f();
  void f() const;  // error: redeclaration
  void g();
  void g() const;  // OK
  void g() &;      // error: redeclaration
};
```

— *end example*]

- Two declarations *potentially conflict* if they correspond and cause their shared name to denote different entities ([basic.link]). The program is ill-formed if, in any scope, a name is bound to two declarations that potentially conflict and one precedes the other ([basic.lookup]). [*Note*: Overload resolution can consider potentially conflicting declarations found in multiple scopes (*e.g.*, via *using-directive*s or for operator functions), in which case it is often ambiguous. — *end note*] [*Example*:

```
void f() {
  int x,y;
  void x();  // error: different entity for x
  int y;     // error: redefinition
}
enum { f };  // error: different entity for ::f
namespace A {}
namespace B = A;
namespace B = A;  // OK: no effect
namespace B = B;  // OK: no effect
namespace A = B;  // OK: no effect
namespace B {}    // error: different entity for B
```

— *end example*]

- A declaration is *nominable* in a class, class template, or namespace *E* at a point *P* if it precedes *P*, it does not inhabit a block scope, and its target scope is the scope associated with *E* or, if *E* is a namespace, any element of the inline namespace set of *E* ([namespace.def]). [*Example*:

```
namespace A {
  void f() {void g();}
  inline namespace B {
    struct S {
      friend void h();
      static int i;
    };
  }
}
```

At the end of this example, the declarations of `f`, `B`, `S`, and `h` are nominable in `A`, but those of `g` and `i` are not. — *end example*]

- When instantiating a templated entity ([temp.pre]), any scope *S* introduced by any part of the template definition is considered to be introduced by the instantiated entity and to contain the instantiations of any declarations that inhabit *S*.

#[basic.scope.pdecl]

Change each paragraph 1–12:

1. The ~~*point of declaration for*~~ *locus* of a ~~name~~declaration ([basic.pre]) that is a declarator is immediately after ~~its~~the complete declarator ([dcl.decl]) ~~and before its initializer (if any), except as noted below~~. [*Example*:

   […] — *end example*]

2. [*Note*: A name from an outer scope remains visible up to the ~~point~~locus of the declaration ~~of the name~~ that hides it. [*Example*:

   […] — *end example*] — *end note*]

3. The ~~point~~locus of ~~declaration for a class or class template first declared by~~ a *class-specifier* is immediately after the *identifier* or *simple-template-id* (if any) in its *class-head* ([class.pre]). The ~~point~~locus of ~~declaration for an enumeration~~an *enum-specifier* or *opaque-enum-declaration* is immediately after the *identifier* (if any) in ~~either its *enum-specifier* ([dcl.enum]) or its first *opaque-enum-declaration*~~it ([dcl.enum]) ~~, whichever comes first~~. The ~~point~~locus of ~~declaration of an alias or alias template~~an *alias-declaration* is immediately ~~follows the *defining-type-id* to which the alias refers~~after it.

4. The ~~point of declaration~~locus of a *using-declarator* that does not name a constructor is immediately after the *using-declarator* ([namespace.udecl]).

5. The ~~point of declaration for an enumerator~~locus of an *enumerator-definition* is immediately after its~~ enumerator-definition~~. [*Example*:

   […] — *end example*]
6. [~~*Note:* After the point of~~ declaration of a class member, the member name can be ~~looked up~~found in the scope of its class~~. [*Note:* This is true~~ even if the class is an incomplete class. […] — *end note*]
7. ~~[…]~~The locus of an *elaborated-type-specifier* that is a declaration ([dcl.type.elab]) is immediately after it.
8. The ~~point of declaration for~~locus of an injected-class-name declaration ([class.pre]) is immediately following the opening brace of the class definition.
9. The ~~point~~locus of the implicit declaration ~~for~~of a function-local predefined variable ([dcl.fct.def.general]) is immediately before the *function-body* of ~~a~~its function~~'~~s definition.
10. The ~~point~~locus of the declaration of a structured binding ([dcl.struct.bind]) is immediately after the *identifier-list* of the structured binding declaration.
11. The ~~point of declaration for the variable or the structured bindings declared in the~~locus of a for-range-declaration of a range-based for statement ([stmt.ranged]) is immediately after the *for-range-initializer*.

12. The ~~point of declaration for a template parameter~~locus of a *template-parameter* is immediately after its~~ complete *template-parameter*~~. [*Example*:

    […]

    — *end example*]

Insert before paragraph 13:

- The locus of a *concept-definition* is immediately after its *concept-name* ([temp.concept]). [*Note*: The *constraint-expression* cannot use the *concept-name*. — *end note*]

- The locus of a *namespace-definition* with an *identifier* is immediately after the *identifier*. [*Note*: An identifier is invented for an *unnamed-namespace-definition* ([namespace.unnamed]). — *end note*]

[*Drafting note*: There is no normative wording yet corresponding to the *constraint-expression* note. — *end drafting note*]

Change paragraph 13:

- [*Note*: Friend declarations ~~refer to~~may introduce functions or classes that ~~are members of~~belong to the nearest enclosing namespace or block scope, but they do not ~~introduce new~~bind names ~~into that namespace~~anywhere ([~~namespace.memdef~~class.friend]). Function declarations at block scope and variable declarations with the `extern` specifier at block scope ~~refer to declarations that are members of an~~declare entities that belong to the nearest enclosing namespace, but they do not ~~introduce new~~bind names ~~into that scope~~in it. — *end note*]

#[basic.scope.block]

Replace contents, retaining the example:

- Each

  1. selection or iteration statement ([stmt.select], [stmt.iter]),
  2. substatement of such a statement,
  3. *handler* ([except.pre]), or
  4. compound statement ([stmt.block]) that is not the *compound-statement* of a *handler*,

  introduces a *block scope* that includes that statement or *handler*. [*Note*: A substatement that is also a block has only one scope. — *end note*] A variable that belongs to a block scope is a *block variable*.

Move the example from [stmt.for]/3 to here.

- If a declaration whose target scope is the block scope *S* of a

  1. *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
  2. substatement of a selection or iteration statement, or
  3. *handler* of a *function-try-block*

  potentially conflicts with a declaration whose target scope is the parent scope of *S*, the program is ill-formed. [*Example*:

  […]

  — *end example*]

#[basic.scope.param]

Replace contents:

- A *parameter-declaration-clause P* introduces a *function parameter scope* that includes *P*. [*Note*: A function parameter cannot be used for its value within the *parameter-declaration-clause* ([dcl.fct.default]). — *end note*] If *P* is associated with a *declarator* and is preceded by a (possibly-parenthesized) *noptr-declarator* of the form *declarator-id attribute-specifier-seq*$_{opt}$, its scope extends to the end of the nearest enclosing *init-declarator*, *member-declarator*, *declarator* of a *parameter-declaration* or a *nodeclspec-function-declaration*, or *function-definition*, but does not include the locus of the associated *declarator*. [*Note*: In this case, *P* declares the parameters of a function (or a function or template parameter declared with function type). A member function's parameter scope is nested within its class's scope. — *end note*] If *P* is associated with a *lambda-declarator*, its scope extends to the end of the *compound-statement* in the *lambda-expression*. If *P* is associated with a *requirement-*

parameter-list, its scope extends to the end of the *requirement-body* of the *requires-expression*. If *P* is associated with a *deduction-guide*, its scope extends to the end of the *deduction-guide*.

#[basic.funscope]

Remove subclause, referring the stable name (which has no cross references) to [stmt.label].

#[basic.scope.namespace]

Replace contents, adapting the example from [namespace.def]/4:

- Any *namespace-definition* for a namespace *N* introduces a *namespace scope* that includes the *namespace-body* for every *namespace-definition* for *N*. For each non-friend redeclaration or specialization whose target scope is or is contained by the scope, the portion after the *declarator-id*, *class-head-name*, or *enum-head-name* is also included in the scope. The global scope is the namespace scope of the global namespace ([basic.namespace]). [*Example*:

```
namespace Q {
  namespace V { void f(); }
  void V::f() {  // in the scope of V
    void h();    // declares Q::V::h
  }
}
```

— *end example*]

#[basic.scope.class]

Replace contents:

- Any declaration of a class or class template *C* introduces a *class scope* that includes the *member-specification* of the *class-specifier* for *C* (if any). For each non-friend redeclaration or specialization whose target scope is or is contained by the scope, the portion after the *declarator-id*, *class-head-name*, or *enum-head-name* is also included in the scope. [*Note*: Lookup from a program point before the *class-specifier* of a class will find no bindings in the class scope. [*Example*:

```
template<class D>
struct B {
  D::type x;           // #1
};

struct A {using type=int;};
struct C : A,B<C> {};  // error at #1: C::type not found
```

— *end example*] — *end note*]

#[basic.scope.enum]

Replace contents:

- Any declaration of an enumeration *E* introduces an *enumeration scope* that includes the *enumerator-list* of the *enum-specifier* for *E* (if any).

#[basic.scope.temp]

Replace contents:

- Each template *template-parameter* introduces a *template parameter scope* that includes the *template-head* of the *template-parameter*.

- Each *template-declaration D* introduces a template parameter scope that extends from the beginning of its *template-parameter-list* to the end of the *template-declaration*. Any declaration outside the *template-parameter-list* that would inhabit that scope instead inhabits the same scope as *D*. The parent scope of any scope *S* that is not a template parameter scope is the smallest scope that contains *S* and is not a template parameter scope. [*Note*: Therefore, only template parameters belong to a template parameter scope, and only template parameter scopes have a template parameter scope as a parent scope. — *end note*]

#[basic.scope.hiding]

Remove subclause, referring the stable name to [basic.lookup].

#[basic.lookup]

Change paragraph 1:

- […] Name lookup associates the use of a name with a set of declarations ([basic.def]) of that name. Unless otherwise specified, the program is ill-formed if no declarations are found. If the declarations found by name lookup all denote functions or function templates, the declarations are said to form an *overload set*. ~~The~~Otherwise, if the declarations found by name lookup ~~shall either~~do not all denote the same entity ~~or form an overload set~~, they are *ambiguous* and the program is ill-formed. Overload resolution ([over.match], [over.over]) takes place after name lookup has succeeded. The access rules ([class.access]) are considered only once name lookup and function overload resolution (if applicable) have

succeeded. Only after name lookup, function overload resolution (if applicable) and access checking have succeeded are the semantic properties introduced by the ~~name's~~ declaration~~s and its reachable ([module.reach]) redeclarations~~ used in further ~~in expression~~ processing ~~([expr])~~.

Insert before paragraph 2:

- A program point *P* is said to follow any declaration in the same translation unit whose locus ([basic.scope.pdecl]) is before *P*. [*Note*: The declaration might appear in a scope that does not contain *P*. — *end note*] A declaration *X precedes* a program point *P* in a translation unit *L* if *P* follows *X*, *X* inhabits a class scope and is reachable from *P*, or else *X* appears in a translation unit *D* and

  1. *P* follows a *module-import-declaration* or *module-declaration* that imports *D* (directly or indirectly), and
  2. *X* appears after the *module-declaration* in *D* (if any) and before the *private-module-fragment* in *D* (if any), and
  3. either *X* is exported or else *D* and *L* are part of the same module and *X* does not inhabit a namespace with internal linkage or declare a name with internal linkage. [*Note*: Names declared by a *using-declaration* have no linkage. — *end note*]

Move the note (and example) from [basic.scope.namespace]/2 to the end of this paragraph.

Replace paragraphs 2 through 4:

- A name "looked up in the context of an expression" is looked up in the scope where the expression is found.

- The injected-class-name of a class ([class.pre]) is also considered to be a member of that class for the purposes of name hiding and lookup.

- [*Note*: [basic.link] discusses linkage issues. The notions of scope, point of declaration and name hiding are discussed in [basic.scope]. — *end note*]

- A *single search* in a scope *S* for a name *N* from a program point *P* finds all declarations that precede *P* to which any name that is the same as *N* ([basic.pre]) is bound in *S*. If any such declaration is a *using-declarator* whose terminal name ([expr.prim.id.unqual]) is not dependent ([temp.dep.type]), it is replaced by the declarations named by the *using-declarator* ([namespace.udecl]).

- In certain contexts, only certain kinds of declarations are included. After any such restriction, any declarations of classes or enumerations are discarded if any other declarations are found. [*Note*: A type (but not a *typedef-name* or template) is therefore hidden by any other entity in its scope. — *end note*] However, if a lookup is *type-only*, only declarations of types and templates whose specializations are types are considered; furthermore, if declarations of a *typedef-name* and of the type to which it refers are found, the declaration of the *typedef-name* is discarded instead of the type declaration.

#[class.member.lookup]

Move to front of [basic.lookup], editing as follows:

Remove paragraph 1:

- Member name lookup determines the meaning […]

Change paragraph 2:

- ~~The~~ A *search* in a scope *X* for a name *N* from a program point *P* is a single search in *X* for *N* from *P* unless *X* is the scope of a class or class template *T*, in which case the following steps define the result of ~~a member name `f` in a class scope `C`~~ the search. [*Note*: The result differs only if *N* is a *conversion-function-id* or if the single search would find nothing. — *end note*]

Replace `f` with *N* and `C` with *C* in the following paragraphs.

Change paragraph 3:

- The *lookup set* for ~~`f`~~ *N* in ~~`C`~~ *C*, called *S*(~~*f*~~*N*,*C*), consists of two component sets: the *declaration set*, a set of members named ~~`f`~~ *N*; and the *subobject set*, a set of subobjects where declarations of these members ~~(possibly including *using-declarations*)~~ were found (possibly via *using-declarations*). In the declaration set, ~~*using-declarations* are replaced by the set of designated members that are not hidden or overridden by members of the derived class ([namespace.udecl]), and~~ type declarations (including injected-class-names) are replaced by the types they designate. *S*(~~*f*~~*N*,*C*) is calculated as follows:

Change paragraph 4:

- ~~If `C` contains a declaration of the name `f`, the declaration set contains every declaration of `f` declared in `C` that satisfies the requirements of the language construct in which the lookup occurs.~~ The declaration set is the result of a single search in the scope of *C* for *N* from immediately after the *class-specifier* of *C* if *P* is in a complete-class context of *C* or from *P* otherwise. ~~[*Note*: […] — *end note*]~~ […]

[*Drafting note*: The plan for CWG2335 is to describe forbidden dependency cycles among the complete-class contexts of a class. — *end drafting note*]

Change paragraph 5:

- […] ~~If `C` has base classes, c~~Calculate the lookup set for ~~`f`~~ *N* in each direct non-dependent ([temp.dep.type]) base class subobject B_i, and merge each such lookup set *S*(~~*f*~~*N*,B_i) in turn into *S*(~~*f*~~*N*,*C*). [*Note*: If *T* is incomplete, only base classes whose *base-specifier* appears before *P* are considered. If *T* is an instantiated class, its base classes are not dependent. — *end note*]

Change paragraph 7:

- The result of ~~name lookup for `f` in `C`~~ the search is the declaration set of *S*(~~*f*~~*N*,~~*C*~~*T*). If it is an invalid set, the program is ill-formed. If it differs from the result of a search in *T* for *N* from immediately after the *class-specifier* of *T*, the program is ill-formed, no diagnostic required. [*Example*:

[…]

S(~~`x`~~`x`,~~`F`~~`E`) is unambiguous because the `A` and `B` base class subobjects of `D` are also base class subobjects of `E`, so *S*(~~`x`~~`x`,~~`D`~~`D`) is discarded in the first merge step. — *end example*]

Replace paragraph 8:

- If the name of an overloaded function is unambiguously found, […]

- If *N* is a non-dependent *conversion-function-id*, conversion function templates that are members of *T* are considered. For each such template *F*, the lookup set *S*(*t*,*T*) is constructed, considering a function template declaration to have the name *t* only if it corresponds to a declaration of *F* ([basic.scope.scope]). The members of the declaration set of each such lookup set, which shall not be an invalid set, are included in the result. [*Note*: Overload resolution will discard those that cannot convert to the type specified by *N* ([temp.over]). — *end note*]

#[basic.lookup.unqual]

Replace contents:

- A *using-directive* is active in a scope *S* at a program point *P* if it precedes *P* and inhabits either *S* or the scope of a namespace nominated by a *using-directive* that is active in *S* at *P*. [*Note*: A *using-directive* is exported if and only if it appears in a header unit. — *end note*]

- An unqualified search in a scope *S* from a program point *P* includes the results of searches from *P* in

  1. *S*, and
  2. for any scope *U* that contains *P* and is or is contained by *S*, each namespace contained by *S* that is nominated by a *using-directive* that is active in *U* at *P*.

  If no declarations are found, the results of the unqualified search are the results of an unqualified search in the parent scope of *S*, if any, from *P*. [*Note*: When a class scope is searched, the scopes of its base classes are also searched ([class.member.lookup]). If it inherits from a single base, it is as if the scope of the base immediately contains the scope of the derived class. Template parameter scopes that are associated with one scope in the chain of parents are also considered ([temp.local]). — *end note*]

- *Unqualified name lookup* from a program point performs an unqualified search in its immediate scope.

- An *unqualified name* is a name that does not follow a *nested-name-specifier* or the `.` or `->` in a class member access expression ([expr.ref]), possibly after a `template` keyword or `~`. Unless otherwise specified, such a name undergoes unqualified name lookup from the point where it appears.

- An unqualified name that is a component name ([expr.prim.id.unqual]) of a *type-specifier* or *ptr-operator* of a *conversion-type-id* is looked up in the same fashion as the *conversion-function-id* in which it appears. If that lookup finds nothing, it undergoes unqualified name lookup; in each case, only names that denote types or templates whose specializations are types are considered.

- In a friend declaration *declarator* whose *declarator-id* is a *qualified-id* whose lookup context is ([basic.lookup.qual]) a class or namespace *S*, lookup for an unqualified name that appears after the *declarator-id* performs a search in the scope associated with *S*. If that lookup finds nothing, it undergoes unqualified name lookup. [*Example*:

```
using I=int;
using D=double;
namespace A {
  inline namespace N {using C=char;}
  using F=float;
  void f(I);
  void f(D);
  void f(C);
  void f(F);
}
struct X0 {using F=float;};
struct W {
  using D=void;
  struct X : X0 {
    void g(I);
    void g(::D);
    void g(F);
  };
};
namespace B {
  typedef short I,F;
  class Y {
    friend void A::f(I);  // error: no void A::f(short)
    friend void A::f(D);  // OK
    friend void A::f(C);  // error: A::N::C not found
    friend void A::f(F);  // OK
    friend void X::g(I);  // error: no void X::g(short)
    friend void X::g(D);  // OK
    friend void X::g(F);  // OK
  };
}
```

— *end example*]

#[basic.lookup.argdep]

Change paragraph 1:

- When the *postfix-expression* in a function call ([expr.call]) is an *unqualified-id*, ~~other namespaces not considered during the usual~~ and unqualified lookup ([basic.lookup.unqual]) ~~may be searched, and in those namespaces, namespace-scope friend function or function template declarations ([class.friend]) not otherwise visible may be found. These modifications to the search~~ for the name in the *unqualified-id* does not find any

  1. declaration of a class member, or
  2. function declaration inhabiting a block scope, or
  3. declaration not of a function or function template

  then lookup for the name also includes the result of *argument-dependent lookup* in a set of *associated namespaces* that depends on the types of the arguments (and for template template arguments, the namespace of the template argument), as specified below. [*Example*:

  […]

  — *end example*]

Move the note from [basic.lookup.unqual]/3 here, inserting it before paragraph 2.

Change paragraph 2:

- For each argument type `T` in the function call, there is ~~a set of zero or more *associated namespaces* and~~ a set of zero or more *associated entities* ~~(other than namespaces)~~ to be considered. The sets of ~~namespaces and~~ entities are determined entirely by the types of the function arguments (and ~~the namespace of~~ any template template arguments). Typedef names and *using-declaration*s used to specify the types do not contribute to this set. The sets of ~~namespaces and~~ entities ~~are~~ is determined in the following way:

  1. If `T` is a fundamental type, its associated sets of ~~namespaces and~~ entities ~~are both~~ is empty.
  2. If `T` is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and its direct and indirect base classes. ~~Its associated namespaces are the innermost enclosing namespaces of its associated entities.~~ Furthermore, if `T` is a class template specialization, its associated ~~namespaces and~~ entities also include: the ~~namespaces and~~ entities associated with the types of the template arguments provided for template type parameters ~~(excluding template template parameters)~~; the templates used as template template arguments; ~~the namespaces of which any template template arguments are members;~~ and the classes of which any member templates used as template template arguments are members. [*Note*: Non-type template arguments do not contribute to the set of associated ~~namespaces~~ entities. — *end note*]
  3. If `T` is an enumeration type, ~~its associated namespace is the innermost enclosing namespace of its declaration, and~~ its associated entities are `T` and, if it is a class member, the member's class.
  4. If `T` is a pointer to `U` or an array of `U`, its associated ~~namespaces and~~ entities are those associated with `U`.
  5. If `T` is a function type, its associated ~~namespaces and~~ entities are those associated with the function parameter types and those associated with the return type.
  6. If `T` is a pointer to a member function of a class `X`, its associated ~~namespaces and~~ entities are those associated with the function parameter types and return type, together with those associated with `X`.
  7. If `T` is a pointer to a data member of class `X`, its associated ~~namespaces and~~ entities are those associated with the member type together with those associated with `X`.

  ~~If an associated namespace is an inline namespace ([namespace.def]), its enclosing namespace is also included in the set. If an associated namespace directly contains inline namespaces, those inline namespaces are also included in the set.~~ In addition, if the argument is ~~the name or address of~~ an overload set or the address of such a set, its associated entities ~~and namespaces~~ are the union of those associated with each of the members of the set, i.e., the entities ~~and namespaces~~ associated with its parameter types and return type. Additionally, if the aforementioned overload set is named with a *template-id*, its associated entities ~~and namespaces~~ also include ~~those of its type *template-arguments* and~~ its template *template-argument*s and those associated with its type *template-arguments*.

Change paragraph 3 and move it to the end of the subclause:

- ~~Let *X* be the lookup set produced by unqualified lookup ([basic.lookup.unqual]) and let *Y* be the lookup set produced by argument dependent lookup (defined as follows). If *X* contains~~

  1. ~~a declaration of a class member, or~~
  2. ~~a block-scope function declaration that is not a *using-declaration*, or~~
  3. ~~a declaration that is neither a function nor a function template~~

  ~~then *Y* is empty. Otherwise *Y* is the set of declarations found in the namespaces associated with the argument types as described below. The set of declarations found by the lookup of the name is the union of *X* and *Y*.~~ [*Note*: The associated namespaces ~~and entities associated with the argument types~~ can include namespaces ~~and entities~~ already considered by ~~the~~ ordinary unqualified lookup. — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 4:

- ~~When considering an~~ The associated namespaces for a call are the innermost enclosing non-inline namespaces for its associated entities as well as every element of the inline namespace set of those namespaces. Argument-dependent lookup finds all declarations of functions and function templates that

  1. are found by a search of any associated namespace `N`, ~~the lookup is the same as the lookup performed when `N` is used as a qualifier ([namespace.qual]) except that:~~ or
  2. ~~Any *using-directives* in `N` are ignored.~~

3. ~~All names except those of (possibly overloaded) functions and function templates are ignored.~~
4. ~~Any namespace scope~~ are declared as a friend ~~functions or friend function templates~~ ([class.friend]) ~~declared in~~ of any class~~es~~ with a reachable definition~~s~~ in the set of associated entities ~~are visible within their respective namespaces even if they are not visible during an ordinary lookup ([namespace.memdef]).~~, or
5. ~~Any~~ are exported ~~declaration `D` in `N` declared within the purview of~~, are attached to a named module `M` ([module.interface]) ~~is visible if there is~~, do not appear in the translation unit containing the point of the lookup, and have the same innermost enclosing non-inline namespace scope as a declaration of an associated entity attached to `M` ~~with the same innermost enclosing non-inline namespace as `D`~~.

~~5.~~ If the lookup is for a dependent name ([temp.dep], [temp.dep.candidate]), ~~any declaration `D` in `N` is visible if `D` would be visible to qualified name lookup ([namespace.qual]) at any~~ the above lookup is also performed from each point in the instantiation context ([module.context]) of the lookup, ~~unless `D` is declared~~ additionally ignoring any declaration that appears in another translation unit, is attached to the global module, and is either discarded ([module.global.frag]) or has internal linkage.

#[basic.lookup.qual]

Change paragraph 1:

- ~~The name of a class or namespace member or enumerator can be referred to after the `::` scope resolution operator ([expr.prim.id.qual]) applied to a *nested-name-specifier* that denotes its class, namespace, or enumeration. If~~ Lookup of an *identifier* followed by a `::` scope resolution operator ~~in a *nested-name-specifier* is not preceded by a *decltype-specifier*, lookup of the name preceding that `::`~~ considers only namespaces, types, and templates whose specializations are types. If ~~the name found does not~~ a name, *template-id*, or *decltype-specifier* is followed by a `::`, it shall designate a namespace ~~or a~~, class, enumeration, or dependent type, ~~the program is ill-formed~~ and the `::` is never interpreted as a complete *nested-name-specifier*. [*Example*:

```
class A {
public:
  static int n;
};
int main() {
  int A;
  A::n = 42;        // OK
  A b;              // error: A does not name a type
}
template<int> struct B : A {};
namespace N {
  template<int> void B();
  int f() {return B<0>::n;}  // error: N::B<0> is not a type
}
```

— *end example*]

Replace paragraphs 2 through 5:

- [*Note*: Multiply qualified names, […]. — *end note*]

- In a declaration in which the *declarator-id* is a *qualified-id*, […]

- A name prefixed by the unary scope operator `::` ([expr.prim.id.qual]) is looked up in global scope, […]

- A name prefixed by a *nested-name-specifier* that nominates an enumeration type shall represent an *enumerator* of that enumeration.

- A member-qualified name is the (unique) component name ([expr.prim.id.unqual]), if any, of

  1. an *unqualified-id*, or
  2. a *nested-name-specifier* of the form *type-name* `::` or *namespace-name* `::`,

  in the *id-expression* of a class member access expression ([expr.ref]). A *qualified name* is a member-qualified name or the terminal name of a *qualified-id*, a *using-declarator*, a *typename-specifier*, a *qualified-namespace-specifier*, or a *nested-name-specifier*, *elaborated-type-specifier*, or *class-or-decltype* that has a *nested-name-specifier* ([expr.prim.id.qual]). The *lookup context* of a member-qualified name is the type of its associated object expression (considered dependent if the object expression is type-dependent). The lookup context of any other qualified name is the type, template, or namespace nominated by the preceding *nested-name-specifier*. [*Note*: When parsing a class member access, the name following the `->` or `.` is a qualified name even though it is not yet known of which kind. — *end note*] [*Example*: In

  ```
  N::C::m.Base::f()
  ```

  `Base` is a member-qualified name; the other qualified names are `C`, `m`, and `f`. — *end example*]

- *Qualified name lookup* in a class, namespace, or enumeration performs a search of the scope associated with it ([class.member.lookup]) except as specified below. Unless otherwise specified, a qualified name undergoes qualified name lookup in its lookup context from the point where it appears unless the lookup context either is dependent and is not the current instantiation ([temp.dep.type]) or is not a class or class template. If nothing is found by qualified lookup for a member-qualified name that is the terminal name of a *nested-name-specifier* and is not dependent, it undergoes unqualified lookup. [*Note*: During lookup for a template specialization, no names are dependent. — *end note*] [*Example*:

  ```
  int f();
  struct A {
    int B,C;
    template<int> using D=void;
    using T=void;
    void f();
  };
  using B=A;
  ```

```
template<int> using C=A;
template<int> using D=A;
template<int> using X=A;

template<class T>
void g(T *p) {            // as instantiated for g<A>:
  p->X<0>::f();           // error: A::X not found in ((p->X) < 0) > ::f()
  p->template X<0>::f();  // OK: ::X found in definition context
  p->B::f();              // OK: non-type A::B ignored
  p->template C<0>::f();  // error: A::C is not a template
  p->template D<0>::f();  // error: A::D<0> is not a class type
  p->T::f();              // error: A::T is not a class type
}
template void g(A*);
```

— *end example*]

Change paragraph 6:

- ~~In a *qualified-id* of the form:~~

  ~~*nested-name-specifier*$_{opt}$ *type-name* :: ~ *type-name*~~

  ~~the second *type-name* is looked up in the same scope as the first.~~ If a qualified name *Q* follows a ~:

  1. If *Q* is a member-qualified name, it undergoes unqualified lookup as well as qualified lookup.
  2. Otherwise, its *nested-name-specifier N* shall nominate a type. If *N* has another *nested-name-specifier S*, *Q* is looked up as if its lookup context were that nominated by *S*.
  3. Otherwise, if the terminal name of *N* is a member-qualified name *M*, *Q* is looked up as if ~*Q* appeared in place of *M* (as above).
  4. Otherwise, *Q* undergoes unqualified lookup.
  5. Each lookup for *Q* considers only types (if *Q* is not followed by a <) and templates whose specializations are types. If it finds nothing or is ambiguous, it is discarded.
  6. The *type-name* that is or contains *Q* shall refer to its (original) lookup context (ignoring *cv*-qualification) under the interpretation established by at least one (successful) lookup performed.

  [*Example*:

```
struct C {
  typedef int I;
};
typedef int I1, I2;
extern int* p;
extern int* q;
void f() {
p->C::I::~I();   // I is looked up in the scope of C
q->I1::~I2();    // I2 is found by unqualified looked up ~~in the scope of the postfix expression~~
}

struct A {
  ~A();
};
typedef A AB;
int main() {
  AB* p;
  p->AB::~AB();     // explicitly calls the destructor for A
}
```

  — *end example*] ~~[*Note:* [basic.lookup.classref] describes how name lookup proceeds after the . and .> operators. — *end note*]~~

#[class.qual]

Remove paragraph 1:

- If the *nested-name-specifier* of a *qualified-id* nominates a class, […]

Change paragraph 2:

- In a lookup for a qualified name whose lookup context is a class `C` in which function names are not ignored[…] ~~and the *nested-name-specifier* nominates a class `C`~~:

  1. if the ~~name specified after the *nested-name-specifier*, when looked up in `C`, is~~ search finds the injected-class-name of `C` ([class.pre]), or
  2. ~~in~~ if the qualified name is dependent and is the terminal name of a *using-declarator* ~~of a *using-declaration*~~ ([namespace.udecl]) that ~~is a *member-declaration*, if the name specified after the *nested-name-specifier* is the same as the *identifier* or the *simple-template-id*'s *template-name* in the last component of the *nested-name-specifier*~~ names a constructor,

  the name is instead considered to name the constructor of class `C`. ~~[*Note:* For example, the constructor is not an acceptable lookup result in an *elaborated-type-specifier* so the constructor would not be used in place of the injected-class-name. — *end note*]~~ Such a constructor name shall be used only in the *declarator-id* of a (friend) declaration ~~that names~~ of a constructor or in a *using-declaration*. [*Example*:

  […]

  — *end example*]

Remove paragraph 3:

- A class member name hidden by a name in a nested declarative region […]

#[namespace.qual]

Remove paragraphs 1 and 2:

- If the *nested-name-specifier* of a *qualified-id* nominates a namespace […]

- For a namespace `x` and name `m`, […]

Replace paragraph 3, retaining its example:

- Given `x::m` (where `x` is a user-declared namespace), […]

- Qualified name lookup in a namespace *N* additionally searches every element of the inline namespace set of *N* ([namespace.def]). If nothing is found, the results of the lookup are the results of qualified name lookup in each namespace nominated by a *using-directive* that precedes the point of the lookup and inhabits *N* or an element of its inline namespace set. [*Note*: If a *using-directive* refers to a namespace that has already been considered, it does not affect the result. — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 6:

- ~~During the lookup of a qualified namespace member name, if the lookup finds more than one declaration of the member, and if one declaration introduces a c~~[*Note*: Class ~~name or~~and enumeration ~~name and the~~declarations are not discarded because of other declarations ~~introduce either the same variable, the same enumerator, or a set of functions, the non-type name hides the class or enumeration name if and only if the declarations are from the same namespace; otherwise (the declarations are from different namespaces), the program is ill-formed~~found in other searches. *— end note*] [*Example*:

  […]

  — *end example*]

Remove paragraph 7:

- In a declaration for a namespace member in which the *declarator-id* is a *qualified-id*, […]

#[basic.lookup.elab]

Remove paragraphs 1 and 2:

- An *elaborated-type-specifier* ([dcl.type.elab]) may be used to refer […]

- If the *elaborated-type-specifier* has no *nested-name-specifier*, […]

Insert before paragraph 3:

- If the *class-key* or `enum` keyword in an *elaborated-type-specifier* is followed by an *identifier* that is not followed by `::`, lookup for the *identifier* is type-only. [*Note*: In general, the recognition of an *elaborated-type-specifier* depends on the following tokens. If the *identifier* is followed by `::`, see [basic.lookup.qual]. — *end note*]

Change paragraph 3:

- If the terminal name of the *elaborated-type-specifier* ~~has~~is a ~~*nested-name-specifier*,~~ qualified name~~ lookup is performed, as described in [basic.lookup.qual], but ignoring any non-type names that have been declared~~, lookup for it is type-only. If the name lookup does not find a previously declared *type-name*, the *elaborated-type-specifier* is ill-formed.

#[basic.lookup.classref]

Remove subclause, referring the stable name to [basic.lookup.qual].

#[basic.link]

Change paragraph 3:

- ~~A~~The name ~~having~~of an entity that belongs to a namespace scope ([basic.scope.namespace]) has internal linkage if it is the name of

  […]

Change paragraph 4:

- An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. ~~A~~The name ~~having~~of an entity that belongs to a namespace scope that has not been given internal linkage above and that is

the name of

[…]

Change paragraph 5:

- In addition, a member function, static data member, a named class or enumeration ~~of~~that inhabits a class scope, or an unnamed class or enumeration defined in a ~~class scope~~ typedef declaration that inhabits a class scope such that the class or enumeration has the typedef name for linkage purposes ([dcl.typedef]), has the same linkage, if any, as the name of the class of which it is a member.

Change paragraph 6:

- ~~The name of a function declared in block scope and the name of a variable declared by a block scope `extern` declaration have linkage. If such a declaration is attached to a named module, the program is ill-formed. If there is a visible declaration of an entity with linkage, ignoring entities declared outside the innermost enclosing namespace scope, such that the block scope declaration would be a (possibly ill-formed) redeclaration if the two declarations appeared in the same declarative region, the block scope declaration declares that same entity and receives the linkage of the previous declaration. If there is more than one such matching entity, the program is ill-formed. Otherwise, if no matching entity is found, the block scope entity receives external linkage. If, within a translation unit, the same entity is declared with both internal and external linkage, the program is ill-formed.~~ [*Example*:

```
static void f();
extern "C" void h();
static int i = 0;               // #1
void g() {
  extern void f();              // internal linkage
  extern void g();              // ::g, external linkage
  extern void h();              // C language linkage
  int i;                        // #2: i has no linkage
  {
    extern void f();            // internal linkage
    extern int i;               // #3: exinternal linkage, ill-formed
  }
}
```

~~Without~~Even though the declaration at line #2 hides the declaration at line #1, the declaration at line #3 ~~would link with the~~still redeclar~~ation~~es ~~at line~~ #1~~. Because the declaration with~~ and receives internal linkage ~~is hidden, however, #3 is given external linkage, making the program ill-formed~~. — *end example*]

Remove paragraph 7 (whose example is moved to [dcl.meaning]):

- When a block scope declaration of an entity with linkage is not found to refer to some other declaration, […]

Replace paragraph 9:

- Two names that are the same ([basic.pre]) and that are declared in different scopes […]

- Two declarations of entities declare the same entity if, considering declarations of unnamed types to introduce their names for linkage purposes, if any ([dcl.typedef], [dcl.enum]), they correspond ([basic.scope.scope]), have the same target scope that is not a function or template parameter scope, and either

  1. they appear in the same translation unit, or
  2. they both declare names with module linkage and are attached to the same module, or
  3. they both declare names with external linkage.

  [*Note*: There are other circumstances in which declarations declare the same entity ([dcl.link], [temp.type], [temp.class.spec]). — *end note*]

- If a declaration *H* that declares a name with internal linkage precedes a declaration *D* in another translation unit *U* and would declare the same entity as *D* if it appeared in *U*, the program is ill-formed. [*Note*: Such an *H* can appear only in a header unit. — *end note*]

Change paragraph 10:

- If ~~a declaration would redeclare a reachable~~two declarations of an entity are attached to ~~a~~ different modules, the program is ill-formed; no diagnostic is required if neither is reachable from the other. [*Example*:

  […]

  — *end example*] As a consequence of these rules, all declarations of an entity are attached to the same module; the entity is said to be *attached* to that module.

Change paragraph 11:

- For any two declarations of an entity:

  1. If one declares it to be a variable or function, the other shall declare it as one of the same type.
  2. If one declares it to be an enumerator, the other shall do so.
  3. If one declares it to be a namespace, the other shall do so.
  4. If one declares it to be a type, the other shall declare it to be a type of the same kind ([dcl.type.elab]).
  5. If one declares it to be a class template, the other shall do so with the same kind and an equivalent *template-head* ([temp.over.link]). [*Note*: The declarations may supply different default template arguments. — *end note*]

6. If one declares it to be a function template or a (partial specialization of a) variable template, the other shall declare it to be one with an equivalent *template-head* and type.
7. If one declares it to be an alias template, the other shall declare it to be one with an equivalent *template-head* and *defining-type-id*.
8. If one declares it to be a concept, the other shall do so.

Types are compared a~~A~~fter all adjustments of types (during which typedefs ([dcl.typedef]) are replaced by their definitions)~~, the types specified by all declarations referring to a given variable or function shall be identical, except that~~; declarations for an array object can specify array types that differ by the presence or absence of a major array bound ([dcl.array]). ~~A violation of this rule on type identity does not require a~~No diagnostic is required if neither declaration is reachable from the other. [*Example:*

```
int f(int x,int x);  // error: different entities for x
void g();            // #1
void g(int);         // OK: different entity from #1
int g();             // error: same entity as #1 with different type
void h();            // #2
namespace h {}       // error: same entity as #2, but not a function
```

— end example]

#[basic.life]

Change the footnote in paragraph 6:

- For example, before the ~~construction of a global object that is initialized via a user-provided constructor~~dynamic initialization of an object with static storage duration ([~~class.edtor~~basic.start.dynamic]).

#[basic.stc]

#[basic.stc.static]

Change paragraph 3:

- The keyword `static` can be used to declare a ~~local~~block variable with static storage duration. [*Note*: [stmt.dcl] and [basic.start.term] describe~~s~~ the initialization ~~of local `static` variables; [basic.start.term] describes the~~and destruction of ~~local `static`~~such variables. — *end note*]

#[basic.stc.auto]

Change paragraph 1:

- ~~Block-scope v~~Variables that belong to a block or parameter scope and are not explicitly declared `static`, `thread_local`, or `extern` have *automatic storage duration*. The storage for these entities lasts until the block in which they are created exits.

#[basic.stc.dynamic]

#[basic.stc.dynamic.allocation]

Change paragraph 1:

- An allocation function ~~shall be~~that is not a class member function ~~or a global function; a program is ill-formed if an allocation function is declared in a namespace scope other than~~shall belong to the global scope ~~or declared static in global scope~~and not have a name with internal linkage. The return type shall be `void*`. The first parameter shall have type `std::size_t` ([support.types]). The first parameter shall not have an associated default argument ([dcl.fct.default]). The value of the first parameter is interpreted as the requested size of the allocation. An allocation function can be a function template. Such a template shall declare its return type and first parameter as specified above (that is, template parameter types shall not be used in the return type and first parameter type). ~~Template a~~Allocation function templates shall have two or more parameters.

#[basic.stc.dynamic.deallocation]

Change paragraph 1:

- A d~~D~~eallocation function~~s shall be~~ that is not a class member function~~s or global functions; a program is ill-formed if deallocation functions are declared in a namespace scope other than~~ shall belong to the global scope~~ or declared static in global scope~~ and not have a name with internal linkage.

#[basic.type.qualifier]

Change paragraph 4:

- [*Note*: See [dcl.fct] and [~~class.this~~over.match.funcs] regarding function types that have *cv-qualifier*s. — *end note*]

#[basic.start]

#[basic.start.main]

Change paragraph 1:

- A program shall contain ~~a global~~exactly one function called `main` ~~attached~~that belongs to the global ~~module~~scope. Executing a program starts a main thread of execution ([intro.multithread], [thread.threads]) in which the `main` function is invoked, and in which variables of static storage duration might be initialized ([basic.start.static]) and destroyed ([basic.start.term]). It is implementation-defined whether a program in a freestanding environment is required to define a `main` function. [*Note*: In a freestanding environment, startup and termination is implementation-defined; startup contains the execution of constructors for non-local objects~~ of namespace scope~~ with static storage duration; termination contains the execution of destructors for objects with static storage duration. — *end note*]

Change paragraph 2:

- An implementation shall not predefine the `main` function. ~~This function shall not be overloaded.~~ Its type shall have C++ language linkage and it shall have a declared return type of type `int`, but otherwise its type is implementation-defined. […]

Change paragraph 3:

- The function `main` shall not be used within a program. The linkage ([basic.link]) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed. The function `main` shall not be a coroutine ([dcl.fct.def.coroutine]). The `main` function shall not be declared with a *linkage-specification* ([dcl.link]). A program that declares a variable `main` ~~at~~that belongs to the global scope, or that declares a function `main` ~~at~~that belongs to the global scope and is attached to a named module, or that declares ~~the~~an entity named `main` with C language linkage (in any namespace) is ill-formed. The name `main` is not otherwise reserved. [*Example*: Member functions, classes, and enumerations can be called `main`, as can entities in other namespaces. — *end example*]

#[basic.start.static]

Change paragraph 2:

- […][*Note*: The dynamic initialization of non-~~local~~block variables is described in [basic.start.dynamic]; that of ~~local~~ static block variables is described in [stmt.dcl]. — *end note*]

Change the note in paragraph 3:

- As a consequence, if the initialization of an object `obj1` refers to an object `obj2`~~ of namespace scope~~ potentially requiring dynamic initialization and defined later in the same translation unit, it is unspecified whether the value of `obj2` used will be the value of the fully initialized `obj2` (because `obj2` was statically initialized) or will be the value of `obj2` merely zero-initialized. […]

Dynamic initialization of non-~~local~~block variables #[basic.start.dynamic]

Replace all occurrences of “non-local variable” with “non-block variable” (including in the name of the subclause).

#[basic.start.term]

Change paragraph 3:

- […] If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block~~-scope object~~ variable with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, the function `std::terminate` is called ([except.terminate]).

Change paragraph 4:

- If a function contains a block~~-scope object~~ variable of static or thread storage duration that has been destroyed and the function is called during the destruction of an object with static or thread storage duration, the program has undefined behavior if the flow of control passes through the definition of the previously destroyed block~~-scope object~~ variable. [*Note*: Likewise, the behavior is undefined if the block~~-scope object~~ variable is used indirectly (~~i.e.~~*e.g.*, through a pointer) after its destruction.*— end note*]

#[expr]

#[expr.prim]

#[expr.prim.this]

Change paragraph 1:

- The keyword `this` names a pointer to the object for which a non-static member function ([class.~~this~~mfct.non-static]) is invoked or a non-static data member’s initializer ([class.mem]) is evaluated.

Insert before paragraph 2:

- The *current class* at a program point is the class associated with the innermost class scope containing that point. [*Note*: A *lambda-expression* does not introduce a class scope. — *end note*]

Change paragraph 2:

- If a declaration declares a member function or member function template of a class `X`, the expression `this` is a prvalue of type "pointer to *cv-qualifier-seq* `X`" wherever `X` is the current class between the optional *cv-qualifier-seq* and the end of the *function-definition*, *member-declarator*, or *declarator*. It ~~shall not appear before the optional *cv-qualifier-seq* and it~~ shall not appear within the declaration of a static member function of the current class (although its type and value category are defined within a static member function as they are within a non-static member function). [*Note*: This is because declaration matching does not occur until the complete declarator is known. — *end note*] [*Note*: In a *trailing-return-type*, the class being defined is not required to be complete for purposes of class member access ([expr.ref]). Class members declared later are not visible. [*Example*:

  […]

  — *end example*] — *end note*]

Change paragraph 3:

- Otherwise, if a *member-declarator* declares a non-static data member ([class.mem]) of a class `X`, the expression `this` is a prvalue of type "pointer to `X`" wherever `X` is the current class within the optional default member initializer ([class.mem]). ~~It shall not appear elsewhere in the *member-declarator*.~~

#[expr.prim.id]

Insert before paragraph 2:

- If an *id-expression* *E* denotes a member *M* of an anonymous union ([class.union.anon]) *U*:

  1. If *U* is a non-static data member, *E* refers to *M* as a member of the lookup context of the terminal name of *E* (after any transformation to a class member access expression ([class.mfct.non-static])). [*Example*: `o.x` is interpreted as `o.`*u*`.x`, where *u* names the anonymous union member. — *end example*]
  2. Otherwise, *E* is interpreted as a class member access ([expr.ref]) that designates the member subobject *M* of the anonymous union variable for *U*. [*Note*: Under this interpretation, *E* no longer denotes a non-static data member. — *end note*][*Example*: `N::x` is interpreted as `N::`*u*`.x`, where *u* names the anonymous union variable. — *end example*]

#[expr.prim.id.unqual]

Insert before paragraph 2:

- A *component name* of an *unqualified-id* *U* is *U* if it is a name or the component name of the *template-id* or *type-name* of *U*, if any. [*Note*: Other constructs that contain names to look up can have several component names ([expr.prim.id.qual], [dcl.type.simple], [dcl.type.elab], [dcl.mptr], [namespace.udecl], [temp.param], [temp.names], [temp.res]). — *end note*] The *terminal name* of a construct is the component name of that construct that appears lexically last.

Change paragraph 2:

- The result is the entity denoted by the ~~identifier~~ *unqualified-id* ([basic.lookup.unqual]). If the entity is a local entity […]

#[expr.prim.id.qual]

Insert before paragraph 1:

- The component names of a *qualified-id* are those of its *nested-name-specifier* and *unqualified-id*. The component names of a *nested-name-specifier* are its *identifier* (if any) and those of its *type-name*, *namespace-name*, *simple-template-id*, and/or *nested-name-specifier*.

- A *nested-name-specifier* is *declarative* if it is part of

  1. a *class-head-name*,
  2. an *enum-head-name*,
  3. a *qualified-id* that is the *id-expression* of a *declarator-id*, or
  4. a declarative *nested-name-specifier*.

  A declarative *nested-name-specifier* shall not have a *decltype-specifier*. A declaration that uses a declarative *nested-name-specifier* shall be a friend declaration or inhabit a scope that contains the entity being redeclared or specialized.

Change paragraph 1:

- The *nested-name-specifier* `::` nominates the global namespace. ~~The type denoted by~~ A *nested-name-specifier* with a *decltype-specifier* ~~in a *nested-name-specifier*~~ nominates the type denoted by the *decltype-specifier*, which shall be a class or enumeration type. If a *nested-name-specifier* *N* is declarative and has a *simple-template-id* with a template argument list *A* that involves a template parameter, let *T* be the template nominated by *N* without *A*. *T* shall be a class template.

  1. If *A* is the template argument list ([temp.arg]) of the corresponding *template-head* *H* ([temp.mem]), *N* nominates the primary template of *T*; *H* shall be equivalent to the *template-head* of *T* ([temp.over.link]).
  2. Otherwise, *N* nominates the partial specialization ([temp.class.spec]) of *T* whose template argument list is equivalent to *A* ([temp.over.link]); the program is ill-formed if no such partial specialization exists.

  Any other *nested-name-specifier* nominates the entity denoted by its *type-name*, *namespace-name*, *identifier*, or *simple-template-id*. If the *nested-name-specifier* is not declarative, the entity shall not be a template.

Insert before paragraph 2:

- A *qualified-id* shall not be of the form *nested-name-specifier* `template`$_{opt}$ `~` *decltype-specifier* nor of the form *decltype-specifier* `::` `~` *type-name*.

Change paragraph 2:

- ~~A *nested-name-specifier* that denotes a class, optionally followed by the keyword `template` ([temp.names]), and then followed by the name of a member of either that class ([class.mem]) or one of its base classes ([class.derived]), is a *qualified-id*; [class.qual] describes name lookup for class members that appear in *qualified-ids*.~~ The result of a *qualified-id* is the ~~member~~entity it denotes ([basic.lookup.qual]). The type of the ~~result~~expression is the type of the ~~member~~result. The result is an lvalue if the member is a function, a variable, a structured binding ([dcl.struct.bind]), a static member function, or a data member and a prvalue otherwise. ~~[*Note*: A class member can be referred to using a *qualified-id* at any point in its potential scope ([basic.scope.class]). — *end note*] Where *type-name* `::` ~ *type-name* is used, the two *type-names* shall refer to the same type (ignoring cv-qualifications); this notation denotes the destructor of the type so named ([expr.prim.id.dtor]). The *unqualified-id* in a *qualified-id* shall not be of the form `~` *decltype-specifier*.~~

Remove paragraphs 3 through 5:

- The *nested-name-specifier* `::` names the global namespace. […]

- A *nested-name-specifier* that denotes an enumeration ([dcl.enum]), followed by the name of an enumerator of that enumeration, […]

- In a *qualified-id*, if the *unqualified-id* is a *conversion-function-id*, […]

#[expr.prim.lambda]

#[expr.prim.lambda.closure]

Change paragraph 4:

- […] ~~[*Note*: Names referenced in the *lambda-declarator* are looked up in the context in which the *lambda-expression* appears. — *end note*]~~ [*Example*:

  […]

  — *end example*]

Change paragraph 12:

- The *lambda-expression*'s *compound-statement* yields the *function-body* ([dcl.fct.def]) of the function call operator, but ~~for purposes of name lookup ([basic.lookup]), determining the type and value of `this` ([class.this]) and transforming *id-expressions* referring to non-static class members into class member access expressions using `(*this)` ([class.mfct.non-static]), the *compound-statement* is considered in the context of the *lambda-expression*~~it is not within the scope of the closure type. [*Example*:

  […]

  — *end example*] Further, a variable `__func__` is implicitly defined at the beginning of the *compound-statement* of the *lambda-expression*, with semantics as described in [dcl.fct.def.general].

#[expr.prim.lambda.capture]

Change paragraph 4:

- The *identifier* in a *simple-capture* ~~is looked up using the usual rules for unqualified name lookup ([basic.lookup.unqual]); each such lookup~~ shall ~~find~~denote a a local entity ([basic.lookup.unqual]). The *simple-capture*s `this` and `* this` denote the local entity `*this`. An entity that is designated by a *simple-capture* is said to be *explicitly captured*.

Change paragraph 6:

- An *init-capture* inhabits the scope of the *lambda-expression*'s *compound-statement*. An *init-capture* without ellipsis behaves as if it declares and explicitly captures a variable of the form "`auto` *init-capture* `;`" ~~whose declarative region is the *lambda-expression*'s *compound-statement*~~, except that:

  […]

Change paragraph 8:

- An entity is *captured* if it is captured explicitly or implicitly. An entity captured by a *lambda-expression* is odr-used ([basic.def.odr]) ~~in the scope containing~~by the *lambda-expression*. […]

Change paragraph 17:

- A *simple-capture* containing an ellipsis is a pack expansion ([temp.variadic]). An *init-capture* containing an ellipsis is a pack expansion that ~~introduces~~declares an *init-capture* pack ([temp.variadic]) ~~whose declarative region is the *lambda-expression*'s *compound-statement*~~. [*Example*:

  […]

  — *end example*]

#[expr.prim.req]

Change the grammar in paragraph 1:

- […]

  requirement-parameter-list:

    (*parameter-declaration-clause*~~*opt*~~)

  […]

Change paragraph 4:

- A *requires-expression* may introduce local parameters using a *parameter-declaration-clause* ([dcl.fct]). A local parameter of a *requires-expression* shall not have a default argument. ~~Each name introduced by a local parameter is in scope from the point of its declaration until the closing brace of the *requirement-body*.~~ These parameters have no linkage, storage, or lifetime; they are only used as notation for the purpose of defining *requirement*s. […]

Remove paragraph 5:

- The *requirement-body* contains a sequence of *requirement*s. […]

#[expr.post]

#[expr.call]

Change paragraph 7:

- […] If the function is a non-static member function, the `this` parameter of the function ([~~class~~expr.prim.this]) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion ([expr.cast]). […][*Example*: The access of the constructor, conversion functions or destructor is checked at the point of call in the calling function. If a constructor or destructor for a function parameter throws an exception, the search for a handler starts in ~~the scope of~~ the calling function; in particular, if the function called has a *function-try-block* ([except.pre]) with a handler that could handle the exception, this handler is not considered. — *end example*]

#[expr.ref]

Change paragraph 1:

- A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword `template` ~~*([temp.names])*~~, and then followed by an *id-expression*, is a postfix expression. The postfix expression before the dot or arrow is evaluated;[…] the result of that evaluation, together with the *id-expression*, determines the result of the entire postfix expression. [*Note*: If the keyword `template` is used, the following unqualified name is considered to refer to a template ([temp.names]). If a *simple-template-id* results and is followed by a `::`, the *id-expression* is a *qualified-id*. — *end note*]

Change paragraph 4:

- Otherwise, the object expression shall be of class type. The class type shall be complete unless the class member access appears in the definition of that class. [*Note*: ~~If the class is incomplete, lookup in~~ The program is ill-formed if the result differs from that when the class is complete ~~class type is required to refer to the same declaration~~ ([~~basic.scope.~~class.member.lookup]). — *end note*] ~~The *id-expression* shall name a member of the class or of one of its base classes. [*Note*: Because the name of a class is inserted in its class scope ([class]), the name of a class is also considered a nested member of that class. — *end note*]~~ [*Note*: [basic.lookup.~~classref~~qual] describes how names are looked up after the `.` and `->` operators. — *end note*]

Change bullet (6.3):

- If `E2` is ~~a (possibly overloaded) member function~~ an overload set, function overload resolution ([over.match]) is used to select the function to which `E2` refers. The type of `E1.E2` is the type of `E2` and `E1.E2` refers to the function referred to by `E2`.

#[expr.unary]

#[expr.unary.op]

Change paragraph 6:

- [*Note*: The address of an overload~~ed function~~set ([over]) can be taken only in a context that uniquely determines which ~~version of the overloaded~~ function is referred to (see [over.over]). Since the context might determine whether the operand is a static or non-static member function, the context can also affect whether the expression has type “pointer to function” or “pointer to member function”. — *end note*]

#[expr.await]

Change paragraph 2:

- […] An *await-expression* shall not appear in the initializer of a block ~~scope~~ variable with static or thread storage duration. A context within a function where an *await-expression* can appear is called a *suspension context* of the function.

Change bullet (3.2):

- ~~*a is the cast-expression* if~~Unless the *await-expression* was implicitly produced by a *yield-expression* ([expr.yield]), an initial suspend point, or a final suspend point ([dcl.fct.def.coroutine])~~. Otherwise~~, a search is performed for the ~~*unqualified-id*~~name `await_transform` ~~is looked up with~~in the scope of ~~`P` by class member access lookup~~ ([~~basic.~~class.member.lookup~~.classref~~])~~, and it~~. If this ~~lookup~~search is performed and finds at least one declaration, then *a* is *p*`.await_transform(`*cast-expression*`)`; otherwise, *a* is the *cast-expression*.

#[expr.new]

Change paragraph 11:

- If the *new-expression* does not begin~~s~~ with a unary `::` operator~~, the allocation function's name is looked up in the global scope. Otherwise, if~~ and the allocated type is a class type `T` or array thereof, a search is performed for the allocation function's name ~~is looked up~~ in the scope of `T` ([class.member.lookup]). ~~If this lookup fails to find the name, or if the allocated type is not a class type~~Otherwise, or if nothing is found, the allocation function's name is looked up by searching for it in the global scope.

Change paragraph 24:

- If the *new-expression* creates an object or an array of objects of class type, access and ambiguity control are done for the allocation function, the deallocation function ([~~class.free~~basic.stc.dynamic.deallocation]), and the constructor ([class.ctor]) selected for the initialization (if any). If the *new-expression* creates an array of objects of class type, the destructor is potentially invoked ([class.dtor]).

Change paragraph 26:

- If the *new-expression* does not begin~~s~~ with a unary `::` operator~~, the deallocation function's name is looked up in the global scope. Otherwise, if~~ and the allocated type is a class type `T` or an array thereof, a search is performed for the deallocation function's name ~~is looked up~~ in the scope of `T`. ~~If this lookup fails to find the name, or if the allocated type is not a class type or array thereof~~Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope.

#[expr.delete]

Change paragraph 8:

- If a deallocation function is called, it is `operator delete` for a single-object delete expression or `operator delete[]` for an array delete expression. [*Note*: An implementation provides default definitions of the global deallocation functions~~ `operator delete` for non-arrays~~ ([new.delete.single])~~ and `operator delete[]` for arrays~~ (, [new.delete.array]). A C++ program can provide alternative definitions of these functions ([replacement.functions]), and/or class-specific versions ([class.free]). — *end note*]

Change paragraph 9 (incorporating some of [class.free]/4):

- ~~When~~If the keyword `delete` in a *delete-expression* is not preceded by the unary `::` operator~~, the deallocation function's name is looked up in global scope. Otherwise, the lookup considers class-specific deallocation functions ([class.free]).~~ and the type of the operand is a pointer to a (possibly cv-qualified) class type `T`:

  1. If `T` has a virtual destructor, the deallocation function is the one selected at the point of definition of the dynamic type's virtual destructor ([class.dtor]).
  2. Otherwise, a search is performed for the deallocation function's name in the scope of `T`.

  ~~If no class-specific deallocation function~~Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope. In any case, any declarations other than of usual deallocation functions ([basic.stc.dynamic.deallocation]) are discarded. [*Note*: If only a placement deallocation function is found in a class, the program is ill-formed because the lookup set is empty ([basic.lookup]). — *end note*]

Change paragraph 10:

- If ~~deallocation function lookup finds~~ more than one ~~usual~~ deallocation function is found, the function to be called is selected as follows:

  1. If any of the deallocation functions is a destroying operator delete, all deallocation functions that are not destroying operator deletes are eliminated from further consideration.
  2. If the type has new-extended alignment, a function with a parameter of type `std::align_val_t` is preferred; otherwise a function without such a parameter is preferred. If any preferred functions are found, all non-preferred functions are eliminated from further consideration.
  3. If exactly one function remains, that function is selected and the selection process terminates.
  4. If the deallocation functions ~~have~~belong to a class scope, the one without a parameter of type `std::size_t` is selected.
  5. If the type is complete and if, for an array delete expression only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multi-dimensional) array thereof, the function with a parameter of type `std::size_t` is selected.
  6. Otherwise, it is unspecified whether a deallocation function with a parameter of type `std::size_t` is selected.

#[expr.const]

Change paragraph 13:

- An expression or conversion is in an *immediate function context* if it is potentially evaluated and its innermost enclosing non-block scope is a function parameter scope of an immediate function. An expression or conversion is an *immediate invocation* if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

Change bullet (15.7):

- a variable ~~whose name appears as~~named by a potentially constant evaluated expression that is either a constexpr variable or is of non-volatile const-qualified integral type or of reference type.

#[stmt.stmt]

#[stmt.pre]

Remove paragraph 5:

- [*Note*: A name introduced in a *selection-statement* or *iteration-statement* outside of any substatement is in scope […] — *end note*]

Change paragraph 7:

- If a *condition* can be syntactically resolved as either an expression or ~~the~~a declaration~~ of a block scope name~~, it is interpreted as ~~a declaration~~the latter.

#[stmt.label]

Change paragraph 1:

- […]

  The optional *attribute-specifier-seq* appertains to the label. ~~An *identifier label* declares the identifier.~~ The only use of a~~n identifier~~ label with an *identifier* is as the target of a `goto`. ~~The scope of a label is the function in which it appears. L~~No two labels ~~shall not be redeclared with~~in a function shall have the same *identifier*. A label can be used in a `goto` statement before its ~~declaration~~introduction by a *labeled-statement*. ~~Labels have their own name space and do not interfere with other identifiers. [*Note*: A label may have the same name as another declaration in the same scope or a *template-parameter* from an enclosing scope. Unqualified name lookup ([basic.lookup.unqual]) ignores labels. — *end note*]~~

#[stmt.block]

Change paragraph 1:

- […]

  [*Note:* A compound statement defines a block scope ([basic.scope]). ~~[*Note:*~~ A declaration is a *statement* ([stmt.dcl]). — _end note_]

#[stmt.select]

Change paragraph 2:

- ~~The substatement in a~~[*Note*: Each *selection-statement* (and each substatement~~, in the `else` form of the `if` statement~~ of a *selection-statement*~~)~~ ~~implicitly defines~~has a block scope ([basic.scope.block]). *— end note*] ~~If the substatement in a *selection-statement* is a single statement and not a *compound-statement*, it is as if it was rewritten to be a *compound-statement* containing the original substatement. [*Example:*~~

  ~~[…]~~

  ~~can be equivalently rewritten as~~

  ~~[…]~~

  ~~Thus after the `if` statement, `i` is no longer in scope. — *end example*]~~

#[stmt.if]

Change paragraph 3:

- […]

  except that ~~names declared in~~ the *init-statement* ~~are~~is in the same ~~declarative region~~scope as ~~those declared in~~ the *condition*.

#[stmt.switch]

Change paragraph 7 as for [stmt.if]/3.

#[stmt.iter]

Remove paragraph 3:

- If a name introduced in an *init-statement* or *for-range-declaration* is redeclared […]

#[stmt.while]

Change paragraph 2:

- ~~When the condition of a `while` statement is a declaration, the scope of the variable that is declared extends from its point of declaration ([basic.scope.pdecl]) to the end of the `while` *statement*.~~ A `while` statement is equivalent to

  […]

  [*Note*: The variable created in the condition is destroyed and created with each iteration of the loop. [*Example*:

  […]

  In the while-loop, the constructor and destructor are each called twice, once for the condition that succeeds and once for the condition that fails. — *end example*] — *end note*]

#[stmt.for]

Change paragraph 1 as for [stmt.if]/3 (up to a comma instead of a period).

Remove paragraph 3 (whose example is moved to [basic.scope.block]):

- If the *init-statement* is a declaration, […]

#[stmt.ranged]

Change bullet (1.3.2):

- if the *for-range-initializer* is an expression of class type `C`~~, the *unqualified-ids*~~ and searches in the scope of `C` ([class.member.lookup]) for the names `begin` and `end` ~~are looked up in the scope of `C` as if by class member access lookup ([basic.lookup.classref]), and if both~~each find at least one declaration, *begin-expr* and *end-expr* are *`range`*`.begin()` and *`range`*`.end()`, respectively;

Change bullet (1.3.3):

- otherwise, *begin-expr* and *end-expr* are `begin(`*`range`*`)` and `end(`*`range`*`)`, respectively, where `begin` and `end` ~~are~~undergo argument-dependent look~~ed~~up ~~in the associated namespaces~~ ([basic.lookup.argdep]). [*Note*: Ordinary unqualified lookup ([basic.lookup.unqual]) is not performed. — *end note*]

#[stmt.jump]

Change paragraph 2:

- [*Note:* On exit from a scope (however accomplished), objects with automatic storage duration ([basic.stc.auto]) that have been constructed in that scope are destroyed in the reverse order of their construction. ~~[*Note:* For temporaries, see [class.temporary].~~ ~~— *end note*] Transfer out of a loop, out of a block, or back past an initialized variable with automatic storage duration involves the destruction of objects with automatic storage duration that are in scope at the point transferred from but not at the point transferred to. (See [stmt.dcl] for transfers into blocks). [*Note:*~~ However, the program can be terminated (by calling `std::exit()` or `std::abort()` ([support.start.term]), for example) without destroying objects with automatic storage duration. — *end note*][*Note*: A suspension of a coroutine ([expr.await]) is not considered to be an exit from a scope. — *end note*]

#[stmt.dcl]

Change paragraph 1:

- A declaration statement introduces one or more new ~~identifiers~~names into a block; it has the form

  declaration-statement:

  block-declaration

  [*Note:* If an identifier introduced by a declaration was previously declared in an outer block, the outer declaration is hidden for the remainder of the block ([basic.lookup.unqual]), after which it resumes its force. — *end note*]

Change paragraph 2:

- A v~~V~~ariable~~s~~ with automatic storage duration ([basic.stc.auto]) is *active* everywhere in the scope to which it belongs after its *init-declarator*~~are initialized each time their *declaration-statement* is executed~~. ~~Variables with automatic storage duration declared in the block are destroyed on exit from the block ([stmt.jump]).~~

Change paragraph 3:

- ~~It is possible to transfer into a block, but not in a way that bypasses declarations with initialization (including ones in *conditions* and *init-statements*). A program that~~Upon each transfer of control (including sequential execution of statements) within a function from point *P* to point *Q*, all variables that are active at *P* and not at *Q* are destroyed in the reverse order of their construction. Then, all variables that are active at *Q* but not at *P* are initialized in declaration order; unless all such variables have vacuous initialization ([basic.life]), the transfer of control shall not be a jump~~s~~[…] ~~from a point where a variable with automatic storage duration is not in scope to a point where it is in scope is ill-formed unless the variable has vacuous initialization ([basic.life])~~. ~~In such a case, the variables with vacuous initialization are constructed in the order of their declaration.~~ When a *declaration-statement* is executed, *P* and *Q* are the points immediately before and after it; when a function returns, *Q* is after its body. [*Example*:

[…]

— end example]

Change paragraph 4:

- Dynamic initialization of a block-~~scope~~ variable with static storage duration ([basic.stc.static]) or thread storage duration ([basic.stc.thread]) is performed the first time control passes through its declaration; such a variable is considered initialized upon the completion of its initialization. […]

Change paragraph 5:

- A~~n block-scope~~ object associated with a block variable with static or thread storage duration will be destroyed if and only if it was constructed. [*Note*: [basic.start.term] describes the order in which ~~block-scope~~such objects ~~with static and thread storage duration~~ are destroyed. *— end note*]

#[stmt.ambig]

Change paragraph 3:

- […] Disambiguation precedes parsing, and a statement disambiguated as a declaration may be an ill-formed declaration. If, during parsing, lookup finds that a name in a template ~~parameter is bound differently than it would be bound during a trial parse~~argument is bound to (part of) the declaration being parsed, the program is ill-formed. No diagnostic is required. ~~[*Note*: This can occur only when the name is declared earlier in the declaration. — *end note*]~~ [*Example*:

  […]

  — end example]

#[dcl.dcl]

#[dcl.pre]

Change paragraph 4:

- ~~A declaration occurs in a scope ([basic.scope]); the scope rules are summarized in [basic.lookup]. A~~Certain declaration~~s that declares a function or defines a class, namespace, template, or function also has~~ contain one or more scopes ~~nested within it~~ ([basic.scope.scope]). ~~These nested scopes, in turn, can have declarations nested within them.~~ Unless otherwise stated, utterances in [dcl.dcl] about components in, of, or contained by a declaration or subcomponent thereof refer only to those components of the declaration that are *not* nested within scopes nested within the declaration.

Change paragraph 5:

- In a *simple-declaration*, the optional *init-declarator-list* can be omitted only when declaring a class ([class.pre]) or enumeration ([dcl.enum]), that is, when the *decl-specifier-seq* contains either a *class-specifier*, an *elaborated-type-specifier* with a *class-key* ([class.name]), or an *enum-specifier*. In these cases and whenever a *class-specifier* or *enum-specifier* is present in the *decl-specifier-seq*, the identifiers in these specifiers are ~~among the names being~~also declared ~~by the declaration~~ (as *class-name*s, *enum-name*s, or *enumerator*s, depending on the syntax). In such cases, the *decl-specifier-seq* shall (re)introduce one or more names into the program~~, or shall redeclare a name introduced by a previous declaration~~. [*Example*:

  […]

  — end example]

Change paragraph 9:

- ~~Each *init-declarator* in the *init-declarator-list* contains exactly one *declarator-id*, which is the~~If a *declarator-id* is a name, the ~~declared by that~~ *init-declarator* and (hence) ~~one of the names declared by~~ the declaration introduce that name. [*Note*: Otherwise, the *declarator-id* is a *qualified-id* or names a destructor or its *unqualified-id* is a *template-id* and no name is introduced. *— end note*] The *defining-type-specifier*s ([dcl.type]) in the *decl-specifier-seq* and the recursive *declarator* structure ~~of the *init-declarator*~~ describe a type ([dcl.meaning]), which is then associated with the ~~name being declared by the *init-declarator*~~*declarator-id*.

Change paragraph 10:

- If the *decl-specifier-seq* contains the `typedef` specifier, the declaration is called a *typedef declaration*, each *declarator-id* must be an *identifier* and ~~the name of each *init-declarator*~~ is declared to be a *typedef-name*, synonymous with its associated type ([dcl.typedef]). If the *decl-specifier-seq* contains no `typedef` specifier, the declaration is called a *function declaration* if the type associated with ~~the name~~a *declarator-id* is a function type ([dcl.fct]) and an *object declaration* otherwise.

#[dcl.spec]

#[dcl.stc]

Change paragraph 1:

- […] If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* or *member-declarator-list* of the declaration shall not be empty (except for an anonymous union declared in a ~~named~~ namespace ~~or in the global namespace, which shall be declared `static`~~scope ([class.union.anon])). The *storage-class-specifier* applies to the name declared by

each *init-declarator* in the list and not to any names declared by other specifiers. [*Note*: See [temp.expl.spec] and [temp.explicit] for restrictions in explicit specializations and explicit instantiations, respectively. — *end note*]

Change paragraph 6:

- ~~The linkages implied by successive~~ All declarations for a given entity shall ~~agree~~ give its name the same linkage. ~~That is, within a given scope, each declaration declaring the same variable name or the same overloading of a function name shall imply the same linkage.~~ [*Note*: The linkage given by some declarations is affected by previous declarations. Overloads are distinct entities. — *end note*] [*Example*:

  […]

  — *end example*]

#[dcl.typedef]

Change paragraph 2:

- A *typedef-name* can also be introduced by an *alias-declaration*. The *identifier* following the `using` keyword is not looked up; it becomes a *typedef-name* and the optional *attribute-specifier-seq* following the *identifier* appertains to that *typedef-name*. […]

Remove paragraphs 3 through 7:

- In a given non-class scope, a `typedef` specifier can be used to redeclare […]

- In a given class scope, a `typedef` specifier can be used to redeclare […]

- If a `typedef` specifier is used to redeclare […]

- In a given scope, a `typedef` specifier shall not be used to redeclare […]

- Similarly, in a given scope, a class or enumeration shall not be declared […]

#[dcl.inline]

Change paragraph 6:

- [*Note*: An inline function or variable with external or module linkage ~~has the same address~~ can be defined in ~~all~~ multiple translation units ([basic.def.odr]), but is one entity with one address. A type or `static` ~~local~~ variable defined in ~~an inline~~ the body of such a function ~~with external or module linkage always refers to the same object~~ is therefore a single entity. ~~A type defined within the body of an inline function with external or module linkage is the same type in every translation unit.~~ — *end note*]

#[dcl.type]

#[dcl.type.simple]

Insert before paragraph 2:

- The component names of a *simple-type-specifier* are those of its *nested-name-specifier*, *type-name*, *simple-template-id*, *template-name*, and/or *type-constraint* (if it is a *placeholder-type-specifier*). The component name of a *type-name* is the first name in it.

#[dcl.type.elab]

Insert before paragraph 1:

- The component names of an *elaborated-type-specifier* are its *identifier* (if any) and those of its *nested-name-specifier* and *simple-template-id* (if any).

Change paragraph 1:

- ~~An *attribute-specifier-seq* shall not appear in an *elaborated-type-specifier* unless the latter is the sole constituent of a declaration.~~ If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization ([temp.expl.spec]), an explicit instantiation ([temp.explicit]) or it has one of the following forms:

  class-key attribute-specifier-seq$_{opt}$ *identifier* ;
  class-key attribute-specifier-seq$_{opt}$ *simple-template-id* ;
  ~~`friend` *class-key* `::`$_{opt}$ *identifier* ;~~
  ~~`friend` *class-key* `::`$_{opt}$ *simple-template-id* ;~~
  ~~`friend` *class-key nested-name-specifier identifier* ;~~
  ~~`friend` *class-key nested-name-specifier* `template`$_{opt}$ *simple-template-id* ;~~

  In the first case, it declares the *identifier* as a *class-name*. The second case shall appear only in an *explicit-specialization* ([temp.expl.spec]) or in a *template-declaration* (where it declares a partial specialization ([temp.decls])). T~~t~~he *attribute-specifier-seq*, if any, appertains to the class or template being declared~~; the attributes in the *attribute-specifier-seq* are thereafter considered attributes of the class whenever it is named~~.

Insert before paragraph 2:

- Otherwise, the *elaborated-type-specifier* shall not have an *attribute-specifier-seq*. If it contains a *identifier* but no *nested-name-specifier* and (unqualified) lookup for the *identifier* finds nothing, it shall not be introduced by the `enum` keyword and declares the *identifier* as a *class-name*. Its target scope is the nearest enclosing namespace or block scope.

- If it appears with the `friend` specifier as an entire declaration, the declaration shall have one of the following forms:

  `friend` *class-key nested-name-specifier*$_{opt}$ *identifier* `;`
  `friend` *class-key simple-template-id* `;`
  `friend` *class-key nested-name-specifier* `template`$_{opt}$ *simple-template-id* `;`

  Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the target scope; no name is bound. [*Note*: A *using-directive* in the target scope is ignored if it refers to a namespace not contained by that scope. [basic.lookup.elab] describes how name lookup proceeds in an *elaborated-type-specifier*. — *end note*]

Change paragraph 2:

- [*Note*: ~~[basic.lookup.elab] describes how name lookup proceeds for the *identifier* in a~~ An *elaborated-type-specifier* may be used to refer to a previously declared *class-name* or *enum-name* even if the name has been hidden by a non-type declaration. — *end note*] If the *identifier* or *simple-template-id* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* ([dcl.type.simple]). […]

#[dcl.type.decltype]

Change bullet (1.3):

- otherwise, if `e` is an unparenthesized *id-expression* or an unparenthesized class member access ([expr.ref]), `decltype(e)` is the type of the entity named by `e`. If there is no such entity~~, or if `e` names a set of overloaded functions~~, the program is ill-formed;

#[dcl.spec.auto]

Change paragraph 11:

- If ~~the name of~~ a~~n entity~~ variable or function with an undeduced placeholder type ~~appears in~~is named by an expression ([basic.def.odr]), the program is ill-formed. Once a non-discarded `return` statement has been seen in a function, however, the return type deduced from that statement can be used in the rest of the function, including in other `return` statements. [*Example*:

  […]

  — *end example*]

Change paragraph 13:

- If a function or function template has a declared return type that uses a placeholder type, r~~R~~edeclarations or specializations (respectively) of ~~a function or function template with a declared return type that uses a placeholder type~~it shall ~~also~~ use that placeholder, not a deduced type~~. Similarly, redeclarations or specializations of a function or function template with a declared return type that does not use a placeholder type~~; otherwise, they shall not use a placeholder.

```
[…]
int f();                                    // error: ~~cannot be overloaded with~~ auto ~~f()~~and int don't match
[…]
```

#[dcl.decl]

Change paragraph 1:

- A declarator declares a single variable, function, or type, within a declaration. The *init-declarator-list* appearing in a ~~declaration~~*simple-declaration* is a comma-separated sequence of declarators, each of which can have an initializer.

  […]

Change paragraph 2:

- ~~The three components of a *simple-declaration* are the attributes ([dcl.attr]), the specifiers (*decl-specifier-seq*; [dcl.spec]) and the declarators (*init-declarator-list*).~~ In all contexts, a *declarator* is interpreted as given below. Where an *abstract-declarator* can be used (or omitted) in place of a *declarator* ([dcl.fct], [except.pre]), it is as if a unique identifier were included in the appropriate place ([dcl.name]). The preceding specifiers indicate the type, storage class or other properties of the entity or entities being declared. ~~The~~Each declarator~~s~~ specif~~y~~ies ~~the names of these~~one entit~~ies~~y and (optionally) names it and/or modif~~y~~ies the type of the specifiers with operators such as `*` (pointer to) and `()` (function returning). ~~Initial values~~[*Note*: An *init-declarator* can also ~~be~~ specif~~ied in a declarator:~~y an initializer~~s are discussed in~~ ([dcl.init]~~ and [class.init]~~). — *end note*]

Change paragraph 3:

- Each *init-declarator* or *member-declarator* in a declaration is analyzed separately as if it ~~was~~were in a declaration by itself. [*Note*: A declaration with several declarators is usually equivalent to the corresponding sequence of declarations each with a single declarator. That is,

```
T D1, D2, ... Dn;
```

is usually equivalent to

```
T D1; T D2; ... T Dn;
```

where `T` is a *decl-specifier-seq* and each `Di` is an *init-declarator* or *member-declarator*. […]

— *end note*]

#[dcl.meaning]

Change paragraph 1:

- A declarator contains exactly one *declarator-id*; it names the ~~identifier~~entity that is declared. ~~An~~If the *unqualified-id* occurring in a *declarator-id* ~~shall be~~is a ~~simple *identifier* except for the declaration of some special~~*template-id*, the declarator shall appear in the *declaration* of a *template-declaration* ([temp.decls]), *explicit-specialization* ([temp.expl.spec]), or *explicit-instantiation* ([temp.explicit]). [*Note*: An *unqualified-id* that is not an *identifier* is used to declare certain functions (~~[class.ctor],~~ [class.conv.fct], [class.dtor], [over.oper], [over.literal]) ~~and for the declaration of template specializations or partial specializations ([temp.spec])~~. — *end note*] ~~When the *declarator-id* is qualified, the declaration shall refer to a previously declared member of the class or namespace to which the qualifier refers (or, in the case of a namespace, of an element of the inline namespace set of that namespace ([namespace.def])) or to a specialization thereof; the member shall not merely have been introduced by a *using-declaration* in the scope of the class or namespace nominated by the *nested-name-specifier* of the *declarator-id*. The *nested-name-specifier* of a qualified *declarator-id* shall not begin with a *decltype-specifier*. [*Note*: If the qualifier is the global `::` scope resolution operator, the *declarator-id* refers to a name declared in the global namespace scope. — *end note*]~~ The optional *attribute-specifier-seq* following a *declarator-id* appertains to the entity that is declared.

Insert before paragraph 2, including the examples from [namespace.memdef]/2 and from [basic.link]/7, with changes as marked for the latter:

- If the declaration is a friend declaration:

  1. The *declarator* binds no name.
  2. If the *id-expression E* in a *declarator-id* is a *qualified-id* or a *template-id*:
     1. If the friend declaration is not a template declaration, then in the lookup for the terminal name of *E*:
        1. if the *unqualified-id* in *E* is a *template-id*, all function declarations are discarded;
        2. otherwise, if the *declarator* corresponds ([basic.scope.scope]) to any declaration found of a non-template function, all function template declarations are discarded;
        3. each remaining function template is replaced with the specialization chosen by deduction from the friend declaration ([temp.deduct.decl]) or discarded if deduction fails.
     2. The *declarator* shall correspond to one or more declarations found by the lookup; they shall all have the same target scope, and the target scope of the *declarator* is that scope.
  3. Otherwise, the terminal name of *E* is not looked up. The declaration's target scope is the innermost enclosing namespace scope; if the declaration is contained by a block scope, it shall correspond to a declaration that inhabits the innermost block scope.

- Otherwise:

  1. If the *id-expression* in a *declarator-id* is a *qualified-id Q*, let *S* be its lookup context ([basic.lookup.qual]); the declaration shall inhabit a namespace scope.
  2. Otherwise, let *S* be the entity associated with the scope inhabited by the *declarator*.
  3. If the *declarator* declares an explicit instantiation or a partial or explicit specialization, the *declarator* binds no name. If it declares a class member, the terminal name of the *declarator-id* is not looked up; otherwise, only those lookup results that are nominable in *S* are considered when identifying any function template specialization being declared ([temp.deduct.decl]). [*Example*:

     ```
     namespace N {
       inline namespace O {
         template<class T> void f(T);   // #1
         template<class T> void g(T) {}
       }
       namespace P {
         template<class T> void f(T*);  // #2, more specialized than #1
         template<class> int g;
       }
       using P::f,P::g;
     }
     template<> void N::f(int*) {}      // OK: #2 is not nominable
     template void N::g(int);           // error: lookup is ambiguous
     ```

     — *end example*]

  4. Otherwise, the terminal name of the *declarator-id* is not looked up. If it is a qualified name, the *declarator* shall correspond to one or more declarations nominable in *S*; all the declarations shall have the same target scope and the target scope of the *declarator* is that scope. [*Example*:

     […] from [namespace.memdef]/2]

     — *end example*]

  5. If the declaration inhabits a block scope *S* and declares a function ([dcl.fct]) or uses the `extern` specifier, the declaration shall not be attached to a named module ([module.unit]); its target scope is the innermost enclosing namespace scope, but the name is bound in *S*. [*Example*:

```
namespace X {
  void p() {
    q();                        // error: q not yet declared
    extern void q();            // q is a member of namespace X
    extern void r();            // r is a member of namespace X
  }

  void middle() {
    q();                        // error: q not ~~yet declared~~found
  }

  void q() { /* ... */ }        // definition of X::q
}

void q() { /* ... */ }          // some other, unrelated q
void X::r() { /* ... */ } // error: r cannot be declared by qualified-id
```

— end example]

[*Drafting note*: Richard proposed making the restriction on use of a *qualified-id* grammatical. — *end drafting note*]

Change paragraph 2:

- A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, or `typedef` specifier or an *explicit-specifier* applies directly to each *declarator-id* in a~~n *init-declarator-list* or *member-declarator-list*~~ declaration; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

Change paragraph 3:

- Thus, (for each *declarator*) a declaration~~ of a particular identifier~~ has the form

  ```
  T D
  ```

  where `T` is of the form *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq* and `D` is a declarator. Following is a recursive procedure for determining the type specified for the contained *declarator-id* by such a declaration.

Change paragraph 5:

- In a declaration *attribute-specifier-seq*$_{opt}$ `T D` where `D` is an unadorned ~~identifier~~name the type of ~~this identifier~~the declared entity is "`T`".

#[dcl.ptr]

Change paragraph 1:

- In a declaration `T D` where `D` has the form

  `*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$ `D1`

  and the type of the ~~identifier~~contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", ~~then~~ the type of the ~~identifier of~~*declarator-id* in `D` is "*derived-declarator-type-list cv-qualifier-seq* pointer to `T`". […]

#[dcl.ref]

Change paragraph 1:

- In a declaration `T D` where `D` has either of the forms

  `&` *attribute-specifier-seq*$_{opt}$ `D1`
  `&&` *attribute-specifier-seq*$_{opt}$ `D1`

  and the type of the ~~identifier~~contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", ~~then~~ the type of the ~~identifier of~~*declarator-id* in `D` is "*derived-declarator-type-list* reference to `T`". […]

#[dcl.mptr]

Prepend paragraph:

- The component names of a *ptr-operator* are those of its *nested-name-specifier*, if any.

Change paragraph 1:

- In a declaration `T D` where `D` has the form

  nested-name-specifier `*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$ `D1`

  and the *nested-name-specifier* denotes a class, and the type of the ~~identifier~~contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", ~~then~~ the type of the ~~identifier of~~*declarator-id* in `D` is "*derived-declarator-type-list cv-qualifier-seq* pointer to member of class *nested-name-specifier* of type `T`". […]

#[dcl.array]

Change paragraph 8:

- Furthermore, if there is a ~~preceding~~reachable declaration of the entity ~~in~~that inhabits the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class. [*Example*:

  […]

  — *end example*]

#[dcl.fct]

Change paragraph 5, appending an example adapted from [over.load]/3:

- The type of a function is determined using the following rules. The type of each parameter (including function parameter packs) is determined from its own ~~*decl-specifier-seq and declarator*~~ *parameter-declaration* ([dcl.decl]). […][*Note*: This transformation does not affect the types of the parameters. For example, `int(*)(const int p, decltype(p)*)` and `int(*)(int, const int*)` are identical types. — *end note*] [*Example:*

```
void f(char*);              // #1
void f(char[]) {}           // defines #1
void f(const char*) {}      // OK: another overload
void f(char *const) {}      // error: redefines #1

void g(char(*)[2]);         // #2
void g(char[3][2]) {}       // defines #2
void g(char[3][3]) {}       // OK: another overload

void h(int x(const int));   // #3
void h(int (*)(int)) {}     // defines #3
```

  — end example]

Change paragraph 10:

- [*Note:* A single name can be used for several different functions in a single scope; this is function overloading ([over]). *— end note*] ~~All declarations for a function shall have equivalent return types, parameter-type-lists, and *requires-clauses* ([temp.over.link]).~~

Change paragraph 15:

- An identifier can optionally be provided as a parameter name; if present in a function definition ([dcl.fct.def]), it names a parameter. [*Note*: In particular, parameter names are also optional in function definitions and names used for a parameter in different declarations and the definition of a function need not be the same. ~~If a parameter name is present in a function declaration that is not a definition, it cannot be used outside of its function declarator because that is the extent of its potential scope ([basic.scope.param]).~~ — *end note*]

#[dcl.fct.default]

Change paragraph 4:

- For non-template functions, default arguments can be added in later declarations of a function ~~in~~that inhabit the same scope. Declarations ~~in~~that inhabit different scopes have completely distinct sets of default arguments. That is, declarations in inner scopes do not acquire default arguments from declarations in outer scopes, and vice versa. […] For a given inline function defined in different translation units, the accumulated sets of default arguments at the end of the translation units shall be the same; no diagnostic is required. If a friend declaration *D* specifies a default argument expression, that declaration shall be a definition and there shall be ~~the only~~no other declaration of the function or function template ~~in the translation unit~~which is reachable from *D* or from which *D* is reachable.

Change paragraph 5:

- The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics ([dcl.init]). The names in the default argument are ~~bound~~looked up, and the semantic constraints are checked, at the point where the default argument appears. Name lookup and checking of semantic constraints for default arguments ~~in function templates and in member~~of templated functions ~~of class templates~~ are performed as described in [temp.inst]. [*Example*: In the following code, `g` will be called with the value `f(2)`:

  […]

  — *end example*] [*Note*: ~~In member function declarations, names in default arguments are looked up as described in [basic.lookup.unqual].~~ A default argument is a complete-class context ([class.mem]). Access checking applies to names in default arguments as described in [class.access]. — *end note*]

Change paragraph 9:

- A default argument is evaluated each time the function is called with no argument for the corresponding parameter. A parameter shall not appear as a potentially-evaluated expression in a default argument. [*Note:* Parameters of a function declared before a default argument are in scope and can hide namespace and class member names. *— end note*] [*Example*:

  […]

— *end example*] A non-static member shall not appear in a default argument unless it appears as the *id-expression* of a class member access expression ([expr.ref]) or unless it is used to form a pointer to member ([expr.unary.op]). [*Example*: The declaration of `X::mem1()` in the following example is ill-formed because no object is supplied for the non-static member `X::a` used as an initializer.

[…]

The declaration of `X::mem2()` is meaningful, however, since no object is needed to access the static member `X::b`. Classes, objects, and members are described in [class.pre]. — *end example*] A default argument is not part of the type of a function. [*Example*:

[…]

— *end example*] When an overload set contains a declaration of a function ~~is introduced by way of a *using-declaration* ([namespace.udecl])~~ that inhabits a scope *S*, any default argument ~~information~~ associated with ~~the~~ any reachable declaration ~~is made known as well~~ that inhabits *S* is available to the call. ~~If the function is redeclared thereafter in the namespace with additional default arguments, the additional arguments are also known at any point following the redeclaration where the *using-declaration* is in scope.~~ [*Note*: The candidate might have been found through a *using-declarator* from which the declaration that provides the default argument is not reachable. — *end note*]

#[dcl.init]

Change paragraph 5:

- A declaration of a block ~~-scope~~ variable with ~~external or internal~~ linkage that has an *initializer* is ill-formed.

Remove paragraph 13:

- An initializer for a static member is in the scope of the member's class. [*Example*:

  […]

  — *end example*]

Change paragraph 22:

- A declaration that specifies the initialization of a variable, whether from an explicit initializer or by default-initialization, is called the *initializing declaration* of that variable. [*Note*: In most cases this is the defining declaration ([basic.def]) of the variable, but the initializing declaration of a non-inline static data member ([class.static.data]) might be the declaration within the class definition and not the definition ~~at namespace scope~~ (if any) outside it. — *end note*]

#[dcl.init.aggr]

Change bullet (4.1):

- If the element is an anonymous union ~~object~~ member and the initializer list is a *designated-initializer-list*, the ~~anonymous union object~~ element is initialized by the *designated-initializer-list* {*D*}, where *D* is the *designated-initializer-clause* naming a member of the anonymous union ~~object~~ member. […]

#[dcl.fct.def]

#[dcl.fct.def.general]

Change paragraph 7:

- ~~In the *function-body*, a~~ A *function-local predefined variable* ~~denotes a block-scope object of~~ is a variable with static storage duration that is implicitly defined ~~(see [basic.scope.block])~~ in a function parameter scope.

#[dcl.fct.def.delete]

Change paragraph 2:

- A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed. [*Note*: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. ~~If a function is overloaded, it is referenced~~ For an overload set, only ~~if~~ the function ~~is~~ selected by overload resolution is referenced. The implicit odr-use ([basic.def.odr]) of a virtual function does not, by itself, constitute a reference. — *end note*]

#[dcl.fct.def.coroutine]

Change paragraph 6:

- If searches for t~~T~~he ~~*unqualified-ids*~~ names `return_void` and `return_value` ~~are looked up~~ in the scope of the promise type ~~. If both are found~~ each find any declarations, the program is ill-formed. [*Note*: If ~~the *unqualified-id* `return_void`~~ is found, flowing off the end of a coroutine is equivalent to a `co_return` with no operand. Otherwise, flowing off the end of a coroutine results in undefined behavior ([stmt.return.coroutine]). — *end note*]

Change paragraph 9:

- An implementation may need to allocate additional storage for a coroutine. This storage is known as the *coroutine state* and is obtained by calling a non-array allocation function ([basic.stc.dynamic.allocation]). The allocation function's name is looked up by searching for it in the scope of the promise type. ~~If this lookup fails, the allocation function's name is looked up in the global scope.~~

    1. If ~~the lookup finds an allocation function in the scope of the promise type~~ any declarations are found, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. The lvalues $\texttt{p}_1$ … $\texttt{p}_n$ are the succeeding arguments.
    2. Otherwise, a search is performed in the global scope.

    If no viable function is found ([over.match.viable]), overload resolution is performed again on a function call created by passing just the amount of space required as an argument of type `std::size_t`.

Change paragraph 10:

- ~~The *unqualified-id*~~ If a search for the name `get_return_object_on_allocation_failure` ~~is looked up~~ in the scope of the promise type ~~by class member access lookup~~ ([~~basic~~class.member.lookup~~.classref~~]). ~~If~~ finds any declarations ~~are found~~, then the result of a call to an allocation function used to obtain storage for the coroutine state is assumed to return `nullptr` if it fails to obtain storage, and if a global allocation function is selected, the `::operator new(size_t, nothrow_t)` form is used. […]

Change paragraph 11:

- The coroutine state is destroyed when control flows off the end of the coroutine or the `destroy` member function ([coroutine.handle.resumption]) of a coroutine handle ([coroutine.handle]) that refers to the coroutine is invoked. In the latter case ~~objects with automatic storage duration that are in scope at the suspend point are destroyed in the reverse order of the construction~~, control in the coroutine is considered to be transferred out of the function ([stmt.dcl]). The storage for the coroutine state is released by calling a non-array deallocation function ([basic.stc.dynamic.deallocation]). If `destroy` is called for a coroutine that is not suspended, the program has undefined behavior.

Change paragraph 12:

- The deallocation function's name is looked up by searching for it in the scope of the promise type. If ~~this lookup fails~~ nothing is found, ~~the deallocation function's name~~ a search is ~~looked up~~ performed in the global scope. If ~~deallocation function lookup finds~~ both a usual deallocation function with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter are found, then the selected deallocation function shall be the one with two parameters. […]

#[dcl.struct.bind]

Change paragraph 4:

- Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named `value`, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. Let `i` be an index prvalue of type `std::size_t` corresponding to $\texttt{v}_i$. ~~The *unqualified-id*~~ If a search for the name `get` ~~is looked up~~ in the scope of `E` ~~by class member access lookup~~ ([~~basic~~class.member.lookup~~.classref~~])~~, and if that~~ finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the initializer is *`e`*`.get<i>()`. Otherwise, the initializer is `get<i>(`*`e`*`)`, where `get` ~~is~~ undergoes argument-dependent look~~ed~~ up ~~in the associated namespaces~~ ([basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*. [*Note*: Ordinary unqualified lookup ([basic.lookup.unqual]) is not performed. — *end note*][…]

#[enum]

#[dcl.enum]

Change paragraph 1:

- […][*Note*: […] — *end note*] The *identifier* in an *enum-head-name* is not looked up and is introduced by the *enum-specifier* or *opaque-enum-declaration*. If the *enum-head-name* of an *opaque-enum-declaration* contains a *nested-name-specifier*, the declaration shall be an explicit specialization ([temp.expl.spec]).

Change paragraph 4:

- If an *enum-head-name* contains a *nested-name-specifier*, ~~it shall not begin with a *decltype-specifier* and~~ the enclosing *enum-specifier* or *opaque-enum-declaration* *D* shall ~~refer to an enumeration that was previously declared directly~~ not inhabit a class scope and shall correspond to one or more declarations nominable in the class, class template, or namespace to which the *nested-name-specifier* refers~~, or in an element of the inline namespace set ([namespace.def]) of that namespace (i.e., neither inherited nor introduced by a *using-declaration*), and the *enum-specifier* or *opaque-enum-declaration* shall appear in a namespace enclosing the previous declaration~~ ([basic.scope.scope]). All those declarations shall have the same target scope; the target scope of *D* is that scope.

Change paragraph 6:

- An enumeration whose underlying type is fixed is an incomplete type ~~from its point of declaration ([basic.scope.pdecl]) to~~ until immediately after its *enum-base* (if any), at which point it becomes a complete type. An enumeration whose underlying type is not fixed is an incomplete type ~~from its point of declaration to immediately after~~ until the closing `}` of its *enum-specifier*, at which point it becomes a complete type.

Change paragraph 11:

- ~~Each *enum-name* and~~ The name of each unscoped ~~*enumerator*~~ enumerator is ~~declared~~ also bound in the scope that immediately contains the *enum-specifier*. ~~Each scoped *enumerator* is declared in the scope of the enumeration.~~ An unnamed enumeration that does not have a typedef name for linkage purposes ([dcl.typedef]) and that has a first enumerator is denoted, for linkage purposes ([basic.link]), by its underlying type and its first

enumerator; such an enumeration is said to have an enumerator as a name for linkage purposes. ~~These names obey the scope rules defined for all names in [basic.scope] and [basic.lookup].~~ [*Note*: Each unnamed enumeration with no enumerators is a distinct type. — *end note*] [*Example*:

[…]

— *end example*] ~~An enumerator declared in class scope can be referred to using the class member access operators (`::`, `.` (dot) and `->` (arrow)), see [expr.ref]. [*Example*:~~

~~[…]~~

~~— *end example*]~~

#[enum.udecl]

Change paragraph 2:

- A *using-enum-declaration* ~~introduces the enumerator names of the named enumeration as if by~~ is equivalent to a *using-declaration* for each enumerator.

Change paragraph 3:

- [*Note*: A *using-enum-declaration* in class scope ~~adds~~ makes the enumerators of the named enumeration ~~as members to the scope. This means they are accessible for~~ available via member lookup. [*Example*:

  […]

  — *end example*] — *end note*]

#[basic.namespace]

Change paragraph 1:

- A namespace is an optionally-named ~~declarative region~~ entity whose scope can contain declarations of any kind of entity. The name of a namespace can be used to access entities ~~declared in~~ that belong to that namespace; that is, the ~~members~~ *members* of the namespace. Unlike other ~~declarative regions~~ entities, the definition of a namespace can be split over several parts of one or more translation units and modules.

Change paragraph 2:

- [*Note*: A ~~namespace name with external linkage~~ *namespace-definition* is exported ~~if any of its *namespace-definitions* is exported, or~~ if it contains any *export-declaration*s ([module.interface]). A namespace is never attached to a named module~~,~~ and never has a name with module linkage ~~even if it is not exported~~. — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 3:

- ~~The outermost declarative region of a translation unit~~ There is a ~~namespace~~ *global namespace* with no declaration; see [basic.scope.namespace]. The global namespace belongs to the global scope; it is not an unnamed namespace ([namespace.unnamed]). [*Note*: Lacking a declaration, it cannot be found by name lookup. — *end note*]

#[namespace.def]

Change paragraph 1:

- Every *namespace-definition* shall ~~appear at~~ inhabit a namespace scope ([basic.scope.namespace]).

Change paragraph 2:

- In a *named-namespace-definition* *D*, the *identifier* is the name of the namespace. ~~If t~~The *identifier*~~, when~~ is looked up ~~([basic.lookup.unqual]), refers to a *namespace-name* (but not a *namespace-alias*) that was introduced~~ by searching for it in the scopes of the namespace *A* in which ~~the *named-namespace-definition* *D*~~ appears ~~or that was introduced in a member~~ and of every element of the inline namespace set of ~~that namespace~~ *A*. If the lookup finds a *namespace-definition* for a namespace *N*, ~~the *namespace-definition* *D* *extends* the previously-declared namespace~~ *N*, and the target scope of *D* is the scope to which *N* belongs. ~~Otherwise~~ If the lookup finds nothing, the *identifier* is introduced as a *namespace-name* into ~~the declarative region in which the *named-namespace-definition* appears~~ *A*.

Remove paragraph 4 (whose example is adapted in [basic.scope.namespace]/1):

- The *enclosing namespaces* of a declaration are […]

Change paragraph 7:

- […] Finally, looking up a name in the enclosing namespace via explicit qualification ([namespace.qual]) will include members of the inline namespace ~~brought in by the *using-directive*~~ even if there are declarations of that name in the enclosing namespace.

Change paragraph 8:

- These properties are transitive: if a namespace `N` contains an inline namespace `M`, which in turn contains an inline namespace `O`, then the members of `O` can be used as though they were members of `M` or `N`. The *inline namespace set* of `N` is the transitive closure of all inline namespaces in `N`. ~~The *enclosing namespace set* of `O` is the set of namespaces consisting of the innermost non-inline namespace enclosing an inline namespace `O`, together with any intervening inline namespaces.~~

#[namespace.memdef]

Remove subclause, referring the stable name to [namespace.def]. Remove the cross-reference parenthetical in [namespace.def]/4 and update those in [temp.class.spec]/6 and [temp.expl.spec]/3 to refer to [dcl.meaning].

The examples in paragraphs 2 and 3 are moved to [dcl.meaning] and [class.friend]/11 respectively.

#[namespace.alias]

Change paragraph 2:

- The *identifier* in a *namespace-alias-definition* ~~is~~becomes a ~~synonym for the name of~~*namespace-alias* and denotes the namespace denoted by the *qualified-namespace-specifier* ~~and becomes a *namespace-alias*~~. [*Note*: When looking up a *namespace-name* in a *namespace-alias-definition*, only namespace names are considered, see [basic.lookup.udir]. — *end note*]

Remove paragraph 3 (whose example is adapted in [basic.scope.scope]):

- In a declarative region, a *namespace-alias-definition* can be used to redefine […]

#[namespace.udir]

Change paragraph 2:

- [*Note:* A *using-directive* ~~specifies that~~makes the names in the nominated namespace ~~can be used~~usable in the scope in which the *using-directive* appears after the *using-directive* ([basic.lookup.unqual], [namespace.qual]). During unqualified name lookup ~~([basic.lookup.unqual])~~, the names appear as if they were declared in the nearest enclosing namespace which contains both the *using-directive* and the nominated namespace. ~~[*Note:* In this context, "contains" means "contains directly or indirectly". —~~ *end note*]

Change paragraph 3:

- [*Note:* A *using-directive* does not ~~add any members to the declarative region in which it appears~~introduce any names. — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 4:

- ~~For unqualified lookup ([basic.lookup.unqual]), the~~[*Note*: A *using-directive* is transitive: if a scope contains a *using-directive* that nominates a ~~second~~ namespace that itself contains *using-directive*s, the ~~effect is as if the~~namespaces nominated by those *using-directive*s ~~from the second namespace~~are also ~~appeared in the first~~eligible to be considered. ~~[*Note*: For qualified lookup, see [namespace.qual].~~ — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 5:

- ~~If~~[*Note*: Declarations in a namespace ~~is extended ([namespace.def])~~that appear after a *using-directive* for that namespace ~~is given, the additional members of the extended namespace and the members of namespaces nominated by~~ can be found through that *using-directive*~~s in the extending *namespace-definition* can be used after the extending *namespace-definition*~~ after they appear. — *end note*]

Change paragraph 7:

- ~~During overload resolution, all functions from the transitive search are considered for argument matching. The set of declarations found by the transitive search is unordered.~~ [*Note*: ~~In particular, t~~The order in which namespaces ~~we~~are considered and the relationships among the namespaces implied by the *using-directive*s do not ~~cause preference to be given to any of the declarations found by the search~~affect overload resolution. ~~— *end note*]~~ ~~An ambiguity exists if the best match finds two functions with~~Neither is any function excluded because another has the same signature, even if one is in a namespace reachable through *using-directive*s in the namespace of the other.[…] — *end note*] [*Example*:

  […]

  — *end example*]

#[namespace.udecl]

Change and split paragraph 1:

- The component names of a *using-declarator* are those of its *nested-name-specifier* and *unqualified-id*. Each *using-declarator* in a *using-declaration*[…] ~~introduces a set of declarations into the declarative region in which the *using-declaration* appears. The~~names the set of declarations ~~introduced by the *using-declarator* is~~ found by ~~performing qualified name~~ lookup ([basic.lookup.qual]~~, [class.member.lookup]~~) for

~~the name in~~ the *using-declarator*, ~~excluding~~ except that class and enumeration declarations that would be discarded are merely ignored when checking for ambiguity ([basic.lookup]), conversion function templates with a dependent return type are ignored, and certain functions ~~that~~ are hidden as described below. If the terminal name of the *using-declarator* is dependent ([temp.dep.type]), the *using-declarator* is considered to name a constructor if and only if the *nested-name-specifier* has a terminal name that is the same as the *unqualified-id*. If the lookup in any instantiation finds that a *using-declarator* that is not considered to name a constructor does do so, or that a *using-declarator* that is considered to name a constructor does not, the program is ill-formed.

- ~~If the *using-declarator* does not name a constructor, the *unqualified-id* is declared in the declarative region in which the *using-declaration* appears as a synonym for each declaration introduced by the *using-declarator*. [*Note*: Only the specified name is so declared; specifying an enumeration name in a *using-declaration* does not declare its enumerators in the *using-declaration*'s declarative region. — *end note*]~~ If the *using-declarator* names a constructor, it declares that the class *inherits* the named set of constructor declarations ~~introduced by the *using-declarator*~~ from the nominated base class. [*Note*: Otherwise, the *unqualified-id* in the *using-declarator*, which is replaced during name lookup with the declarations it names ([basic.lookup]). If such a declaration is of an enumeration, the names of its enumerators are not bound. For the keyword `typename`, see [temp.res]. — *end note*]

Remove paragraph 2 (part of whose example is merged into another below):

- Every *using-declaration* is a *declaration* and a *member-declaration* […]

Change paragraph 3, combining its latter two examples with that from /2 (in the given order and with changes as marked):

- In a *using-declaration* used as a *member-declaration*, each *using-declarator* shall either name an enumerator or have a *nested-name-specifier* naming a base class of the current class ~~being defined~~. [*Example*:

  […]

  — *end example*] If a *using-declarator* names a constructor, its *nested-name-specifier* shall name a direct base class of the current class ~~being defined~~. If the immediate (class) scope is associated with a class template, it shall derive from the specified base class or have at least one dependent base class. ~~[*Example:*~~

  ~~[…]~~

  ~~— *end example*]~~ [*Example*:

```
struct B {
  void f(char);
  void g(char);
  enum E { e };
  union { int x; };
};

struct D : B {
  using B::f;
  void f(int) { f('c'); }       // calls B::f(char)
  void g(int) { g('c'); }       // recursively calls D::g(int)
};

class struct C {
  int gf();
};

class struct D2 : public B {
  using B::f;                   // OK: B is a base of D2
  using B::e;                   // OK: e is an enumerator of base B
  using B::x;                   // OK: x is a union member of base B
  using C::gf;                   // error: C isn't a base of D2
  void f(int) { f('c'); }       // calls B::f(char)
  void g(int) { g('c'); }       // recursively calls D::g(int)
};

template <typename... bases>
struct X : bases... {
  using bases::gf...;
};

X<B, DC> x;                     // OK: B::gf and DC::gf introducednamed
```

  — *end example*]

Change paragraph 4:

- [*Note*: Since destructors do not have names, a *using-declaration* cannot refer to a destructor for a base class. ~~Since specializations of member templates for conversion functions are not found by name lookup, they are not considered when a *using-declaration* specifies a conversion function ([temp.mem]).~~ — *end note*] If a constructor or assignment operator brought from a base class into a derived class has the signature of a copy/move constructor or assignment operator for the derived class ([class.copy.ctor], [class.copy.assign]), the *using-declaration* does not by itself suppress the implicit declaration of the derived class member; the member from the base class is hidden or overridden by the implicitly-declared copy/move constructor or assignment operator of the derived class, as described below.

Replace paragraphs 8 and 9:

- Members declared by a *using-declaration* can be referred to by explicit qualification just like other member names ([namespace.qual]). [*Example*:

  […]

— end example]

- A *using-declaration* is a *declaration* and can therefore be used repeatedly where (and only where) multiple declarations are allowed. [*Example*:

  […]

  — end example]

- If a declaration is named by two *using-declarator*s that inhabit the same class scope, the program is ill-formed.

Change paragraph 10:

- [*Note*: ~~For a~~A *using-declarat~~ion~~*or whose *nested-name-specifier* names a namespace~~, members~~ does not name declarations added to the namespace after ~~the *using-declaration* are not in the set of introduced declarations, so they are not considered when a use of the name is made~~it. Thus, additional overloads added after the *using-declaration* are ignored, but default function arguments ([dcl.fct.default]), default template arguments ([temp.param]), and template specializations ([temp.class.spec], [temp.expl.spec]) are considered. *— end note*] [*Example*:

  […]

  — end example]

Remove paragraph 11:

- [*Note*: Partial specializations of class templates are found by looking up the primary class template […] *— end note*]

Change paragraph 12:

- ~~Since a *using-declaration* is a declaration, the restrictions on declarations of the same name in the same declarative region ([basic.scope]) also apply to *using-declarations*.~~ If a declaration named by a *using-declaration* that inhabits the target scope of another declaration potentially conflicts with it ([basic.scope.scope]) and either is reachable from the other, the program is ill-formed. If two declarations named by *using-declaration*s that inhabit the same scope potentially conflict, either is reachable from the other, and they do not both declare functions or function templates, the program is ill-formed. [*Note*: Overload resolution may not be able to distinguish between conflicting function declarations. *— end note*] [*Example*:

```
namespace A {
  int x;
  int f(int);
  int g;
  void h();
}

namespace B {
  int i;
  struct g { };
  struct x { };
  void f(int);
  void f(double);
  void g(char);     // OK: hides struct g
}

void func() {
  int i;
  using B::i;       // error: i declared twiceconflicts
  void f(char);
  using B::f;       // OK: each f is a function
  using A::f;       // OK, but interferes with B::f(int)
  f(1);             // error: ambiguous
  static_cast<int(*)(int)>(f)(1);  // OK: calls A::f
  f(3.5);           // calls B::f(double)
  using B::g;
  g('a');           // calls B::g(char)
  struct g g1;      // g1 has class type B::g
  using A::g;       // error: conflicts with B::g
  void h();
  using A::h;       // error: conflicts
  using B::x;
  using A::x;       // OK: hides struct B::x
  x = 99;           // assigns to A::x
  struct x x1;      // x1 has class type B::x
}
```

— end example]

Remove paragraph 13 (whose example is merged into the above):

- If a function declaration in namespace scope or block scope has the same name and the same parameter-type-list ([dcl.fct]) as […]

Change paragraph 14:

- ~~When~~The set of declarations named by a *using-declarator* ~~brings declarations from a base class into a derived class,~~that inhabits a class `C` does not include member functions and member function templates ~~in the derived~~of a base class ~~override and/or hide member functions and member function templates with the same name, parameter-type-list ([dcl.fct]), trailing *requires-clause* (if any), cv-qualification, and *ref-qualifier* (if any), in a base class~~that correspond to (~~rather than~~and thus would conflict~~ing~~ with) a declaration of a function or function template in `C`. ~~Such hidden or overridden declarations are excluded from the set of declarations introduced by the *using-declarator*.~~ [*Example*:

[…]

— *end example*]

Change paragraph 15:

- [*Note*: For the purpose of forming a set of candidates during overload resolution, the functions ~~that are introduced~~named by a *using-declaration* in~~to~~ a derived class are treated as though they were direct members of the derived class ~~([class.member.lookup])~~. In particular, the implicit object parameter is treated as if it were a reference to the derived class rather than to the base class ([over.match.funcs]). This has no effect on the type of the function, and in all other respects the function remains ~~a member~~part of the base class. — *end note*]

Change paragraph 16:

- Constructors that are ~~introduced~~named by a *using-declaration* are treated as though they were constructors of the derived class when looking up the constructors of the derived class ([class.qual]) or forming a set of overload candidates ([over.match.ctor], [over.match.copy], [over.match.list]). [*Note*: If such a constructor is selected to perform the initialization of an object of class type, all subobjects other than the base class from which the constructor originated are implicitly initialized ([class.inhctor.init]). A constructor of a derived class is sometimes preferred to a constructor of a base class if they would otherwise be ambiguous ([over.match.best]). — *end note*]

Change paragraph 17:

- In a *using-declarator* that does not name a constructor, ~~all members of the set of introduced~~every declaration~~s~~ named shall be accessible. In a *using-declarator* that names a constructor, no access check is performed. ~~In particular, if a derived class uses a *using-declarator* to access a member of a base class, the member name shall be accessible. If the name is that of an overloaded member function, then all functions named shall be accessible. The base class members mentioned by a *using-declarator* shall be visible in the scope of at least one of the direct base classes of the class where the *using-declarator* is specified.~~

Change paragraph 19:

- A ~~synonym created by a~~ *using-declaration* has the usual accessibility for a *member-declaration*. ~~A~~Base-class constructors considered because of a *using-declarator* ~~that names a constructor does not create a synonym; instead, the additional constructors~~ are accessible if they would be accessible when used to construct an object of the ~~corresponding~~ base class~~, and~~; the accessibility of the *using-declaration* is ignored. [*Example*:

  […]

  — *end example*]

Remove paragraph 20:

- ~~If a *using-declarator* uses the keyword `typename` and specifies a dependent name ([temp.dep]), […]~~

#[dcl.link]

Change paragraph 1:

- All ~~function types,~~ function~~s names with external linkage,~~ and variable~~s~~ whose names ~~with~~have external linkage and all function types have a *language linkage*. [*Note*: Some of the properties associated with an entity with language linkage are specific to each implementation and are not described here. For example, a particular language linkage may be associated with a particular form of representing names of objects and functions with external linkage, or with a particular calling convention, etc. — *end note*] The default language linkage of all function types, function~~s names~~, and variable~~s names~~ is C++ language linkage. Two function types with different language linkages are distinct types even if they are otherwise identical.

Change paragraph 3:

- Every implementation shall provide for linkage to ~~functions written in~~ the C programming language, `"C"`, and ~~linkage to~~ C++ ~~functions~~, `"C++"`. [*Example*:

  […]

  — *end example*]

Change paragraph 5:

- Linkage specifications nest. When linkage specifications nest, the innermost one determines the language linkage. [*Note:* A linkage specification does not establish a scope. ~~— *end note*~~] A *linkage-specification* shall ~~occur only in~~inhabit a namespace scope ~~([basic.scope])~~. In a *linkage-specification*, the specified language linkage applies to the function types of all function declarators~~;~~ and to all function~~s names with external linkage,~~ and variable~~s names with external linkage declared within the *linkage-specification*~~. [*Example*:

```
extern "C"                      // ~~the name~~ f1 and its function type have C language linkage;
  void f1(void(*pf)(int));      // pf is a pointer to a C function

extern "C" typedef void FUNC();
FUNC f2;                        // ~~the name~~ f2 has C++ language linkage and~~ the~~
                                // ~~function's~~its type has C language linkage

extern "C" FUNC f3;             // ~~the name of function~~ f3 and ~~the function's~~its type have C language linkage

void (*pf2)(FUNC*);             // ~~the name of~~ the variable pf2 has C++ linkage~~ and the~~; its type
                                // ~~of pf2~~ is "pointer to C++ function that takes one parameter of type
```

```
                                // pointer to C function"
extern "C" {
  static void f4();             // the name of the function f4 has internal linkage (not C language linkage),
                                // and the function's so f4 has no language linkage; its type has C language linkage.
}
```

[…]

— *end example*] A C language linkage is ignored in determining the language linkage of the names of class members and the function type of class member functions. [*Example*:

```
extern "C" typedef void FUNC_c();

class C {
  void mf1(FUNC_c*);            // the name of the function mf1 and the member function's its type have C++ language linkage;
                                // C++ language linkage; the parameter has type "pointer to C function"

  FUNC_c mf2;                   // the name of the function mf2 and the member function's its type have C++ language linkage
                                // C++ language linkage

  static FUNC_c* q;             // the name of the data member q has C++ language linkage and;
                                // the data member's its type is "pointer to C function"
};

extern "C" {
  class X {
    void mf();                  // the name of the function mf and the member function's its type have C++ language linkage
                                // C++ language linkage
    void mf2(void(*)());        // the name of the function mf2 has C++ language linkage;
                                // the parameter has type "pointer to C function"
  };
}
```

— *end example*]

Change paragraph 6:

- If two declarations declare functions with the same name and parameter-type-list ([dcl.fct]) to be members of the same namespace or declare objects with the same name to be members of the same namespace and the declarations of an entity give the names it different language linkages, the program is ill-formed; no diagnostic is required if the neither declarations appear in different translation units is reachable from the other. Except for functions with C++ linkage, a function declaration without a linkage specification shall not precede the first linkage specification for that function. A function can be declared redeclaration of an entity without a linkage specification after an explicit linkage specification has been seen; the linkage explicitly specified in the earlier declaration is not affected by such a function declaration inherits the language linkage of the entity and (if applicable) its type.

Change paragraph 7:

- At most one function with a particular name can have C language linkage. Two declarations for declare the same entity if they (re)introduce the same name, one declares a function or variable with C language linkage with the same function name (ignoring the namespace names that qualify it) that appear in different namespace scopes refer to the same function, and the other declares such an entity or a variable that belongs to the global scope. Two declarations for a variable with C language linkage with the same name (ignoring the namespace names that qualify it) that appear in different namespace scopes refer to the same variable. An entity with C language linkage shall not be declared with the same name as a variable in global scope, unless both declarations denote the same entity; no diagnostic is required if the declarations appear in different translation units. A variable with C language linkage shall not be declared with the same name as a function with C language linkage (ignoring the namespace names that qualify the respective names); no diagnostic is required if the declarations appear in different translation units. [*Note*: Only one definition for an entity with a given name with C language linkage may appear in the program (see [basic.def.odr]); this implies that such an entity must not be defined in more than one namespace scope. — *end note*] [*Example*:

[…]

— *end example*]

#[dcl.attr.nodiscard]

Change paragraph 2:

- An name or entity declared without the `nodiscard` attribute can later be redeclared with the attribute and vice-versa. […]

#[module]

#[module.unit]

Change paragraph 7:

- A *module* is either a named module or the global module. A declaration is *attached* to a module as follows:

  1. If the declaration is a non-dependent friend declaration that nominates a function with a *declarator-id* that is a *qualified-id* or *template-id* or that nominates a class other than with an *elaborated-type-specifier* with neither a *nested-name-specifier* nor a *simple-template-id*, it is attached to the module to which the friend is attached ([basic.link]).
  2. If Otherwise, if the declaration

1. is a replaceable global allocation or deallocation function ([new.delete.single], [new.delete.array]), or
2. is a *namespace-definition* with external linkage, or
3. appears within a *linkage-specification*,

it is attached to the global module.

3. Otherwise, the declaration is attached to the module in whose purview it appears.

#[module.interface]

Change paragraph 1:

- An *export-declaration* shall ~~appear only at~~inhabit a namespace scope and ~~only~~appear in the purview of a module interface unit. An *export-declaration* shall not appear directly or indirectly within an unnamed namespace or a *private-module-fragment*. An *export-declaration* has the declarative effects of its *declaration*, *declaration-seq* (if any), or *module-import-declaration*. ~~An *export-declaration* does not establish a scope and its~~The *declaration* or *declaration-seq* of an *export-declaration* shall not contain an *export-declaration* or *module-import-declaration*. [*Note*: An *export-declaration* does not establish a scope. — *end note*]

Change paragraph 2:

- A declaration is *exported* if it is~~a namespace-scope declaration~~ declared within an *export-declaration*~~;~~ and inhabits a namespace scope or it is

  1. a *namespace-definition* that contains an exported declaration~~;~~ or
  2. a declaration within a header unit ([module.import]) that introduces at least one name.

Change paragraph 6:

- A redeclaration of ~~an exported declaration of~~ an entity or *typedef-name X* is implicitly exported if *X* was introduced by an exported declaration; otherwise it shall not be exported. ~~An exported redeclaration of a non-exported declaration of an entity is ill-formed.~~ [*Example*:

  […]

  — *end example*]

Change paragraph 7:

- ~~A name is *exported* by a module if it is introduced or redeclared by an exported declaration in the purview of that module.~~ [*Note*: Names introduced by e~~E~~xported ~~names~~declarations have either external linkage or no linkage; see [basic.link]. Namespace-scope ~~names~~declarations exported by a module ~~are visible to~~can be found by name lookup in any translation unit importing that module~~; see~~ ([basic.~~scope.namespace~~lookup]). Class and enumeration member names ~~are visible to~~can be found by name lookup in any context in which a definition of the type is reachable. — *end note*] [*Example*:

  […]

  — *end example*]

Remove paragraph 8:

- [*Note*: Redeclaring a name in an *export-declaration* cannot change the linkage of the name ([basic.link]). [*Example*:

  […]

  — *end example*] — *end note*]

#[module.import]

Change paragraph 1:

- A *module-import-declaration* shall ~~only appear at~~inhabit the global ~~namespace~~ scope. In a module unit, all *module-import-declaration*s and *export-declaration*s exporting *module-import-declaration*s shall ~~precede~~appear before all other *declaration*s in the *declaration-seq* of the *translation-unit* and of the *private-module-fragment* (if any). The optional *attribute-specifier-seq* appertains to the *module-import-declaration*.

Change paragraph 2:

- A *module-import-declaration imports* a set of translation units determined as described below. [*Note*: Namespace-scope ~~names~~declarations exported by the imported translation units ~~become visible~~can be found by name lookup ([basic.~~scope.namespace~~lookup]) in the importing translation unit and declarations within the imported translation units become reachable ([module.reach]) in the importing translation unit after the import declaration. — *end note*]

Change the footnote in paragraph 7:

- This is consistent with the lookup rules for ~~visibility of~~ imported names ([basic.~~scope.namespace~~lookup]).

#[module.global.frag]

Change bullet (3.3):

- *S* contains a~~n expression~~ dependent call ~~of the form~~

~~*postfix-expression* (*expression-list*$_{opt}$)~~

~~whose *postfix-expression* denotes a dependent name, or for an operator expression whose operator denotes a dependent name,~~ ([temp.dep]) and *D* is found by name lookup for the ~~corresponding~~dependent name in an expression synthesized from `E` by replacing each type-dependent argument or operand with a value of a placeholder type with no associated namespaces or entities, or

Change bullet (3.4):

- *S* contains an expression that takes the address of an overload~~ed function~~ set ([over.over]) ~~whose set of overloads~~that contains *D* and for which the target type is dependent, or

Change bullet (3.7):

- *D* redeclares the entity declared by *M* or *M* redeclares the entity declared by *D*, and *D* ~~is~~ neither is a friend declaration nor inhabits a block~~-~~ scope ~~declaration~~, or

#[module.context]

Change paragraph 1:

- The *instantiation context* is a set of points within the program that determines which ~~names~~declarations are ~~visible to~~found by argument-dependent name lookup ([basic.lookup.argdep]) and which ~~declarations~~ are reachable ([module.reach]) in the context of a particular declaration or template instantiation.

#[module.reach]

Change paragraph 1:

- A translation unit *U* is *necessarily reachable* from a point *P* if *U* is a module interface unit on which the translation unit containing *P* has an interface dependency, or the translation unit containing *P* imports *U*, in either case prior to *P* ([module.import]). [*Note*: While module interface units are reachable even when they are only transitively imported via a non-exported import declaration, namespace-scope names from such module interface units are not ~~visible to~~found by name lookup ([basic.~~scope.namespace~~lookup]). — *end note*]

Change paragraph 3:

- A declaration *D* is *reachable from* ~~if, for any~~a point *P* ~~in the instantiation context ([module.context]),~~if

  1. *D* appears prior to *P* in the same translation unit, or
  2. *D* is not discarded ([module.global.frag]), appears in a translation unit that is reachable from *P*, and does not appear within a *private-module-fragment*.

  A declaration is *reachable* if it is reachable from any point in the instantiation context ([module.context]). [*Note*: Whether a declaration is exported has no bearing on whether it is reachable. — *end note*]

Change paragraph 4:

- The accumulated properties of all reachable declarations of an entity within a context determine the behavior of the entity within that context. [*Note*: These reachable semantic properties include type completeness, type definitions, initializers, default arguments of functions or template declarations, attributes, ~~visibility of class or enumeration member~~ names ~~to ordinary lookup~~bound, etc. Since default arguments are evaluated in the context of the call expression, the reachable semantic properties of the corresponding parameter types apply in that context. [*Example*:

  […]

  — *end example*] — *end note*]

Change paragraph 5:

- [*Note*: Declarations of a~~A~~n entity can ~~have~~be reachable ~~declarations~~ even ~~if it is not visible to~~where they cannot be found by name lookup. — *end note*] [*Example*:

  […]

  — *end example*]

#[class]

#[class.pre]

Change paragraph 1:

- […]

  A class declaration where the *class-name* in the *class-head-name* is a *simple-template-id* shall be an explicit specialization ([temp.expl.spec]) or a partial specialization ([temp.class.spec]). A *class-specifier* whose *class-head* omits the *class-head-name* defines an unnamed class. [*Note*: An unnamed class thus can't be `final`. — *end note*] Otherwise, the *class-name* is an *identifier*; it is not looked up, and the *class-specifier* introduces it.

Change paragraph 2:

- ~~A *class-name* is inserted into the scope in which it is declared immediately after the *class-name* is seen.~~ The *class-name* is also ~~inserted into~~bound in the scope of the class (template) itself; this is known as the *injected-class-name*. […]

Change paragraph 3:

- If a *class-head-name* contains a *nested-name-specifier*, the *class-specifier* shall ~~refer to a class that was previously declared directly in~~not inhabit a class scope. If its *class-name* is an *identifier*, the *class-specifier* shall correspond to one or more declarations nominable in the class, class template, or namespace to which the *nested-name-specifier* refers~~, or in an element of the inline namespace set ([namespace.def]) of that namespace (i.e., not merely inherited or introduced by a *using-declaration*), and~~; they shall all have the same target scope, and the target scope of the *class-specifier* ~~the *class-specifier* shall appear in a namespace enclosing the previous declaration~~is that scope. ~~In such cases, the *nested-name-specifier* of the *class-head-name* of the definition shall not begin with a *decltype-specifier*.~~ [*Example*:

```
namespace N {
  template<class>
  struct A {
    struct B;
  };
}
using N::A;
template<class T> struct A<T>::B {};  // OK
template<> struct A<void> {};         // error: A not nominable in ::
```

~~— *end example*~~]

#[class.prop]

Change bullet (3.7.1):

- If `X` is a non-union class type with no ~~(possibly inherited ([class.derived]))~~ non-static data members, the set *M*(`X`) is empty.

#[class.name]

Change paragraph 1:

- A class definition introduces a new type. [*Example*:

  […]

  declare ~~an~~ overload~~ed~~s ([over]) ~~function~~named `f`~~()~~ and not simply a single function `f()` twice. For the same reason,

  […] — *end example*]

Change paragraph 2:

- ~~A class declaration introduces the class name into the scope where it is declared and hides any class, variable, function, or other declaration of that name in an enclosing scope ([basic.scope]). If~~[*Note*: It may be necessary to use an *elaborated-type-specifier* to refer to a class ~~name is declared in~~that belongs to a scope ~~where~~in which its name is also bound to a variable, function, or enumerator ~~of the same name is also declared, then when both declarations are in scope, the class can be referred to only using an *elaborated-type-specifier*~~ ([basic.lookup.elab]). [*Example*:

```
struct stat {
  // ...
};

stat gstat;                     // use plain stat to define variable

int stat(struct stat*);         // redeclare stat as now also names a function

void f() {
  struct stat* ps;              // struct prefix needed to name struct stat
  stat(ps);                     // call stat() function
}
```

  — *end example*] An *elaborated-type-specifier* ~~*declaration* consisting solely of *class-key identifier*;~~ ~~is either a redeclaration of the name in the current scope or a forward declaration of the~~may also be used to declare an identifier as a class name. ~~It introduces the class name into the current scope.~~ [*Example*:

  […]

  — *end example*] ~~[*Note:*~~ Such declarations allow definition of classes that refer to each other. [*Example*:

  […]

  Declaration of friends is described in [class.friend], operator functions in [over.oper]. — *end example*] — *end note*]

Change paragraph 3:

- [*Note*: An *elaborated-type-specifier* ([dcl.type.elab]) can also be used as a *type-specifier* as part of a declaration. It differs from a class declaration in that ~~if a class of the elaborated name is in scope the elaborated name will~~it can refer to ~~it~~an existing class of the given name. — *end note*] [*Example*:

[…]

— end example]

#[class.mem]

Change paragraph 1:

- […] A *direct member* of a class `X` is a member of `X` that was first declared within the *member-specification* of `X`, including anonymous union ~~objects~~members ([class.union.anon]) and direct members thereof. […]

Insert before paragraph 6:

- A redeclaration of a class member outside its class definition shall be a definition, an explicit specialization, or an explicit instantiation ([temp.expl.spec], [temp.explicit]). The member shall not be a non-static data member.

Change paragraph 6:

- A *complete-class context* of a class (template) is a

  1. function body ([dcl.fct.def.general]),
  2. default argument ([dcl.fct.default]),
  3. default template argument ([temp.param]), or
  4. *noexcept-specifier* ([except.spec]), or
  5. default member initializer

  within the *member-specification* of the class or class template. [*Note*: […] *— end note*][…]

#[class.mfct]

Change paragraph 1:

- ~~A~~If a member function ~~may be~~is attached to the global module and is defined ([dcl.fct.def]) in its class definition, ~~in which case~~ it is ~~an~~ inline ([dcl.inline])~~ member function if it is attached to the global module, or it may be defined outside of its class definition if it has already been declared but not defined in its class definition~~. [*Note*: A member function is also inline if it is declared `inline`, `constexpr`, or `consteval`. *— end note*]

Remove paragraph 2:

- A member function definition that appears outside of the class definition shall appear in a namespace scope enclosing the class definition. […]

Change paragraph 3:

- ~~If the definition of a member function is lexically outside its class definition, the member function name shall be qualified by its class name using the `::` operator. [*Note:* A name used in a member function definition (that is, in the *parameter-declaration-clause* including the default arguments ([dcl.fct.default]) or in the member function body) is looked up as described in [basic.lookup]. *— end note*]~~ [*Example*:

  […]

  The definition of the member function `f` of class `X` ~~is defined in~~inhabits the global scope; the notation `X::f` ~~specifies~~indicates that the function `f` is a member of class `X` and in the scope of class `X`. In the function definition, the parameter type `T` refers to the typedef member `T` declared in class `X` and the default argument `count` refers to the static data member `count` declared in class `X`. *— end example*]

Remove paragraphs 4 and 5:

- [*Note*: A `static` local variable or local type in a member function always refers to the same entity, whether or not the member function is inline. *— end note*]

- Previously declared member functions may be mentioned in friend declarations.

#[class.mfct.non-static]

Change paragraph 3:

- When an *id-expression* ([expr.prim.id]) that is not part of a class member access syntax ([expr.ref]) and not used to form a pointer to member ([expr.unary.op]) is used ~~in a member of~~where the current class is `X` ~~in a context where `this` can be used~~ ([expr.prim.this]), if name lookup ([basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class `C`, and if either the *id-expression* is potentially evaluated or `C` is `X` or a base class of `X`, the *id-expression* is transformed into a class member access expression ([expr.ref]) using `(*this)`~~ ([class.this])~~ as the *postfix-expression* to the left of the `.` operator. […]

Change and combine paragraphs 4 and 5:

- [~~*Note:*~~ A non-static member function may be declared ~~`const`, `volatile`, or `const volatile`. These~~with *cv-qualifiers*, which affect the type of the `this` pointer ([~~class~~expr.prim.this]). ~~They also affect the function type ([dcl.fct]) of the member function; a member function declared `const` is a *const member function*, a member function declared `volatile` is a *volatile member function* and a member function declared `const volatile` is a *const volatile member function*. [*Example*:~~

[…]

`X::g` is a const member function and `X::h` is a const volatile member function. — *end example*] A non-static member function may be declared with and/or a *ref-qualifier* ([dcl.fct]); see; both affect overload resolution ([over.match.funcs]). — *end note*]

#[class.this]

Remove subclause, referring the stable name to [expr.prim.this].

#[class.ctor]

Change paragraph 1:

- A *constructor* is introduced by a declaration whose *declarator* declares a *constructor* if it is a function declarator ([dcl.fct]) of the form

  ptr-declarator (*parameter-declaration-clause*) *noexcept-specifier*$_{opt}$ *attribute-specifier-seq*$_{opt}$

  where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

  1. in a friend declaration ([class.friend]), the *id-expression* is a *qualified-id* that names a constructor ([class.qual]);
  2. otherwise, in a *member-declaration* that belongs to the *member-specification* of a class or class template but is not a friend declaration ([class.friend]), the *id-expression* is the injected-class-name ([class.pre]) of the immediately-enclosing entity or;
  3. in a declaration at namespace scope or in a friend declaration otherwise, the *id-expression* is a *qualified-id* that names a constructor ([class.qual]) whose *unqualified-id* is the injected-class-name of its lookup context.

  Constructors do not have names. In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, `constexpr`, or an *explicit-specifier*. [*Example*:

  […]

  — *end example*]

Change paragraph 2:

- A constructor is used to initialize objects of its class type. [*Note:* Because constructors do not have names, they are never found during unqualified name lookup; however an explicit type conversion using the functional notation ([expr.type.conv]) will cause a constructor to be called to initialize an object. [*Note:* The syntax looks like an explicit call of the constructor. — *end note*] [*Example*:

  […]

  — *end example*] [*Note*: For initialization of objects of class type see [class.init]. — *end note*]

#[class.copy.assign]

Change paragraph 8:

- Because a copy/move assignment operator is implicitly declared for a class if not declared by the user, a base class copy/move assignment operator is always hidden by the corresponding assignment operator of a derived class ([over.ass]). [*Note:* A *using-declaration* ([namespace.udecl]) that brings in from a base class names an assignment operator with a parameter type that could be that of a copy/move assignment operator for the a derived class is not considered an explicit declaration of such an operator and does not suppress the implicit declaration of the derived class operator; the operator introduced by the *using-declaration* is hidden by the implicitly-declared operator in the derived class ([namespace.udecl]). — end note]

#[class.dtor]

Change paragraph 1:

- A *prospective destructor* is introduced by a declaration whose *declarator-id* is has an *unqualified-id* that begins with a ~ declares a *prospective destructor*; its *declarator* shall be a function declarator ([dcl.fct]) of the form

  ptr-declarator (*parameter-declaration-clause*) *noexcept-specifier*$_{opt}$ *attribute-specifier-seq*$_{opt}$

  where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

  1. in a *member-declaration* that belongs to the *member-specification* of a class or class template but is not a friend declaration ([class.friend]), the *id-expression* is *~class-name* and the *class-name* is the injected-class-name ([class.pre]) of the immediately-enclosing entity or
  2. in a declaration at namespace scope or in a friend declaration otherwise, the *id-expression* is *nested-name-specifier ~class-name* and the *class-name* names is the injected-class-name of the same class as nominated by the *nested-name-specifier*.

  A prospective destructor shall take no arguments ([dcl.fct]). Each *decl-specifier* of the *decl-specifier-seq* of a prospective destructor declaration (if any) shall be `friend`, `inline`, `virtual`, `constexpr`, or `consteval`.

Change paragraph 12:

- A prospective destructor can be declared `virtual` ([class.virtual]) or pure `virtual` ([class.abstract]). If the destructor of a class is virtual and any objects of that class or any derived class are created in the program, the destructor shall be defined. ~~If a class has a base class with a virtual destructor, its destructor (whether user- or implicitly-declared) is virtual.~~

#[class.conv]

Remove paragraph 5 (adapted in [class.conv.fct]):

- User-defined conversions are used implicitly only if they are unambiguous. […]

#[class.conv.fct]

Change paragraph 1:

- A member function of a class `X` ~~having no parameters~~ with a name of the form

  conversion-function-id:

  `operator` *conversion-type-id*

  conversion-type-id:

  type-specifier-seq conversion-declarator$_{opt}$

  conversion-declarator:

  ptr-operator conversion-declarator$_{opt}$

  shall have no parameters and specifies a conversion from `X` to the type specified by the *conversion-type-id*, interpreted as a *type-id* ([dcl.name]). […]

Replace paragraph 4 with [class.conv]/5, with changes as marked:

- Conversion functions are inherited.

- ~~User-defined conversions are used implicitly only if they are unambiguous.~~ [*Note:* A conversion function in a derived class ~~does not~~ hides only ~~a~~ conversion function~~s~~ in ~~a~~ base class~~es~~ ~~unless the two functions~~that convert to the same type. A conversion function template with a dependent return type hides only templates in base classes that correspond to it ([class.member.lookup]); otherwise, it hides and is hidden as a non-template function. Function overload resolution ([over.match.best]) selects the best conversion function to perform the conversion. *— end note*] [*Example*:

  […]

  — end example]

#[class.static]

Remove paragraph 2:

- A static member may be referred to directly in the scope of its class or in the scope of a class derived ([class.derived]) from its class; […]

#[class.static.mfct]

Change paragraph 2:

- [*Note*: A static member function does not have a `this` pointer ([~~class~~expr.prim.this]). ~~*— end note*] A static member function shall not be `virtual`. There shall not be a static and a non-static member function with the same name and the same parameter types ([over.load]).~~ A static member function ~~shall not be declared~~cannot be qualified with `const`, `volatile`, or ~~`const volatile`~~`virtual`. *— end note*]

#[class.static.data]

Change paragraph 3:

- The declaration of a non-inline static data member in its class definition is not a definition and may be of an incomplete type other than *cv* `void`. ~~The definition for a static data member that is not defined inline in the class definition shall appear in a namespace scope enclosing the member's class definition. In the definition at namespace scope, the name of the static data member shall be qualified by its class name using the `::` operator.~~ [*Note:* The *initializer* ~~expression~~ in the definition of a static data member is in the scope of its class ([basic.scope.class]). *— end note*] [*Example*:

  […]

  The definition of the static data member `run_chain` of class `process` ~~is defined in~~inhabits the global scope; the notation `process::run_chain` ~~specifies~~indicates that the member `run_chain` is a member of class `process` and in the scope of class `process`. In the static data member definition, the *initializer* expression refers to the static data member `running` of class `process`. *— end example*]

[*Note*: Once the static data member has been defined, it exists even if no objects of its class have been created. [*Example*: In the example above, `run_chain` and `running` exist even if no objects of class `process` are created by the program. — *end example*] The initialization and destruction of static data members are described in [basic.start.static], [basic.start.dynamic], and [basic.start.term]. — *end note*]

Change paragraph 4:

- […] The member shall still be defined in a namespace scope if it is odr-used ([basic.def.odr]) in the program and the namespace scope definition shall not contain an *initializer*. The declaration of a~~A~~n inline static data member ~~may be defined in the class~~(which is a definition ~~and~~) may specify a *brace-or-equal-initializer*. If the member is declared with the `constexpr` specifier, it may be redeclared in namespace scope with no initializer (this usage is deprecated; see [depr.static.constexpr]). Declarations of other static data members shall not specify a *brace-or-equal-initializer*.

Remove paragraph 7 (re-expressed as the added note text above):

- Static data members are initialized and destroyed exactly like non-local variables ([basic.start.static], [basic.start.dynamic], [basic.start.term]).

#[class.nest]

Change paragraph 1:

- A class can be declared within another class. A class declared within another is called a *nested class*. ~~The name of a nested class is local to its enclosing class. The nested class is in the scope of its enclosing class.~~ [*Note*: See [expr.prim.id] for restrictions on the use of non-static data members and non-static member functions. — *end note*]

  [*Example*:

  […]

  ```
  inner* p = 0;                   // error: inner not ~~in scope~~found
  ```

  — *end example*]

Change and merge paragraphs 2 and 3:

- [*Note*: Nested classes can be defined either in the enclosing class or in an enclosing namespace; m~~M~~ember functions and static data members of a nested class can be defined either in the nested class or in an enclosing namespace scope ~~enclosing the definition of their class~~. [*Example*:

  […]

  ~~— *end example*]~~

  ~~If class `x` is defined in a namespace scope, a nested class `y` may be declared in class `x` and later defined in the definition of class `x` or be later defined in a namespace scope enclosing the definition of class `x`. [*Example*:~~

  […]

  — *end example*] *— end note*]

Change paragraph 4:

- ~~Like a member function, a~~A friend function ([class.friend]) defined within a nested class ~~is in the lexical scope of that class; it obeys the same rules for name binding as a static member function of that class ([class.static]), but it~~ has no special access rights to members of an enclosing class.

#[class.nested.type]

Remove subclause, referring the stable name to [diff.basic]. Update the one cross reference (in [diff.class]/5) to refer to [class.member.lookup].

#[class.union.anon]

Change paragraph 1:

- A union of the form

  `union {` *member-specification* `} ;`

  is called an *anonymous union*; it defines an unnamed type and an unnamed object of that type called an *anonymous union* ~~*object*~~*member* if it is a non-static data member or an *anonymous union variable* otherwise. Each *member-declaration* in the *member-specification* of an anonymous union shall either define ~~a~~one or more non-static data member~~s~~ or be a *static_assert-declaration*. Nested types, anonymous unions, and functions shall not be declared within an anonymous union. The names of the members of an anonymous union ~~shall be distinct from the names of any other entity~~ are bound in the scope ~~in which~~ inhabited by the ~~anonymous~~ union ~~is declared~~ declaration. ~~For the purpose of name lookup, after the anonymous union definition, the members of the anonymous union are considered to have been defined in the scope in which the anonymous union is declared.~~ [*Example*:

  […]

  Here `a` and `p` are used like ordinary (non-member) variables, but since they are union members they have the same address. — *end example*]

Change paragraph 2:

- Anonymous unions declared ~~in a named~~in the scope of a namespace ~~or in the global namespace~~with external linkage shall be declared `static`. Anonymous unions declared at block scope shall be declared with any storage class allowed for a block ~~scope~~ variable, or with no storage class. A storage class is not allowed in a declaration of an anonymous union in a class scope. An anonymous union shall not have private or protected members ([class.access]). An anonymous union shall not have member functions.

#[class.local]

Change paragraph 1:

- A class can be declared within a function definition; such a class is called a *local class*. ~~The name of a local class is local to its enclosing scope. The local class is in the scope of the enclosing scope, and has the same access to names outside the function as does the enclosing function.~~ [*Note*: […] — *end note*] [*Example*:

  […]

  ```
  local* p = 0;                   // error: local not ~~in scope~~found
  ```

  — *end example*]

#[class.derived]

Change paragraph 2:

- The component names of a *class-or-decltype* are those of its *nested-name-specifier*, *type-name*, and/or *simple-template-id*. A *class-or-decltype* shall denote a (possibly cv-qualified) class type that is not an incompletely defined class ([class.mem]); any cv-qualifiers are ignored. The class denoted by the *class-or-decltype* of a *base-specifier* is called a *direct base class* for the class being defined. ~~During t~~The lookup for ~~a base class~~the component name~~, non-type names are ignored~~ of the *type-name* or *simple-template-id* is type-only ([basic.~~scope.hiding~~lookup]). A class `B` is a base class of a class `D` if it is a direct base class of `D` or a direct base class of one of `D`'s base classes. A class is an *indirect base class* of another if it is a base class but not a direct base class. A class is said to be (directly or indirectly) *derived* from its (direct or indirect) base classes. [*Note*: See [class.access] for the meaning of *access-specifier*. — *end note*] ~~Unless redeclared in the derived class, members of a base class are also considered to be members of the derived class.~~ Members of a base class ~~other than constructors~~ are ~~said to be *inherited* by the~~also members of the derived class. [*~~Note~~:* Constructors of a base class can ~~also~~ be explicitly inherited ~~as described in~~ ([namespace.udecl]). ~~Inherited~~Base class members can be referred to in expressions in the same manner as other members of the derived class, unless their names are hidden or ambiguous ([class.member.lookup]). ~~[*Note:*~~ The scope resolution operator `::` ([expr.prim.id.qual]) can be used to refer to a direct or indirect base member explicitly~~. This allows access to a name that has been redeclared~~, even if it is hidden in the derived class. A derived class can itself serve as a base class subject to access control; see [class.access.base]. A pointer to a derived class can be implicitly converted to a pointer to an accessible unambiguous base class ([conv.ptr]). An lvalue of a derived class type can be bound to a reference to an accessible unambiguous base class ([dcl.init.ref]). — *end note*]

#[class.mi]

Change the note in paragraph 3:

- A class can be an indirect base class more than once and can be a direct and an indirect base class. There are limited things that can be done with such a class~~. The~~; lookup that finds its non-static data members and member functions ~~of the direct base class cannot be referred to~~ in the scope of the derived class will be ambiguous. However, the static members, enumerations and types can be unambiguously referred to.

#[class.virtual]

Change paragraph 1:

- A non-static member function is a *virtual function* if it is first declared with the keyword `virtual` or if it overrides a virtual member function declared in a base class (see below).[…] [*Note*: Virtual functions support dynamic binding and object-oriented programming. — *end note*] A class ~~that declares or inherits~~with a virtual member function is called a *polymorphic class*.[…]

Change paragraph 2:

- If a virtual member function ~~`vf`~~*F* is declared in a class ~~`Base`~~*B* and in a class ~~`Derived`~~*D* derived (directly or indirectly) from ~~`Base`~~*B* a declaration of a member function ~~`vf` with the same name, parameter-type-list ([dcl.fct]), cv-qualification, and ref-qualifier (or absence of same) as `Base::vf` is declared~~*G* that corresponds ([basic.scope.scope]) to a declaration of *F*, ignoring trailing *requires-clauses*, then ~~`Derived::vf`~~*G* *overrides*[…] ~~`Base::vf`~~*F*. For convenience we say that any virtual function overrides itself. A virtual member function ~~`C::vf`~~*V* of a class object `S` is a *final overrider* unless the most derived class ([intro.object]) of which `S` is a base class subobject (if any) ~~declares or inherits~~has another member function that overrides ~~`vf`~~*V*. […]

Remove paragraph 7:

- Even though destructors are not inherited, a destructor in a derived class overrides a base class destructor declared virtual; see [class.dtor] and [class.free].

Change paragraph 9:

- If the class type in the covariant return type of `D::f` differs from that of `B::f`, the class type in the return type of `D::f` shall be complete at the ~~point~~locus of the overriding declaration ~~of `D::f`~~ or shall be the class type `D`. When the overriding function is called as the final overrider of the

overridden function, its result is converted to the type returned by the (statically chosen) overridden function ([expr.call]). [*Example*:

[…]

— *end example*]

#[class.abstract]

Change paragraph 4:

- A class is abstract if it ~~contains or inherits~~has at least one pure virtual function for which the final overrider is pure virtual. [*Example*:

[…] — *end example*]

#[class.access]

Change paragraph 1:

- A member of a class can be

1. private; that is, its~~ name~~ can be ~~us~~named only by members and friends of the class in which it is declared.
2. protected; that is, its~~ name~~ can be ~~us~~named only by members and friends of the class in which it is declared, by classes derived from that class, and by their friends (see [class.protected]).
3. public; that is, its~~ name~~ can be ~~us~~named anywhere without access restriction.

[*Note*: A constructor or destructor can be named by an expression ([basic.def.odr]) even though it has no name. — *end note*]

Change paragraph 2:

- A member of a class can also access all the ~~name~~members to which the class has access. A local class of a member function may access the same ~~name~~members that the member function itself may access.[…]

Change paragraph 4:

- Access control is applied uniformly to ~~all names, whether the names are referred to from~~ declarations ~~or~~and expressions. [*Note*: Access control applies to ~~names~~members nominated by friend declarations ([class.friend]) and *using-declaration*s ([namespace.udecl]). — *end note*] When a *using-declarator* is named, access control is applied to it, not to the declarations that replace it. ~~In the case of overloaded function names~~For an overload set, access control is applied only to the function selected by overload resolution. [*Note*: Because access control applies to ~~names~~the declarations named, if access control is applied to a typedef name, only the accessibility of the typedef name itself is considered. The accessibility of the entity referred to by the typedef is not considered. For example,

[…]

— *end note*]

Change paragraph 5:

- [*Note*: Access~~ to members and base classes is~~ control~~led, not their visibility ([basic.scope.hiding]). Names of~~ does not prevent members ~~are still visible, and~~from being found by name lookup or implicit conversions to base classes ~~are still~~from being considered~~, when those members and base classes are inaccessible~~. — *end note*] The interpretation of a given construct is established without regard to access control. If the interpretation established makes use of inaccessible member~~ name~~s or base classes, the construct is ill-formed.

Change paragraph 6:

- All access controls in [class.access] affect the ability to ~~access~~name a class member~~ name~~ from the declaration of a particular entity, including parts of the declaration preceding the name of the entity being declared and, if the entity is a class, the definitions of members of the class appearing outside the class's *member-specification*. [*Note*: This access also applies to implicit references to constructors, conversion functions, and destructors. — *end note*]

Change paragraph 8:

- ~~The names in~~Access is checked for a default argument ([dcl.fct.default]) ~~are bound~~ at the point of declaration~~, and access is checked at that point~~ rather than at any points of use of the default argument. Access checking for default arguments in function templates and in member functions of class templates is performed as described in [temp.inst].

Change paragraph 9:

- ~~The names in~~Access for a default *template-argument* ([temp.param]) ~~have their access~~is checked in the context in which ~~they~~it appears rather than at any points of use of ~~the default *template-argument*~~it. [*Example*:

[…]

— *end example*]

#[class.access.base]

Change paragraph 3:

- [*Note*: A member of a private base class might be inaccessible as ~~an~~ inherited ~~member name~~, but accessible directly. […]

  — *end note*]

Change paragraph 5:

- […] The access to a member is affected by the class in which the member is named. This naming class is the class in ~~which the member name was looked up and found~~ whose scope name lookup performed a search that found the member. […]

#[class.friend]

Change paragraph 1:

- A friend of a class is a function or class that is given permission to ~~use~~ name the private and protected member ~~names from~~ s of the class. […]

Change paragraph 2:

- Declaring a class to be a friend implies that ~~the names of~~ private and protected members ~~from~~ of the class granting friendship can be ~~accessed~~ named in the *base-specifier*s and member declarations of the befriended class. [*Example*:

  […]

  — *end example*] [*Example*:

  […]

  — *end example*]

  ~~A class shall not be defined in a friend declaration. [*Example:*~~

  ~~[…]~~

  ~~— *end example*]~~

Change the example in paragraph 3:

- […]

```
friend D;                       // error: no type name D in scope not found
```

  […]

Change paragraph 5:

- ~~When a~~ [*Note*: A friend declaration refers to an ~~overloaded~~ entity, not (all overloads of) a name ~~or operator, only the function specified by the parameter types becomes a friend~~. A member function of a class `X` can be a friend of a class `Y`. [*Example*:

  […]

  — *end example*] — *end note*]

Change paragraph 6:

- A function ~~can~~ may be defined in a friend declaration of a class if and only if the class is a non-local class ([class.local]) ~~,~~ and the function name is unqualified ~~, and the function has namespace scope~~. [*Example*:

  […]

  — *end example*]

Change paragraph 7:

- Such a function is implicitly an inline ([dcl.inline]) function if it is attached to the global module. ~~A friend function defined in a class is in the (lexical) scope of the class in which it is defined.~~ [*Note*: If a ~~A~~ friend function is defined outside ~~the~~ a class, it is not in the scope of the class ~~([basic.lookup.unqual])~~. — *end note*]

Change paragraph 9:

- A ~~name~~ member nominated by a friend declaration shall be accessible in ~~the scope of~~ the class containing the friend declaration. The meaning of the friend declaration is the same whether the friend declaration appears in the private, protected, or public ([class.mem]) portion of the class *member-specification*.

Replace paragraph 11, adding the example from [namespace.memdef]/3 and then retaining its example:

- If a friend declaration appears in a local class ([class.local]) and the name specified is an unqualified name, […]

- [*Note*: A friend declaration never binds any names ([dcl.meaning], [dcl.type.elab]). — *end note*] [*Example*:

  […] from [namespace.memdef]/2]

— end example] [*Example*:

[… from [class.friend]/11]

— end example]

#[class.paths]

Change paragraph 1:

- If a ~~name~~declaration can be reached by several paths through a multiple inheritance graph, the access is that of the path that gives most access. [*Example*:

  […] *— end example*]

#[class.init]

#[class.base.init]

Change paragraph 2:

- ~~In~~Lookup for an unqualified name in a *mem-initializer-id* ~~an initial unqualified *identifier* is looked up in the scope of the constructor's class and, if not found in that scope, it is looked up in the scope containing the constructor's definition~~ignores the constructor's function parameter scope. [*Note*: If the constructor's class contains a member with the same name as a direct or virtual base class of the class, a *mem-initializer-id* naming the member or base class and composed of a single identifier refers to the class member. A *mem-initializer-id* for the hidden base class may be specified using a qualified name. *— end note*] Unless the *mem-initializer-id* names the constructor's class, a non-static data member of the constructor's class, or a direct or virtual base of that class, the *mem-initializer* is ill-formed.

Change paragraph 4:

- If a *mem-initializer-id* is ambiguous because it designates both a direct non-virtual base class and an ~~inherited~~indirect virtual base class, the *mem-initializer* is ill-formed. [*Example*:

  […]

  — end example]

Change paragraph 15:

- ~~Names in t~~[*Note*: The *expression-list* or *braced-init-list* of a *mem-initializer* ~~are evaluated~~is in the function parameter scope of the constructor ~~for which the *mem-initializer* is specified.~~ and can use `this` to refer to the object being initialized. *— end note*] [*Example*:

  […]

  initializes `X::r` to refer to `X::a`, initializes `X::b` with the value of the constructor parameter `i`, initializes `X::i` with the value of the constructor parameter `i`, and initializes `X::j` with the value of `X::i`; this takes place each time an object of class `X` is created. *— end example*] ~~[*Note*: Because the *mem-initializer* are evaluated in the scope of the constructor, the `this` pointer can be used in the *expression-list* of a *mem-initializer* to refer to the object being initialized. *— end note*]~~

#[class.copy.elision]

Change bullet (1.2):

- in a *throw-expression* ([expr.throw]), when the operand is the name of a non-volatile object with automatic storage duration (other than a function or catch-clause parameter) ~~whose~~that belongs to a scope that does not ~~extend beyond the end of~~contain the innermost enclosing *compound-statement* associated with a *try-block* (if there is one), the copy/move operation can be omitted by constructing the object directly into the exception object

Change paragraph 2:

- [*Example*:

  […]

  Here the criteria for elision can eliminate the copying of the object `t` with automatic storage duration into the result object for the function call `f()`, which is the ~~global~~non-local object `t2`. Effectively, the construction of ~~the local object~~ `t` can be viewed as directly initializing ~~the global object~~ `t2`, and that object's destruction will occur at program exit. Adding a move constructor to `Thing` has the same effect, but it is the move construction from the object with automatic storage duration to `t2` that is elided. *— end example*]

Change bullet (3.2):

- if the operand of a *throw-expression* ([expr.throw]) is a (possibly parenthesized) *id-expression* that names an implicitly movable entity ~~whose~~that belongs to a scope that does not ~~extend beyond~~contain the *compound-statement* of the innermost *try-block* or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* ~~encloses~~contains the *throw-expression*,

#[class.free]

Remove paragraph 3:

- When an object is deleted with a *delete-expression* ([expr.delete]), […]

Remove paragraph 4, part of which is incorporated in [expr.delete]:

- Class-specific deallocation function lookup is a part of general deallocation function lookup ([expr.delete]) and occurs as follows. […]

Change paragraph 6:

- Since member allocation and deallocation functions are `static` they cannot be virtual. [*Note*: However, when the *cast-expression* of a *delete-expression* refers to an object of class type with a virtual destructor, because the deallocation function ~~actually called~~ is ~~looked up in the scope of the class that is the dynamic type of the object if~~ chosen by the destructor ~~is virtual~~ of the dynamic type of the object, the effect is the same in that case. […] — *end note*] [*Note*: […]

  — *end note*]

Change paragraph 7:

- Access to the deallocation function is checked statically~~. Hence~~, even though a different one might actually be executed~~, the statically visible deallocation function is required to be accessible~~. [*Example*: For the call on line "// 1" above, if `B::operator delete()` had been private, the delete expression would have been ill-formed. — *end example*]

#[over]

#[over.pre]

Change paragraph 1:

- ~~When~~ [*Note*: Each of two or more ~~different declarations are specified for a single~~ entities with the same name in the same scope, ~~that name is said to be *overloaded*, and the declarations are called *overloaded declarations*. Only~~ which must be functions ~~and~~ or function templates ~~declarations can be overloaded; variable and type declarations cannot be overloaded~~, is commonly called an "overload". — *end note*]

Change paragraph 2:

- When a function ~~name~~ is ~~us~~named in a call, which function declaration is being referenced and the validity of the call are determined by comparing the types of the arguments at the point of use with the types of the parameters in the declarations ~~that are visible at the point of use~~ in the overload set. This function selection process is called *overload resolution* and is defined in [over.match]. [*Example*:

  […]

  — *end example*]

#[over.load]

Remove subclause, referring the stable name to [basic.scope.scope]. Update the surviving cross reference (in [class.mem]/5) to refer to [basic.scope.scope].

Parts of the notes and examples are adapted in [basic.scope.scope]/3 and [dcl.fct]/5.

#[over.dcl]

Remove subclause, referring the stable name (which has no cross references) to [basic.link].

The first example in paragraph 1 is moved to [temp.over.link]/5.

#[over.match]

#[over.match.funcs]

Change paragraph 4:

- […] For conversion functions, the function is considered to be a member of the class of the implied object argument for the purpose of defining the type of the implicit object parameter. For non-conversion functions ~~introduced~~ nominated by a *using-declaration* in~~to~~ a derived class, the function is considered to be a member of the derived class for the purpose of defining the type of the implicit object parameter. […]

Insert before paragraph 7:

- In each case where conversion functions of a class `S` are considered for initializing an object or reference of type `T`, the candidate functions include the result of a search for the *conversion-function-id* `operator T` in `S`. [*Note*: This search may find a specialization of a conversion function template ([basic.lookup]). — *end note*] Each such case also defines sets of *permissible types* for explicit and non-explicit conversion functions; each (non-template) conversion function that is a non-hidden member of `S`, yields a permissible type, and, for the former set, is non-explicit is also a candidate function. If initializing an object, for any permissible type *cv* `U`, any *cv2* `U`, *cv2* `U&`, or *cv2* `U&&` is also a permissible type. If the set of permissible types for explicit conversion functions is empty, any candidates that are explicit are discarded.

Change paragraph 7:

- In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction ([temp.over], [temp.deduct]). If a constructor template or conversion function template has an *explicit-specifier* whose *constant-expression* is value-dependent ([temp.dep]), template argument deduction is performed first and then, if the context ~~requires a~~admits only candidate~~s~~ that ~~is~~are not explicit and the generated specialization is explicit ([dcl.fct.spec]), it will be removed from the candidate set. Those candidates are then handled as candidate functions in the usual way.[…] A given name can refer to, or a conversion can consider, one or more function templates ~~and also to~~as well as a set of non-template functions. In such a case, the candidate functions generated from each function template are combined with the set of non-template candidate functions.

#[over.call.func]

Change paragraph 1:

- Of interest in [over.call.func] are only those function calls in which the *postfix-expression* ultimately contains an *id-expression* ~~name~~ that denotes one or more functions that might be called. […]

Change paragraph 2:

- In qualified function calls, the ~~name to be resolved~~function is named by an *id-expression* ~~and is~~ preceded by an `->` or `.` operator. Since the construct `A->B` is generally equivalent to `(*A).B`, the rest of [over] assumes, without loss of generality, that all member function calls have been normalized to the form that uses an object and the `.` operator. Furthermore, [over] assumes that the *postfix-expression* that is the left operand of the `.` operator has type "*cv* `T`" where `T` denotes a class.[…] ~~Under this assumption, the *id-expression* in the call is looked up as a member function of `T` following the rules for looking up names in classes ([class.member.lookup]).~~ The function declarations found by ~~that~~name lookup ([class.member.lookup]) constitute the set of candidate functions. The argument list is the *expression-list* in the call augmented by the addition of the left operand of the `.` operator in the normalized member function call as the implied object argument ([over.match.funcs]).

Change paragraph 3:

- In unqualified function calls, the function is named ~~is not qualified~~ by ~~an `->` or `.` operator and has the more general form of~~ a *primary-expression*. ~~The name is looked up in the context of the function call following the normal rules for name lookup in expressions ([basic.lookup]).~~ The function declarations found by ~~that~~name lookup ([basic.lookup]) constitute the set of candidate functions. Because of the rules for name lookup, the set of candidate functions consists (1) entirely of non-member functions or (2) entirely of member functions of some class `T`. In case (1), the argument list is the same as the *expression-list* in the call. In case (2), the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If the current class is, or is derived, from `T` and the keyword `this` ([~~class~~expr.prim.this]) ~~is in scope and~~ refers to ~~class `T`, or a derived class of `T`~~it, then the implied object argument is `(*this)`. ~~If the keyword `this` is not in scope or refers to another class, then~~Otherwise, a contrived object of type `T` becomes the implied object argument~~.~~[…]~~;~~ ~~If the argument list is augmented by a contrived object and~~if overload resolution selects ~~one of the~~a non-static member function~~s of `T`~~, the call is ill-formed.

#[over.call.object]

Change paragraph 1:

- If the *postfix-expression* `E` in the function call syntax evaluates to a class object of type "*cv* `T`", then the set of candidate functions includes at least the function call operators of `T`. The function call operators of `T` are ~~obtained by ordinary lookup~~the results of a search for the name `operator()` in the ~~context~~scope of ~~`(E).operator()`~~`T`.

#[over.match.oper]

Change bullet (3.1):

- If `T1` is a complete class type or a class currently being defined, the set of member candidates is the result of ~~the qualified lookup of~~a search for ~~`T1::`~~`operator@` ~~([over.call.func])~~ in the scope of `T1`; otherwise, the set of member candidates is empty.

Change bullet (3.2):

- ~~The~~For the operators `=`, `[]`, or `->`, the set of non-member candidates is empty; otherwise, it includes the result of ~~the~~ unqualified lookup ~~of~~for `operator@` ~~in the context of the expression according to the usual rules for name lookup in unqualified~~in the rewritten function call~~s~~ ([basic.lookup.unqual], [basic.lookup.argdep])~~ except that~~, ignoring all member functions ~~are ignored~~. However, if no operand has a class type, only those non-member functions in the lookup set that have a first parameter of type `T1` or "reference to *cv* `T1`", when `T1` is an enumeration type, or (if there is a right operand) a second parameter of type `T2` or "reference to *cv* `T2`", when `T2` is an enumeration type, are candidate functions.

#[over.match.copy]

Change bullet (1.2):

- When the type of the initializer expression is a class type "*cv* `S`", ~~the non-explicit~~ conversion functions ~~of `S` and its base classes~~ are considered. The permissible types for non-explicit conversion functions are `T` and any class derived from `T`. When initializing a temporary object ([class.mem]) to be bound to the first parameter of a constructor where the parameter is of type "reference to *cv2* `T`" and the constructor is called with a single argument in the context of direct-initialization of an object of type "*cv3* `T`", the permissible types for explicit conversion functions are ~~also considered~~the same; otherwise there are none. ~~Those that are not hidden within `S` and yield a type whose cv-unqualified version is the same type as `T` or is a derived class thereof are candidate functions. A call to a conversion function returning "reference to `X`" is a glvalue of type `X`, and such a conversion function is therefore considered to yield `X` for this process of selecting candidate functions.~~

#[over.match.conv]

Change paragraph 1:

- Under the conditions specified in [dcl.init], as part of an initialization of an object of non-class type, a conversion function can be invoked to convert an initializer expression of class type to the type of the object being initialized. Overload resolution is used to select the conversion function to be invoked. Assuming that "*cv1* `T`" is the type of the object being initialized~~, and "*cv* `S`" is the type of the initializer expression, with `S` a class type~~, the candidate functions are selected as follows:

  1. The ~~conversion functions of `S` and its base classes are considered. Those~~ permissible types for non-explicit conversion functions ~~that~~ are ~~not hidden within `S` and yield type `I` or a type~~ those that can be converted to type `T` via a standard conversion sequence ([over.ics.scs])~~ are candidate functions~~. For direct-initialization, ~~those~~ the permissible types for explicit conversion functions ~~that~~ are ~~not hidden within `S` and yield type `I` or a type~~ those that can be converted to type `T` with a (possibly trivial) qualification conversion ([conv.qual])~~ are also candidate functions~~; otherwise there are none. ~~Conversion functions that return a cv-qualified type are considered to yield the cv-unqualified version of that type for this process of selecting candidate functions. A call to a conversion function returning "reference to `X`" is a glvalue of type `X`, and such a conversion function is therefore considered to yield `X` for this process of selecting candidate functions.~~

#[over.match.ref]

Change paragraph 1:

- Under the conditions specified in [dcl.init.ref], a reference can be bound directly to the result of applying a conversion function to an initializer expression. Overload resolution is used to select the conversion function to be invoked. Assuming that "reference to *cv1* `T`" is the type of the reference being initialized~~, and "*cv* `S`" is the type of the initializer expression, with `S` a class type~~, the candidate functions are selected as follows:

  1. ~~The conversion functions of `S` and its base classes are considered. Those non-explicit conversion functions that are not hidden within `S` and yield~~Let *R* be a set of types including "lvalue reference to *cv2* `T2`" (when initializing an lvalue reference or an rvalue reference to function) ~~or~~and "*cv2* `T2`" ~~or~~and "rvalue reference to *cv2* `T2`" (when initializing an rvalue reference or an lvalue reference to function)~~;~~ for any `T2`. The permissible types for non-explicit conversion functions are the members of *R* where "*cv1* `T`" is reference-compatible ([dcl.init.ref]) with "*cv2* `T2`"~~, are candidate functions~~. For direct-initialization, ~~those~~ the permissible types for explicit conversion functions ~~that~~ are ~~not hidden within `S` and yield type "lvalue reference to *cv2* `T2`" (when initializing an lvalue reference or an rvalue reference to function) or "rvalue reference to *cv2* `T2`" (when initializing an rvalue reference or an lvalue reference to function),~~ the members of *R* where `T2` ~~is the same type as `I` or~~ can be converted to type `T` with a (possibly trivial) qualification conversion ([conv.qual])~~, are also candidate functions~~; otherwise there are none.

#[over.match.best]

Change paragraph 4:

- If the best viable function resolves to a function for which multiple declarations were found, and if ~~at least~~any two of these declarations ~~— or the declarations they refer to in the case of *using-declarations* —~~ inhabit different scopes and specify a default argument that made the function viable, the program is ill-formed. [*Example*:

  […]

  — *end example*]

Address of an overloaded~~ function~~ set #[over.over]

Change the name of the subclause.

Change paragraph 1:

- ~~A use of a function name~~ An *id-expression* whose terminal name refers to an overload set *S* and that appears without arguments is resolved to a function, a pointer to function, or a pointer to member function for a specific function that is chosen from a set of ~~selected~~ functions selected from S determined based on the target type required in the context (if any), as described below. The target can be

  1. an object or reference being initialized ([dcl.init], [dcl.init.ref], [dcl.init.list]),
  2. the left side of an assignment ([expr.ass]),
  3. a parameter of a function ([expr.call]),
  4. a parameter of a user-defined operator ([over.oper]),
  5. the return value of a function, operator function, or conversion ([stmt.return]),
  6. an explicit type conversion ([expr.type.conv], [expr.static.cast], [expr.cast]), or
  7. a non-type *template-parameter* ([temp.arg.nontype]).

  The ~~function name~~ *id-expression* can be preceded by the `&` operator. [*Note*: Any redundant set of parentheses surrounding the function name is ignored ([expr.prim.paren]). — *end note*]

Change paragraph 3:

- ~~For each function template designated by the name,~~ The specialization, if any, generated by template argument deduction~~ is done~~ ([temp.over], [temp.deduct.funcaddr], [temp.arg.explicit])~~, and if the argument deduction succeeds, the resulting template argument list is used to generate a single~~ for each function template ~~specialization, which~~named is added to the set of selected functions considered. ~~[*Note*: […] — *end note*]~~

Change paragraph 4:

- Non-member functions and static member functions match targets of function pointer type or reference to function type. Non-static member functions match targets of pointer-to-member-function type. [*Note:* If a non-static member function is ~~selected~~chosen, the ~~reference to the overloaded function name is required to have the form of~~result can be used only to form a pointer to member ~~as described in~~ ([expr.unary.op]). *— end note*]

Change paragraph 7:

- [*Note*: If `f`~~`()`~~ and `g`~~`()`~~ are both overload~~ed functions~~ sets, the cross product of possibilities must be considered to resolve `f(&g)`, or the equivalent expression `f(g)`. — *end note*]

#[over.oper]

Change paragraph 1:

- A ~~function~~ declaration ~~having one of the following~~whose *declarator-id* is an *operator-function-id*~~s as its name~~ shall declare~~s~~a function or function template or an explicit instantiation or specialization of a function template. A function so declared is an *operator function*. A function template ~~declaration having one of the following *operator-function-ids* as its name declares~~so declared is an *operator function template*. A specialization of an operator function template is also an operator function. An operator function is said to *implement* the operator named in its *operator-function-id*.

#[over.unary]

Change paragraph 2:

- [*Note:* The unary and binary forms of the same operator ~~are considered to~~ have the same name. ~~[*Note:*~~ Consequently, a unary operator can hide a binary operator from an enclosing scope, and vice versa. — *end note*]

#[over.literal]

Change paragraph 2:

- A declaration whose *declarator-id* is a *literal-operator-id* shall ~~be a declaration of~~declare a ~~namespace-scope~~ function or function template that belongs to a namespace (it could be a friend function ([class.friend])~~,~~ or an explicit instantiation or specialization of a function template~~, or a *using-declaration* ([namespace.udecl])~~. A function declared with a *literal-operator-id* is a *literal operator*. A function template declared with a *literal-operator-id* is a *literal operator template*.

#[temp]

#[temp.pre]

Change paragraph 4:

- [*Note:* A *template-declaration* can appear only as a namespace scope or class scope declaration. *— end note*] Its *declaration* shall not be an *export-declaration*. In a function template declaration, the ~~last component~~*unqualified-id* of the *declarator-id* shall ~~not~~ be a ~~*template-id*~~name. [*Note*: ~~That last component may be an *identifier*, an *operator-function-id*, a *conversion-function-id*, or a *literal-operator-id*.~~ In a class template declaration, if the ~~class name~~*class-name* is a *simple-template-id*, the declaration declares a class template partial specialization ([temp.class.spec]). — *end note*]

Change paragraph 6:

- ~~A template name has linkage ([basic.link]).~~ A s~~S~~pecialization~~s~~ (explicit or implicit) of ~~a~~one template ~~that has internal linkage are~~is distinct from all specializations ~~in~~of any other ~~translation units~~template. A template, a template explicit specialization ([temp.expl.spec]), and a class template partial specialization shall not have C linkage. ~~Use of a linkage specification other than `"C"` or `"C++"` with any of these constructs is conditionally-supported, with implementation-defined semantics. Template definitions shall obey the one-definition rule ([basic.def.odr]).~~ [*Note*: Default arguments for function templates and for member functions of class templates are considered definitions for the purpose of template instantiation ([temp.decls]) and must ~~also~~ obey the one-definition rule ([basic.def.odr]). — *end note*]

Change paragraph 7:

- [*Note:* A ~~class~~ template ~~shall~~ cannot have the same name as any other ~~template, class, function, variable, enumeration, enumerator, namespace, or type~~name bound in the same scope ([basic.scope.scope]), except ~~as specified in [temp.class.spec]. Except~~ that a function template can ~~be overloaded either by~~share a name with non-template functions ([dcl.fct]) ~~with the same name or by other~~ and/or function templates ~~with the same name~~ ([temp.over])~~, a template name declared in namespace scope or in class scope shall be unique in that scope~~. Specializations, including partial specializations ([temp.class.spec]), do not reintroduce or bind names. Their target scope is the target scope of the primary template, so all specializations of a template belong to the same scope as it does. *— end note*]

Change paragraph 8:

- [*Note*: A local class, a local or block variable, or a friend function defined in a templated entity is a templated entity. — *end note*]

#[temp.param]

Change paragraph 1:

- […]

The component names of a *type-constraint* are its *concept-name* and those of its *nested-name-specifier* (if any). [*Note*: The > token following the *template-parameter-list* of a *type-parameter* may be the product of replacing a >> token by two consecutive > tokens ([temp.names]). — *end note*]

Change paragraph 3:

- The *identifier* in a *type-parameter* is not looked up. A *type-parameter* whose ~~identifier~~*identifier* does not follow an ellipsis defines its *identifier* to be a *typedef-name* (if declared without `template`) or *template-name* (if declared with `template`) in the scope of the template declaration. [*Note*: […]

  — *end note*]

Change paragraph 12:

- […] If a friend function template declaration *D* specifies a default *template-argument*, that declaration shall be a definition and there shall be ~~the only~~no other declaration of the function template ~~in the translation unit~~which is reachable from *D* or from which *D* is reachable.

Change paragraph 15:

- A *template-parameter* shall not be given default arguments by two different declarations ~~in the same scope~~if one is reachable from the other. [*Example*:

  […]

  — *end example*]

#[temp.names]

Change paragraph 1:

- […]

  ~~[*Note*: The name lookup rules ([basic.lookup]) are used to associate the use of a name with a template declaration; that is, to identify a name as a *template-name*. — *end note*]~~ The component name of a *simple-template-id*, *template-id*, or *template-name* is the first name in it.

Change paragraph 2:

- ~~For a *template-name* to be explicitly qualified by the template arguments, the name must be considered to refer to a template. [*Note*: Whether a name actually refers to a template cannot be known in some cases until after argument dependent lookup is done ([basic.lookup.argdep]). — *end note*]~~ A ~~name~~< is ~~considered to refer to~~interpreted as the delimiter of a ~~template~~*template-argument-list* if it follows a name that is not a *conversion-function-id* and

  1. that follows the keyword `template` or a ~ after a *nested-name-specifier* or in a class member access expression, or
  2. for which name lookup finds ~~a *template-name* or an overload set that contains a function template.~~the injected-class-name of a class template or finds any declaration of a template, or
  3. ~~A name is also considered to refer to a template if it~~that is an ~~*unqualified-id* followed by a < and~~unqualified name for which name lookup either finds one or more functions or finds nothing~~.~~, or
  4. that is a terminal name in a *using-declarator* ([namespace.udecl]), a *declarator-id* ([dcl.meaning]), or a type-only context other than a *nested-name-specifier* ([temp.res]).

  [*Note*: If the name is an *identifier*, it is then interpreted as a *template-name*. The keyword `template` is used to indicate that a dependent qualified name ([temp.dep.type]) denotes a template where an expression might appear. — *end note*]

Move the example from paragraph 4 to the end of paragraph 2.

Change paragraph 3:

- ~~When a name is considered to be a *template-name*, and it is followed by a <, the < is always taken as the delimiter of a *template-argument-list* and never as the less-than operator.~~ When parsing a *template-argument-list*, the first non-nested >[…] is taken as the ending delimiter rather than a greater-than operator. Similarly, the first non-nested >> is treated as two consecutive but distinct > tokens, the first of which is taken as the end of the *template-argument-list* and completes the *template-id*. [*Note*: The second > token produced by this replacement rule may terminate an enclosing *template-id* construct or it may be part of a different construct (e.g., a cast).— *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 4:

- The keyword `template` ~~is said to appear at the top level in a *qualified-id* if it appears outside of a *template-argument-list* or *decltype-specifier*. In a *qualified-id* of a *declarator-id* or in a *qualified-id* formed by a *class-head-name* ([class.pre]) or *enum-head-name* ([dcl.enum]), the keyword `template`~~ shall not appear ~~at the top level~~immediately after a declarative *nested-name-specifier* ([expr.prim.id.qual]). ~~In a *qualified-id* used as the name in a *typename-specifier* ([temp.res]), *elaborated-type-specifier* ([dcl.type.elab]), *using-declaration* ([namespace.udecl]), or *class-or-decltype* ([class.derived]), an optional keyword `template` appearing at the top level is ignored. In these contexts, a < token is always assumed to introduce a *template-argument-list*. In all other contexts, when naming a template specialization of a member of an unknown specialization ([temp.dep.type]), the member template name shall be prefixed by the keyword `template`. [*Example*:~~

  ~~[…]~~

~~— *end example*~~]

Change paragraph 5:

- A name prefixed by the keyword `template` shall be ~~*a template-id*~~ followed by a template argument list or ~~the name shall~~ refer to a class template or an alias template. The latter case is deprecated ([depr.template.template]). The keyword `template` shall not appear immediately before a ~ token (as to name a destructor). [*Note*: The keyword `template` may not be applied to non-template members of class templates. — *end note*] [*Note*: As is the case with the `typename` prefix, the `template` prefix is allowed ~~in cases where it is not strictly necessary; i.e., when the *nested-name-specifier* or the expression on the left of the `.` or `->` is not dependent on a *template-parameter*, or the use does not appear in the scope of~~ even when lookup for the name would already find a template. — *end note*] [*Example*:

```
template <class T> struct A {
  void f(int);
  template <class U> void f(U);
};

template <class T> void f(T t) {
  A<T> a;
  a.template f<>(t);                    // OK: calls template
  a.template f(t);                      // error: not a template-id
}

template <class T> struct B {
  template <class T2> struct C { };
};

// ~~OK~~deprecated: T::~~template~~ C would be assumed to name~~s~~ a class template:
template <class T, template <class X> class TT = T::template C> struct D { };
D<B<int> > db;
```

— *end example*]

Change paragraph 7:

- When the *template-name* of a *simple-template-id* names a constrained non-function template or a constrained template *template-parameter*~~, but not a member template that is a member of an unknown specialization ([temp.res])~~, and all *template-argument*s in the *simple-template-id* are non-dependent ([temp.dep.temp]), the associated constraints ([temp.constr.decl]) of the constrained template shall be satisfied ([temp.constr.constr]). [*Example*:

  […]

  — *end example*]

#[temp.arg]

Insert before paragraph 2:

- The template argument list of a *template-head* is a template argument list in which the n^{th} template argument has the value of the n^{th} template parameter of the *template-head*. If the n^{th} template parameter is a template parameter pack ([temp.variadic]), the n^{th} template argument is a pack expansion whose pattern is the name of the template parameter pack.

Change paragraph 3:

- ~~The name of~~ [*Note*: Names used in a *template-argument* ~~shall be accessible at the point where it is used as a *template-argument*~~ are subject to access control where they appear. ~~[*Note*: If the name of the *template-argument* is accessible at the point where it is used as a *template-argument*, there is no further access restriction in the resulting instantiation where the corresponding~~ Since a *template-parameter* ~~name~~ is ~~used~~ not a class member, no access control applies. — *end note*] [*Example*:

  […]

  — *end example*] For a *template-argument* that is a class type or a class template, the template definition has no special access rights to the members of the *template-argument*. [*Example*:

  […]

  — *end example*]

Change paragraph 7:

- When name lookup for the component name ~~in~~of a *template-id* finds an overload set, both non-template functions in the overload set and function templates in the overload set for which the *template-argument*s do not match the *template-parameter*s are ignored. [*Note:* If none of the function templates have matching *template-parameter*s, the program is ill-formed. — *end note*]

#[temp.arg.template]

Change paragraph 2:

- Any partial specializations ([temp.class.spec]) associated with the primary class template or primary variable template are considered when a specialization based on the template *template-parameter* is instantiated. If a specialization is not ~~visible at~~ reachable from the point of

instantiation, and it would have been selected had it been ~~visible~~reachable, the program is ill-formed, no diagnostic required. [*Example*:

[…]

— *end example*]

#[temp.decls]

Replace paragraph 1 (adapting [temp.class.spec]/1):

- A *template-id*, that is, the *template-name* followed by a *template-argument-list* […]

- A *primary template* declaration is one in which the name of the template is not followed by a *template-argument-list*. The template argument list of a primary template is the template argument list of its *template-head* ([temp.arg]). A template declaration in which the name of the template is followed by a *template-argument-list* is a *partial specialization* of the template named in the declaration ([temp.class.spec]).

#[temp.class]

Change paragraph 3:

- [*Note:* When a member ~~function, a member class, a member enumeration, a static data member or a member template~~ of a class template is defined outside of the class template definition, the member definition is defined as a template definition ~~in which the~~with a *template-head* ~~is~~ equivalent to that of the class template ~~([temp.over.link])~~. The names of the template parameters used in the definition of the member ~~may be different~~can differ from the template parameter names used in the class template definition. The ~~template argument list following the~~ class template name in the member definition ~~shall name the parameters in the same order as the one used in the template parameter list of the member~~is followed by the template argument list of the *template-head* ([temp.arg]). ~~Each template parameter pack shall be expanded with an ellipsis in the template argument list.~~ [*Example*:

  […]

  — *end example*] ~~— *end note*~~]

Change paragraph 4:

- In a ~~redeclaration,~~ partial specialization, explicit specialization or explicit instantiation of a class template, the *class-key* shall agree in kind with the original class template declaration ([dcl.type.elab]).

#[temp.deduct.guide]

Change paragraph 1:

- Deduction guides are used when a *template-name* appears as a type specifier for a deduced class type ([dcl.type.class.deduct]). Deduction guides are not found by name lookup. Instead, when performing class template argument deduction ([over.match.class.deduct]), ~~any~~all reachable deduction guides declared for the class template are considered.

Change paragraph 3:

- […] A *deduction-guide* shall ~~be declared in~~inhabit the ~~same~~ scope ~~as~~to which the corresponding class template belongs and, for a member class template, ~~with~~have the same access. Two deduction guide declarations ~~in the same translation unit~~ for the same class template shall not have equivalent *parameter-declaration-clauses* if either is reachable from the other.

#[temp.mem]

Change paragraph 5:

- [*Note:* A specialization of a conversion function template is referenced in the same way as a non-template conversion function that converts to the same type ([class.conv.fct]). [*Example*:

  […]

  — *end example*] ~~*Note:*~~ There is no syntax to form a *template-id* ([temp.names]) by providing an explicit template argument list ([temp.arg.explicit]) for a conversion function template ~~([class.conv.fct])~~. — *end note*]

Remove paragraphs 6 through 8:

- A specialization of a conversion function template is not found by name lookup. […]

- A *using-declaration* in a derived class cannot refer to a specialization of a conversion function template in a base class.

- Overload resolution ([over.ics.rank]) and partial ordering ([temp.func.order]) are used to select the best conversion function […]

#[temp.friend]

Change paragraph 1:

- A friend of a class or class template can be a function template or class template, a specialization of a function template or class template, or a non-template function or class. ~~For a friend function declaration that is not a template declaration:~~

  1. ~~if the name of the friend is a qualified or unqualified *template-id*, the friend declaration refers to a specialization of a function template, otherwise,~~
  2. ~~if the name of the friend is a *qualified-id* and a matching non-template function is found in the specified class or namespace, the friend declaration refers to that function, otherwise,~~
  3. ~~if the name of the friend is a *qualified-id* and a matching function template is found in the specified class or namespace, the friend declaration refers to the deduced specialization of that function template ([temp.deduct.decl]), otherwise,~~
  4. ~~the name shall be an *unqualified-id* that declares (or redeclares) a non-template function.~~

  [*Example*:

  […] — *end example*]

Move [temp.inject]/1 before paragraph 2 and change it:

- Friend classes ~~or~~, class templates, functions, or function templates can be declared within a class template. When a template is instantiated, ~~the names of~~ its friend~~s are treated~~ declarations are found by name lookup as if the specialization had been explicitly declared at its point of instantiation. [*Note*: They may introduce entities that belong to an enclosing namespace scope ([dcl.meaning]), in which case they are attached to the same module as the class template ([module.unit]). — *end note*]

Change paragraph 4:

- […] In this case, a member of a specialization *S* of the class template is a friend of the class granting friendship if deduction of the template parameters of *C* from *S* succeeds, and substituting the deduced template arguments into the friend declaration produces a declaration that ~~would be a valid redeclaration of~~corresponds to the member of the specialization. [*Example*:

  […]

  — *end example*]

Remove paragraph 5 (whose substance is incorporated in the above):

- [*Note*: A friend declaration may first declare a member of an enclosing namespace scope ([temp.inject]). — *end note*]

#[temp.class.spec]

Change paragraph 1 (moving part of it to [temp.decls]):

- ~~A *primary class template* declaration is one in which the class template name is an identifier. A template declaration in which the class template name is a *simple-template-id* is a *partial specialization* of the class template named in the *simple-template-id*.~~ A partial specialization of a class template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization ([temp.class.spec.match]). ~~The~~A declaration of the primary template shall ~~be declared before~~precede any specialization~~s~~ of that template. A partial specialization shall ~~be declared before the first~~be reachable from any use of a class template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation ~~in every translation unit in which such a use occurs~~; no diagnostic is required.

Change paragraph 2:

- Two partial specialization declarations declare the same entity if they are partial specializations of the same template and have equivalent *template-head*s and template argument lists ([temp.over.link]). Each class template partial specialization is a distinct template and definitions shall be provided for the members of a template partial specialization ([temp.class.spec.mfunc]).

Change paragraph 5:

- The template parameters are specified in the angle bracket enclosed list that immediately follows the keyword `template`. ~~For partial specializations, t~~The template argument list of a partial specialization is ~~explicitly written immediately following the class template name~~the *template-argument-list* of the *class-name*. ~~For primary templates, this list is implicitly described by the template parameter list. Specifically, the order of the template arguments is the sequence in which they appear in the template parameter list. [*Example*: The template argument list for the primary template in the example above is `<T1, T2, I>`. — *end example*] [*Note*: The template argument list shall not be specified in the primary template declaration. For example,~~

  ~~[…]~~

  ~~— *end note*]~~

Change paragraph 7:

- Partial specialization declarations ~~themselves are not found by name lookup~~do not introduce a name. ~~Rather~~Instead, when the primary template name is used, any ~~previously declared~~reachable partial specializations of the primary template are also considered. One consequence is that a *using-declaration* which refers to a class template does not restrict the set of partial specializations which may be found through the *using-declaration*. [*Example*:

  […]

  — *end example*]

#[temp.class.spec.mfunc]

Change paragraph 1:

- ~~The template parameter list of a member of a class template partial specialization shall match the template parameter list of the class template partial specialization. The template argument list of a member of a class template partial specialization shall match the template argument list of the class template partial specialization.~~ A class template partial specialization is a distinct template. The members of the class template partial specialization are unrelated to the members of the primary template. Class template partial specialization members that are used in a way that requires a definition shall be defined; the definitions of members of the primary template are never used as definitions for members of a class template partial specialization. An explicit specialization of a member of a class template partial specialization is declared in the same way as an explicit specialization of the primary template. [*Example*:

  […]

  — *end example*]

#[temp.fct]

Change paragraph 2:

- [*Note:* A function template can ~~be overloaded with~~have the same name as other function templates and ~~with~~ non-template functions ([dcl.fct]) in the same scope. *— end note*] A non-template function is not related to a function template (i.e., it is never considered to be a specialization), even if it has the same name and type as a potentially generated function template specialization.[…]

#[temp.over.link]

Change paragraph 5, adding the first example from [over.dcl]/1:

- […] For determining whether two dependent names ([temp.dep]) are equivalent, only the name itself is considered, not the result of name lookup ~~in the context of the template~~. [*Note:* If ~~multiple declarations of the same function template differ in the result of this~~such a dependent name ~~lookup~~is unqualified, ~~the result for~~it is looked up from the first declaration ~~is used~~of the function template ([temp.dep.candidate]). *— end note*] [*Example*:

  […]

```
  int i = h<int>();                               // template argument substitution fails; g(int)
                                                  // ~~was~~ not ~~in scope~~considered at the first declaration of h()
```

  […]

  — *end example*] […][*Note*: For instance, one could have redundant parentheses. — *end note*] [*Example*:

  […]

  — *end example*]

Change paragraph 7:

- ~~Two function templates are *equivalent* if they are declared in the same scope, have the same name, have equivalent *template-heads*, and have return types, parameter lists, and trailing *requires-clauses* (if any) that are equivalent using the rules described above to compare expressions involving template parameters. Two function templates are *functionally equivalent* if they are declared in the same scope, have the same name, accept and are satisfied by the same set of template argument lists, and have return types and parameter lists that are functionally equivalent using the rules described above to compare expressions involving template parameters.~~ If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required.

#[temp.func.order]

Change paragraph 1:

- If ~~a~~multiple function template~~s~~ ~~is overloaded~~share a name, the use of ~~a function template specialization~~that name might be ambiguous because template argument deduction ([temp.deduct]) may ~~associate the function template~~identify a specialization ~~with~~for more than one function template ~~declaration~~. […]

#[temp.concept]

Change paragraph 3:

- A *concept-definition* shall ~~appear at~~inhabit a namespace scope ([basic.scope.namespace]).

#[temp.res]

Replace paragraph 1:

- Three kinds of names can be used within a template definition:

  1. […]

- A name that appears in a declaration *D* of a template *T* is looked up from where it appears in an unspecified declaration of *T* that is or is reachable from *D* and from which no other declaration of *T* that contains the usage of the name is reachable. If the name is *dependent* (as specified in [temp.dep]), it is looked up for each specialization (after substitution) because the lookup depends on a template parameter. [*Note*: Some dependent names are also looked up during parsing to determine that they are dependent or to interpret following < tokens. Uses of other names might be type-dependent or value-dependent ([temp.dep.expr], [temp.dep.constexpr]). A *using-declarator* is never dependent in a specialization and is therefore replaced during lookup for that specialization ([basic.lookup]). — *end note*] [*Example*:

```
struct A {operator int();};
template<class B,class T>
struct D : B {
  T get() {return operator T();}     // conversion-function-id is dependent
};
int f(D<A,int> d) {return d.get();}  // OK: lookup finds A::operator int
```

— *end example*]

Move the example from paragraph 10 and the second example from [temp.dep]/4 to the end of paragraph 1.

Replace paragraph 2 (but not the grammar following it):

- A name used in a template declaration or definition and that is dependent on a *template-parameter* is assumed not to name a type unless the applicable name lookup finds a type name or the name is qualified by the keyword `typename`. [*Example*:

  […]

  — *end example*]

- If the validity or meaning of the program would be changed by considering a default argument or default template argument introduced in a declaration that is reachable from the point of instantiation of a specialization ([temp.point]) but is not found by lookup for the specialization, the program is ill-formed, no diagnostic required.

Change paragraph 3:

- The component names of a *typename-specifier* are its *identifier* (if any) and those of its *nested-name-specifier* and *simple-template-id* (if any). A *typename-specifier* denotes the type or class template denoted by the *simple-type-specifier* ([dcl.type.simple]) formed by omitting the keyword `typename`. [*Note*: The usual qualified name lookup ([basic.lookup.qual]) ~~is used to find the *qualified-id*~~ applies even in the presence of `typename`. — *end note*] [*Example*:

  […]

  — *end example*]

Remove paragraph 4:

- ~~A qualified name used as the name in a *class-or-decltype* ([class.derived]) or an *elaborated-type-specifier* is implicitly assumed to name a type, […]~~

Change paragraph 5:

- A ~~*qualified-id*~~ qualified or unqualified name is ~~assumed~~ said to ~~name~~ be in a type-only context if it is the terminal name of

  1. ~~it is a qualified name in~~ a ~~type-id-only context (see below)~~ *typename-specifier*, *nested-name-specifier*, *elaborated-type-specifier*, *class-or-decltype*, or
  2. a *type-specifier* of a *new-type-id*, *defining-type-id*, *conversion-type-id*, *trailing-return-type*, default argument of a *type-parameter*, or *type-id* of a `static_cast`, `const_cast`, `reinterpret_cast`, or `dynamic_cast`, or
  3. ~~it is~~ a *decl-specifier* of the *decl-specifier-seq* of a
     1. *simple-declaration* or a *function-definition* in namespace scope,
     2. *member-declaration*,
     3. *parameter-declaration* in a *member-declaration*[…], unless that *parameter-declaration* appears in a default argument,
     4. *parameter-declaration* in a *declarator* of a function or function template declaration whose *declarator-id* is qualified, unless that *parameter-declaration* appears in a default argument,
     5. *parameter-declaration* in a *lambda-declarator*, unless that *parameter-declaration* appears in a default argument, or
     6. *parameter-declaration* of a (non-type) *template-parameter*.

  ~~A qualified name is said to be in a *type-id-only context* if it appears in a *type-id*, *new-type-id*, or *defining-type-id* and the smallest enclosing *type-id*, *new-type-id*, or *defining-type-id* is a *new-type-id*, *defining-type-id*, *trailing-return-type*, default argument of a *type-parameter* of a template, or *type-id* of a `static_cast`, `const_cast`, `reinterpret_cast`, or `dynamic_cast`.~~ [*Example*:

  […]

  — *end example*]

Change paragraph 6:

- A *qualified-id* ~~that refers to a member of an unknown specialization, that is not prefixed by `typename`, and~~ whose terminal name is dependent and that is ~~not otherwise assumed to name~~ in a type-only context ~~(see above)~~ is considered to denote~~s~~ a ~~non-~~type. A name that refers to a *using-declarator* whose terminal name is dependent is interpreted as a *typedef-name* if the *using-declarator* uses the keyword `typename`. [*Example*:

  […]

— *end example*]

Remove paragraph 7:

- Within the definition of a class template or within the definition of a member of a class template following the *declarator-id*, […]

Change paragraph 8:

- […] [*Example*:

```
int j;
template<class T> class X {
  void f(T t, int i, char* p) {
    t = i;          // diagnosed if X::f is instantiated, and the assignment to t is an error
    p = i;          // may be diagnosed even if X::f is not instantiated
    p = j;          // may be diagnosed even if X::f is not instantiated
    X<T>::g(t);     // OK
    X<T>::h();      // may be diagnosed even if X::f is not instantiated
  }
  void g(T t) {
    +;              // may be diagnosed even if X::g is not instantiated
  }
};

template<class... T> struct A {
  void operator++(int, T... t);                    // error: too many parameters
};
template<class... T> union X : T... { };           // error: union with base class
template<class... T> struct A : T...,  T... { };   // error: duplicate base class
```

— *end example*]

Remove paragraphs 9 and 10 (whose example is moved to paragraph 1):

- When looking for the declaration of a name used in a template definition, […]

- If a name does not depend on a *template-parameter* (as defined in [temp.dep]), […]

#[temp.local]

Change paragraph 2:

- ~~Within~~When the ~~scope~~injected-class-name of a class template specialization or partial specialization~~, when the injected-class-name~~ is used as a *type-name*, it is equivalent to the *template-name* followed by the *template-argument*s of the class template specialization or partial specialization enclosed in <>. [*Example*:

  […]

  — *end example*]

Change paragraph 3:

- The injected-class-name of a class template or class template specialization can be used as either a *template-name* or a *type-name* wherever it is ~~in scope~~named. [*Example*:

```
template <class T> struct Base {
  Base* p;
};

template <class T> struct Derived: public Base<T> {
  typename Derived::Base* p;            // meaning Derived::Base<T>
};

template<class T, template<class> class U = T::~~template~~ Base> struct Third { };
Third<Derived<int> > t;                 // OK: default argument uses injected-class-name as a template
```

— *end example*]

Change paragraph 6:

- The name of a *template-parameter* shall not be ~~redeclared within its~~bound to any following declaration contained by the scope ~~(including nested scopes)~~to which the *template-parameter* belongs. ~~A *template-parameter* shall not have the same name as the template name.~~ [*Example*:

```
template<class T, int i> class Y {
  int T;                                // error: template-parameter ~~redeclared~~hidden
  void f() {
    char T;                             // error: template-parameter ~~redeclared~~hidden
  }
  friend void T();                      // OK: no name bound
};

template<class X> class X;              // error: hidden by template-parameter~~ redeclared~~
```

— *end example*]

Change paragraph 7:

- ~~In the definition of a member~~Unqualified name lookup considers the template parameter scope of a *template-declaration* immediately after the outermost scope associated with the template declared (even if its parent scope does not contain the *template-parameter-list*). [*Note*: The scope of a class template ~~that appears outside of the class template definition, the name of a member of the class template hides the name of a *template-parameter* of any enclosing class templates (but not a *template-parameter* of the member if the member is a class or function template)~~, including its non-dependent base classes ([temp.dep.type], [class.member.lookup]), is searched before its template parameter scope. *— end note*] [*Example*:

```
struct B { };
namespace N {
  typedef void V;
  template<class T> struct A : B {
    struct B { /* ... */ };
    typedef void C;
    void f();
    template<class U> void g(U);
  };
}

template<class BV> void N::A<BV>::f() {  // N::V not considered here
  BV bv;              // A's B, notV is still the template parameter, not N::V
}

template<class B> template<class C> void N::A<B>::g(C) {
  B b;              // A's Bthe base class, not the template parameter
  C c;              // the template parameter C, not A's C
}
```

— end example]

Remove paragraphs 8 and 9 (whose examples are merged into the above):

- In the definition of a member of a class template that appears outside of the namespace containing the class template definition, […]

- In the definition of a class template or in the definition of a member of such a template that appears outside of the template definition, […]

#[temp.dep]

Change paragraph 2:

- ~~In~~ A *dependent call* is an expression, possibly formed as a non-member candidate for an operator ([over.match.oper]), of the form:

  postfix-expression (*expression-list*$_{opt}$)

  where the *postfix-expression* is an *unqualified-id*~~, the *unqualified-id* denotes a *dependent name* if~~ and

  1. any of the expressions in the *expression-list* is a pack expansion ([temp.variadic]),
  2. any of the expressions or *braced-init-lists* in the *expression-list* is type-dependent ([temp.dep.expr]), or
  3. the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameter.

  ~~If an operand of an operator is a type-dependent expression, the operator also denotes a dependent name.~~ The component name of an *unqualified-id* is dependent if

  1. it is a *conversion-function-id* whose *conversion-type-id* is dependent, or
  2. it is `operator=` and the current class is a templated entity, or
  3. the *unqualified-id* is the *postfix-expression* in a dependent call.

  [*Note*: Such names ~~are unbound and~~ are looked up only at the point of the template instantiation ([temp.point]) in both the context of the template definition and the context of the point of instantiation ([temp.dep.candidate]). *— end note*]

Remove paragraph 4 (whose second example is moved to [temp.res]/1):

- In the definition of a class or class template, the scope of a dependent base class ([temp.dep.type]) is not examined […]

#[temp.dep.type]

Change paragraph 1:

- A name or *template-id* refers to the *current instantiation* if it is

  1. in the definition of a class template, a nested class of a class template, a member of a class template, or a member of a nested class of a class template, the injected-class-name ([class.pre]) of the class template or nested class,
  2. in the definition of a primary class template or a member of a primary class template, the name of the class template followed by the template argument list of ~~the primary template~~ its *template-head* (~~as described below~~[temp.arg]) enclosed in <> (or an equivalent template alias specialization),
  3. in the definition of a nested class of a class template, the name of the nested class referenced as a member of the current instantiation, or
  4. in the definition of a partial specialization or a member of a partial specialization, the name of the class template followed by ~~the~~a template argument list equivalent to that of the partial specialization ([temp.class.spec]) enclosed in <> (or an equivalent template alias

specialization). ~~If the *n*th template parameter is a template parameter pack, the *n*th template argument is a pack expansion ([temp.variadic]) whose pattern is the name of the template parameter pack.~~

Remove paragraph 2 (adapted in [temp.arg]):

~~- The template argument list of a primary template is a template argument list […]~~

Change paragraph 5:

- A qualified ([basic.lookup.qual]) or unqualified name is a *member of the current instantiation* if ~~it is~~

  1. its lookup context, if it is a qualified name, is the current instantiation, and
  2. ~~An unqualified name that, when looked up, refers to at least one~~ lookup for it finds any member of a class that is the current instantiation ~~or a non-dependent base class thereof~~. ~~[*Note*: This can only occur when looking up a name in a scope enclosed by the definition of a class template. — *end note*]~~
  3. ~~A *qualified-id* in which the *nested-name-specifier* refers to the current instantiation and that, when looked up, refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [*Note*: If no such member is found, and the current instantiation has any dependent base classes, then the *qualified-id* is a member of an unknown specialization; see below. — *end note*]~~
  4. ~~An *id-expression* denoting the member in a class member access expression ([expr.ref]) for which the type of the object expression is the current instantiation, and the *id-expression*, when looked up ([basic.lookup.classref]), refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [*Note*: If no such member is found, and the current instantiation has any dependent base classes, then the *id-expression* is a member of an unknown specialization; see below. — *end note*]~~

  [*Example*:

  […]

  — *end example*]

  A qualified or unqualified name ~~is~~ names a *dependent member of the current instantiation* if it is a member of the current instantiation that, when looked up, refers to at least one member declaration (including a *using-declarator* whose terminal name is dependent) of a class that is the current instantiation.

Change paragraph 6:

- A qualified name ([basic.lookup.qual]) is ~~*a member of an unknown specialization*~~ dependent if it is

  1. it is a *conversion-function-id* whose *conversion-type-id* is dependent, or
  2. ~~A *qualified-id* in which the *nested-name-specifier* names~~ its lookup context is ~~a~~ dependent ~~type that~~ and is not the current instantiation~~,~~, or
  3. its lookup context is the current instantiation and it is `operator=`[*Footnote*: Every instantiation of a class template declares a different set of assignment operators. — *end footnote*], or
  4. ~~A *qualified-id* in which the *nested-name-specifier* refers to~~ its lookup context is the current instantiation~~, the current instantiation~~ and has at least one dependent base class, and qualified name lookup ~~of~~ for the ~~*qualified-id*~~ name ~~does not~~ finds nothing ([basic.lookup.qual]) ~~any member of a class that is the current instantiation or a non-dependent base class thereof~~.
  5. ~~[…]~~ [*Example*:

```
struct A {
  using B=int;
  A f();
};
struct C : A {};
template<class T>
void g(T t) {
  decltype(t.A::f())::B i;  // error: typename needed to interpret B as a type
}
template void g(C);         // ...even though A is ::A here
```

  — *end example*]

Remove paragraph 7 (part of whose example is added to that in [temp.res]/8):

~~- If a *qualified-id* in which the *nested-name-specifier* refers to the current instantiation is not a member of the current instantiation or a member of an unknown specialization, […]~~

Change paragraph 8:

- If, for a given set of template arguments, a specialization of a template is instantiated that refers to a member of the current instantiation with a ~~*qualified-id* or class member access expression~~ qualified name, the name ~~in the *qualified-id* or class member access expression~~ is looked up in the template instantiation context. If the result of this lookup differs from the result of name lookup in the template definition context, name lookup is ambiguous. [*Example*:

  […]

  — *end example*]

Change paragraph 9:

- A type is dependent if it is

1. a template parameter,
2. denoted by a ~~member of an unknown specialization~~dependent (qualified) name,
3. a nested class or enumeration that is a ~~dependent~~direct member of a class that is the current instantiation,
4. a cv-qualified type where the cv-unqualified type is dependent,
5. a compound type constructed from any dependent type,
6. an array type whose element type is dependent or whose bound (if any) is value-dependent,
7. a function type whose exception specification is value-dependent,
8. denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion [*Note*: This includes an injected-class-name ([class.pre]) of a class template used without a *template-argument-list*. — *end note*], or
9. denoted by `decltype(`*expression*`)`, where *expression* is type-dependent ([temp.dep.expr]).

#[temp.dep.expr]

Change paragraph 2:

- `this` is type-dependent if the current class ~~type of the enclosing member function~~ ([expr.prim.this]) is dependent ([temp.dep.type]).

Change paragraph 3:

- An *id-expression* is type-dependent if it is a *template-id* that is not a concept-id and ~~it contains~~is dependent; or if its terminal name is

  1. ~~an *identifier*~~ associated by name lookup with one or more declarations declared with a dependent type,
  2. ~~an *identifier*~~ associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type ([dcl.spec.auto]),
  3. ~~an *identifier*~~ associated by name lookup with a variable declared with a type that contains a placeholder type ([dcl.spec.auto]) where the initializer is type-dependent,
  4. ~~an *identifier*~~ associated by name lookup with one or more declarations of member functions of a class that is the current instantiation declared with a return type that contains a placeholder type,
  5. ~~an *identifier*~~ associated by name lookup with a structured binding declaration ([dcl.struct.bind]) whose *brace-or-equal-initializer* is type-dependent,
  6. the *identifier* `__func__` ([dcl.fct.def.general]), where any enclosing function is a template, a member of a class template, or a generic lambda,
  7. ~~a *template-id* that is dependent,~~
  8. a *conversion-function-id* that specifies a dependent type, or
  9. ~~a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization~~dependent;

  or if it names a dependent member of the current instantiation that is a static data member of type "array of unknown bound of `T`" for some `T` ([temp.static]). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

  […]

Change paragraph 5:

- A class member access expression ([expr.ref]) is type-dependent if the terminal name of its *id-expression*, if any, is dependent or the expression refers to a member of the current instantiation and the type of the referenced member is dependent~~, or the class member access expression refers to a member of an unknown specialization~~. [*Note*: In an expression of the form `x.y` or `xp->y` the type of the expression is usually the type of the member `y` of the class of `x` (or the class pointed to by `xp`). However, if `x` or `xp` refers to a dependent type that is not the current instantiation, the type of `y` is always dependent. ~~If `x` or `xp` refers to a non-dependent type or refers to the current instantiation, the type of `y` is the type of the class member access expression.~~ — *end note*]

#[temp.dep.temp]

Change paragraph 4:

- A template *template-argument* is dependent if it names a *template-parameter* or its terminal name is ~~a *qualified-id* that refers to a member of an unknown specialization~~dependent.

#[temp.nondep]

Remove subclause, referring the stable name to [temp.res]. Remove the cross reference in [basic.def.odr]/14. Update the one surviving cross reference (in [basic.link]/13.3) to refer to [temp.res].

#[temp.dep.candidate]

Change paragraph 1:

- ~~For a function call where the *postfix-expression* is a dependent name, the candidate functions are found using the usual lookup rules from the template definition context ([basic.lookup.unqual], [basic.lookup.argdep]). [*Note*: For the part of the lookup using associated namespaces ([basic.lookup.argdep]), function declarations found in the template instantiation context are found by this lookup, as described in [basic.lookup.argdep]. — *end note*]~~ If ~~the~~a dependent call ([temp.dep]) would be ill-formed or would find a better match had the lookup ~~within the associated namespaces~~ for its dependent name considered all the function declarations with external linkage introduced in ~~those~~the associated namespaces in all translation units, not just considering those declarations found in the template definition and template instantiation contexts ([basic.lookup.argdep]), then the program ~~has undefined behavior~~is ill-formed, no diagnostic required.

#[temp.inject]

Remove subclause, referring the stable name (which has no surviving cross references) to [temp.friend]. Paragraph 1 is adapted in [temp.friend].

#[temp.spec]

Change paragraph 3:

- An explicit specialization may be declared for a function template, a variable template, a class template, a member of a class template, or a member template. An explicit specialization declaration is introduced by `template<>`. In an explicit specialization declaration for a variable template, a class template, a member of a class template, or a class member template, ~~the name of~~ the variable or class that is explicitly specialized shall be specified with a *simple-template-id*. In the explicit specialization declaration for a function template or a member function template, ~~the name of~~ the function or member function explicitly specialized may be specified using a *template-id*. [*Example*:

  […]

  — *end example*]

Change paragraph 5:

- For a given template and a given set of *template-argument*s, 1. an explicit instantiation definition shall appear at most once in a program, 1. an explicit specialization shall be defined at most once in a program, as specified in [basic.def.odr], and 1. both an explicit instantiation and a declaration of an explicit specialization shall not appear in a program unless the explicit ~~instantiation follows a declaration of~~specialization is reachable from the explicit ~~specialization~~instantiation. An implementation is not required to diagnose a violation of this rule if neither declaration is reachable from the other.

#[temp.inst]

Change paragraph 2:

- […] [*Example*:

  […]

  — *end example*] If ~~a class~~the template selected for the specialization ([temp.class.spec.match]) has been declared, but not defined, at the point of instantiation ([temp.point]), the instantiation yields an incomplete class type ([basic.types]). [*Example*:

  […]

  — *end example*] […]

Change paragraph 4:

- Unless a member of a templated class ~~template or a member template~~ is a declared specialization,

Change paragraph 11:

- An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, or a member class, ~~a~~or static data member of a templated class ~~template~~, or a substatement of a constexpr if statement ([stmt.if]), unless such instantiation is required. […]

Remove paragraph 12:

- Implicitly instantiated class, function, and variable template specializations are placed in the namespace where the template is defined. […]

#[temp.explicit]

Change paragraph 4:

- If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id*; otherwise, the *declaration* shall be a *simple-declaration* whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*. ~~If the explicit instantiation is for a function or member function, the *unqualified-id* in the *declarator* shall be either a *template-id* or, where all template arguments can be deduced, a *template-name* or *operator-function-id*. [*Note*: The declaration may declare a *qualified-id*, in which case the *unqualified-id* of the *qualified-id* must be a *template-id*. — *end note*] If the explicit instantiation is for a member function, a member class or a static data member of a class template specialization, the name of the class template specialization in the *qualified-id* for the member name shall be a *simple-template-id*.~~ If the explicit instantiation is for a variable template specialization, the *unqualified-id* in the *declarator* shall be a *simple-template-id*. ~~An explicit instantiation shall appear in an enclosing namespace of its template. If the name declared in the explicit instantiation is an unqualified name, the explicit instantiation shall appear in the namespace where its template is declared or, if that namespace is inline ([namespace.def]), any namespace from its enclosing namespace set. [*Note*: Regarding qualified names in declarators, see [dcl.meaning]. — *end note*]~~ [*Example*:

  […]

  — *end example*]

Change paragraph 5:

- An explicit instantiation does not introduce a name ([basic.scope.scope]). A declaration of a function template, a variable template, a member function or static data member of a class template, or a member function template of a class or class template shall ~~precede~~be reachable from any explicit instantiation of that entity. A definition of a class template, a member class of a class template, or a member class template of a class or class template shall ~~precede~~be reachable from any explicit instantiation of that entity unless ~~the explicit instantiation is preceded by~~ an explicit specialization of the entity with the same template arguments is reachable therefrom. If the *declaration* of the explicit instantiation names an implicitly-declared special member function ([special]), the program is ill-formed.

Remove paragraph 8:

- An explicit instantiation of a class, function template, or variable template specialization is placed in the namespace in which the template is defined. […]

Change paragraph 9:

- A trailing *template-argument* can be left unspecified in an explicit instantiation of a function template specialization or of a member function template specialization provided it can be deduced ~~from the type of a function parameter~~ ([temp.deduct.decl]). If all template arguments can be deduced, the empty template argument list <> may be omitted. [*Example*:

  […]

  — *end example*]

Change paragraph 11:

- An explicit instantiation that names a class template specialization is also an explicit instantiation of the same kind (declaration or definition) of each of its direct non-template members ~~(not including members inherited from base classes and members that are templates)~~ that has not been previously explicitly specialized in the translation unit containing the explicit instantiation, provided that the associated constraints, if any, of that member are satisfied by the template arguments of the explicit instantiation ([temp.constr.decl], [temp.constr.constr]), except as described below. [*Note*: In addition, it will typically be an explicit instantiation of certain implementation-dependent data about the class. — *end note*]

Change paragraph 13:

- An explicit instantiation of a prospective destructor ([class.dtor]) shall ~~name~~correspond to the selected destructor of the class.

#[temp.expl.spec]

Change paragraph 4:

- An explicit specialization does not introduce a name ([basic.scope.scope]). A declaration of a function template, class template, or variable template being explicitly specialized shall ~~precede~~be reachable from the declaration of the explicit specialization. [*Note*: A declaration, but not a definition of the template is required. — *end note*] The definition of a class or class template shall ~~precede~~be reachable from the declaration of an explicit specialization for a member template of the class or class template. [*Example*:

  […]

  — *end example*]

Change paragraph 5:

- A member function, a member function template, a member class, a member enumeration, a member class template, a static data member, or a static data member template of a class template may be explicitly specialized for a class specialization that is implicitly instantiated; in this case, the definition of the class template shall ~~precede~~be reachable from the explicit specialization for the member of the class template. If such an explicit specialization for the member of a class template names an implicitly-declared special member function ([special]), the program is ill-formed.

Change paragraph 6:

- A member of an explicitly specialized class is not implicitly instantiated from the member declaration of the class template; instead, the member of the class template specialization shall itself be explicitly defined if its definition is required. ~~In this case, t~~The definition of the class template explicit specialization shall be ~~in scope at the point at which~~reachable from the definition of any member ~~is defined~~of it. […]

Remove paragraph 9:

- A template explicit specialization is in the scope of the namespace in which the template was defined. [*Example*:

  […]

  — *end example*]

Change paragraph 11:

- A trailing *template-argument* can be left unspecified in the *template-id* naming an explicit function template specialization provided it can be deduced ~~from the function argument type~~ ([temp.deduct.decl]). [*Example*:

  […]

  — *end example*]

#[temp.fct.spec]

#[temp.arg.explicit]

Change paragraph 4:

- Trailing template arguments that can be deduced ([temp.deduct]) or obtained from default *template-argument*s may be omitted from the list of explicit *template-argument*s. [*Note:* A trailing template parameter pack ([temp.variadic]) not otherwise deduced will be deduced as an empty sequence of template arguments. *— end note*] If all of the template arguments can be deduced, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted. ~~In contexts where deduction is done and fails, or in contexts where deduction is not done, if a template argument list is specified and it, along with any default template arguments, identifies a single function template specialization, then the *template-id* is an lvalue for the function template specialization.~~ [*Example*:

  […]

  — end example]

#[temp.deduct]

Change example in bullet (11.4):

- [*Example*:

```
template <int I> struct X { };
template <template <class T> class> struct Z { };
template <class T> void f(typename T::Y*){}
template <class T> void g(X<T::N>*){}
template <class T> void h(Z<T::template TT>*){}
struct A {};
struct B { int Y; };
struct C {
  typedef int N;
};
struct D {
  typedef int TT;
};

int main() {
  // Deduction fails in each of these cases:
  f<A>(0);          // A does not contain a member Y
  f<B>(0);          // The Y member of B is not a type
  g<C>(0);          // The N member of C is not a non-type
  h<D>(0);          // The TT member of D is not a template
}
```

  — end example]

#[temp.deduct.funcaddr]

Change paragraph 1:

- Template arguments can be deduced from the type specified when taking the address of an overload~~ed function~~ set ([over.over]). […]

#[temp.deduct.conv]

Change paragraph 1:

- Template argument deduction is done by comparing the return type of the conversion function template (call it `P`) with the type ~~that is required as the result~~specified by the *conversion-type-id* of the ~~conversion~~*conversion-function-id* being looked up (call it `A`~~; see [dcl.init], [over.match.conv], and [over.match.ref] for the determination of that type~~) as described in [temp.deduct.type]. If the *conversion-function-id* is constructed during overload resolution ([over.match.funcs]), the following transformations apply.

Change paragraph 5:

- In general, the deduction process attempts to find template argument values that will make the deduced `A` identical to `A`. However, ~~there are four cases that allow a difference~~certain attributes of `A` may be ignored:

  1. If the original `A` is a reference type, any cv-qualifiers of `A` ~~can be more cv-qualified than the deduced `A`~~ (i.e., the type referred to by the reference).
  2. If the original `A` is a function pointer or pointer-to-member-function type, ~~`A` can be "pointer to function" even if the deduced `A` is "pointer to its~~ `noexcept` ~~function"~~.
  3. ~~If the original `A` is a pointer-to-member-function type, `A` can be "pointer to member of type function" even if the deduced `A` is "pointer to member of type `noexcept` function".~~
  4. ~~The deduced~~Any cv-qualifiers in `A` ~~can be another pointer or pointer-to-member type~~ that can be ~~converted to `A` via~~restored by a qualification conversion.

Change paragraph 6:

- These ~~alternatives~~attributes are ~~considered~~ignored only if type deduction would otherwise fail. If ~~they yield~~ignoring them allows more than one possible deduced `A`, the type deduction fails.

#[temp.over]

Change paragraph 1:

- When a call ~~to the name~~ of a function or function template is written (explicitly, or implicitly using the operator notation), template argument deduction ([temp.deduct]) and checking of any explicit template arguments ([temp.arg]) are performed for each function template to find the template argument values (if any) that can be used with that function template to instantiate a function template specialization that can be invoked with the call arguments or, for conversion function templates, that can convert to the required type. For each function template, if the argument deduction and checking succeeds, the *template-argument*s (deduced and/or explicit) are used to synthesize the declaration of a single function template specialization which is added to the candidate functions set to be used in overload resolution. If, for a given function template, argument deduction fails or the synthesized function template specialization would be ill-formed, no such function is added to the set of candidate functions for that template. The complete set of candidate functions includes all the synthesized declarations and all of the non-template ~~overloaded~~ functions ~~of the same name~~found by name lookup. The synthesized declarations are treated like any other functions in the remainder of overload resolution, except as explicitly noted in [over.match.best].[…]

#[except]

#[except.handle]

Remove paragraph 11:

- The scope and lifetime of the parameters of a function or constructor extend into the handlers of a *function-try-block*.

Change paragraph 12:

- Exceptions thrown in destructors of objects with static storage duration or in constructors of ~~namespace-scope~~ objects associated with non-block variables with static storage duration are not caught by a *function-try-block* on the `main` function ([basic.start.main]). Exceptions thrown in destructors of objects with thread storage duration or in constructors of ~~namespace-scope~~ objects associated with non-block variables with thread storage duration are not caught by a *function-try-block* on the initial function of the thread.

#[except.spec]

Change bullet (13.1):

- in an expression, the function is ~~the unique lookup result or the~~ selected ~~member of a set of~~by overload~~ed functions~~ resolution (~~[basic.lookup],~~ [over.match], [over.over]);

#[except.terminate]

Change bullet (1.5):

- when initialization of a non-~~local~~block variable with static or thread storage duration ([basic.start.dynamic]) exits via an exception, or

#[cpp.replace]

Change paragraph 12:

- […] The parameters are specified by the optional list of identifiers~~, whose scope extends from their declaration in the identifier list until the new-line character that terminates the `#define` preprocessing directive~~. […]

#[concept.booleantestable]

Change paragraph 2:

- Let `e` be an expression such that `decltype((e))` is `T`. `T` models *`boolean-testable-impl`* only if:
  1. either `remove_cvref_t<T>` is not a class type, or ~~name lookup~~a search for the names `operator&&` and `operator||` ~~with~~in the scope of `remove_cvref_t<T>` ~~as if by class member access lookup ([class.member.lookup]) results in an empty declaration set~~finds nothing; and
  2. ~~name~~argument-dependent lookup ~~([basic.lookup.argdep])~~ for the names `operator&&` and `operator||` ~~in the associated namespaces and entities of~~with `T` ~~([basic.lookup.argdep])~~ as the only argument type finds no disqualifying declaration (defined below).

Change bullet (5.2.3):

- the declaration declares a function template ~~that is not visible in its namespace~~to which no name is bound ([~~namespace.memdef~~dcl.meaning]).

#[diff.basic]

Change paragraph 2 (the example is based on that from the removed [class.nested.type]):

- **Affected subclause:** [basic.scope]
  Change: A `struct` is a scope in C++, not in C.
  For example,

```
struct X {
  struct Y { int a; } b;
};
struct Y c;
```

  is valid in C but not in C++, which would require `X::Y c;`.
  Rationale: Class scope is crucial to C++, and a struct is a class.
  Effect on original feature: Change to semantics of well-defined feature.
  Difficulty of converting: Semantic transformation.
  How widely used: C programs use `struct` extremely frequently, but the change is only noticeable when `struct`, enumeration, or enumerator names are referred to outside the `struct`. The latter is probably rare.

#[depr.template.template]

Add this subclause before [depr.c.headers]:

- The use of the keyword `template` before the qualified name of a class or alias template without a template argument list is deprecated.

Acknowledgments

Thanks to Richard Smith for helping discover and interpret several of the issues addressed and for feedback on their resolutions, and to Rachel Ertl for helping decide on them.